

Лабораторна робота №1. Використання фреймворку FastAPI та SQLAlchemy ORM для створення вебзастосунку мовою Python

Завдання. Для заданої предметної галузі розробити RESTfull API вебзастосунок

- 1) Контент повинен генеруватися на сервері і у вигляді HTML відправляється на клієнт.
- 2) Дані мусять зберігатися у реляційній БД, наприклад, SQLite. Схема даних повинна містити щонайменша 3 сутності.
- 3) Для доступу до даних слід використовувати SQLAlchemy ORM.
- 4) Передбачити режими роботи акторів двох ролей: адміністратор та користувач.
- 5) Хоча б для однієї сутності реалізувати усі чотири CRUD-операції (create, read, update, delete).
- 6) Внести зміни в OpenAPI документацію за замовченням.

ЗМІСТ

1 Основи FastAPI	4
1.1 Що таке FastAPI	4
Необхідні інструменти та встановлення	4
Створення першого додатка	6
Запуск програми.....	8
1.2 Клас FastAPI та обробка запиту	9
1.3 Надсилання відповіді	13
Response.....	14
PlainTextResponse.....	16
HTMLResponse	16
Встановлення типу відповіді через методи FastAPI.....	17
1.4 Надсилання файлів із сервера	18
1.5 Параметри шляху.....	20
Черговість визначення шляхів	23
Обмеження типу параметрів	24
Path.....	28
1.6 Параметри рядка запиту	31
Значення за замовчуванням	32
Обмеження за типом.....	33
Query	33
Отримання списків значень	36
Поєднання параметрів шляху та рядка запиту.....	37
1.7 Надсилання статусних кодів	38
Надсилання статусних кодів у FastAPI і Python.....	40
Визначення статусного коду у відповіді.....	42
Зміна статусного коду	43
1.8 Переадресація.....	44
1.9 Статичні файли	46
Встановлення головної сторінки.....	47
1.10 Отримання даних запиту.....	49
Body	49
Отримання окремих значень.....	52
Валідація	53
1.11 Отримання даних запиту у вигляді об'єкта класу	54
Необов'язкові атрибути	56
Детальне налаштування атрибутів і клас Field	57
Отримання списків.....	59
Отримання вкладених списків	60
Вкладені моделі	61
1.12 Створення найпростішого API.....	62
Створення сервера.....	62
Визначення клієнта	67
1.13 Надсилання та отримання заголовків.....	76
Надсилання заголовків	76
Отримання заголовків	79
1.14 Кукі	80
Встановлення кукі	80

Отримання кукі.....	82
1.15 Надсилання форм	84
Валідація	86
Значення за замовчуванням	88
Надсилання списків	88
2 Робота з базою даних через SQLAlchemy	91
2.1 Підключення до бази даних і створення таблиць	91
Визначення рядка підключення до БД	91
Визначення рушія.....	92
Визначення моделей	93
Створення бази даних і таблиць	95
2.2 Взаємодія з базою даних	97
Створення сесії бази даних	97
CRUD	98
2.3 Базові операції з базою даних у веб-додатку	106
Визначення підключення до бази даних і моделей	107
Створення сервера.....	107
Визначення клієнта	112
2.4 Запуск вебзастосунку.....	119

1 Основи FastAPI

1.1 Що таке FastAPI

FastAPI представляє швидкий високопродуктивний фреймворк для створення веб-додатків мовою Python.

Офіційний сайт проекту: <https://fastapi.tiangolo.com/>. Вихідний код фреймворку доступний на github за адресою: <https://github.com/tiangolo/fastapi>

Наразі підтримується Python версії 2.8 і вище.

Необхідні інструменти та встановлення

Для роботи з FastAPI природно потрібен інтерпретатор Python.

Для встановлення пакетів FastAPI потрібен пакетний менеджер *pip*. Менеджер *pip* дає змогу завантажувати пакети і керувати ними. Зазвичай під час встановлення python також встановлюється і менеджер *pip*.

Для встановлення пакетів FastAPI відкриємо термінал і введемо команду:

```
pip install fastapi
```

```

C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.22621.3097]
(c) Корпорація Майкрософт. Усі права захищені.

C:\Users\barba>pip install fastapi
Collecting fastapi
  Downloading fastapi-0.109.0-py3-none-any.whl.metadata (24 kB)
Collecting pydantic[1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4] (from fastapi)
  Downloading pydantic-2.5.3-py3-none-any.whl.metadata (65 kB)
    65.6/65.6 kB 252.8 kB/s eta 0:00:00
Collecting starlette<0.36.0,>=0.35.0 (from fastapi)
  Downloading starlette-0.35.1-py3-none-any.whl.metadata (5.8 kB)
Collecting typing_extensions<=4.9.0-py3-none-any.whl.metadata (3.0 kB)
Collecting annotated-types<=0.4.0 (from pydantic[1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4]->fastapi)
  Downloading annotated_types-0.6.0-py3-none-any.whl.metadata (12 kB)
Collecting pydantic-core==2.14.6 (from pydantic[1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4]->fastapi)
  Downloading pydantic_core-2.14.6-cp312-none-win_amd64.whl.metadata (6.6 kB)
Collecting anyio<5,>=3.4.0 (from starlette<0.36.0,>=0.35.0->fastapi)
  Downloading anyio-4.2.0-py3-none-any.whl.metadata (4.6 kB)
Collecting idna<=2.8 (from anyio<5,>=3.4.0->starlette<0.36.0,>=0.35.0->fastapi)
  Downloading idna-3.6-py3-none-any.whl.metadata (9.9 kB)
Collecting sniffio<=1.1 (from anyio<5,>=3.4.0->starlette<0.36.0,>=0.35.0->fastapi)
  Downloading sniffio-1.3.0-py3-none-any.whl (10 kB)
Downloaded fastapi-0.109.0-py3-none-any.whl (92 kB)
    92.0/92.0 kB 2.6 MB/s eta 0:00:00
Downloaded pydantic-2.5.3-py3-none-any.whl (381 kB)
    381.9/381.9 kB 1.7 MB/s eta 0:00:00
Downloaded pydantic_core-2.14.6-cp312-none-win_amd64.whl (1.9 MB)
    1.9/1.9 MB 873.5 kB/s eta 0:00:00
Downloaded starlette-0.35.1-py3-none-any.whl (71 kB)
    71.1/71.1 kB 779.9 kB/s eta 0:00:00
Downloaded typing_extensions-4.9.0-py3-none-any.whl (32 kB)
Downloaded annotated_types-0.6.0-py3-none-any.whl (12 kB)
Downloaded anyio-4.2.0-py3-none-any.whl (85 kB)
    85.5/85.5 kB 482.4 kB/s eta 0:00:00
Downloaded idna-3.6-py3-none-any.whl (61 kB)
    61.6/61.6 kB 658.0 kB/s eta 0:00:00
Installing collected packages: typing_extensions, sniffio, idna, annotated-types, pydantic-core, anyio, starlette, pydantic, fastapi
Successfully installed annotated-types-0.6.0 anyio-4.2.0 fastapi-0.109.0 idna-3.6 pydantic-2.5.3 pydantic-core-2.14.6 sniffio-1.3.0 starlette-0.35.1 typing_extensions-4.9.0

[notice] A new release of pip is available: 23.3.1 -> 23.3.2
[notice] To update, run: python.exe -m pip install --upgrade pip

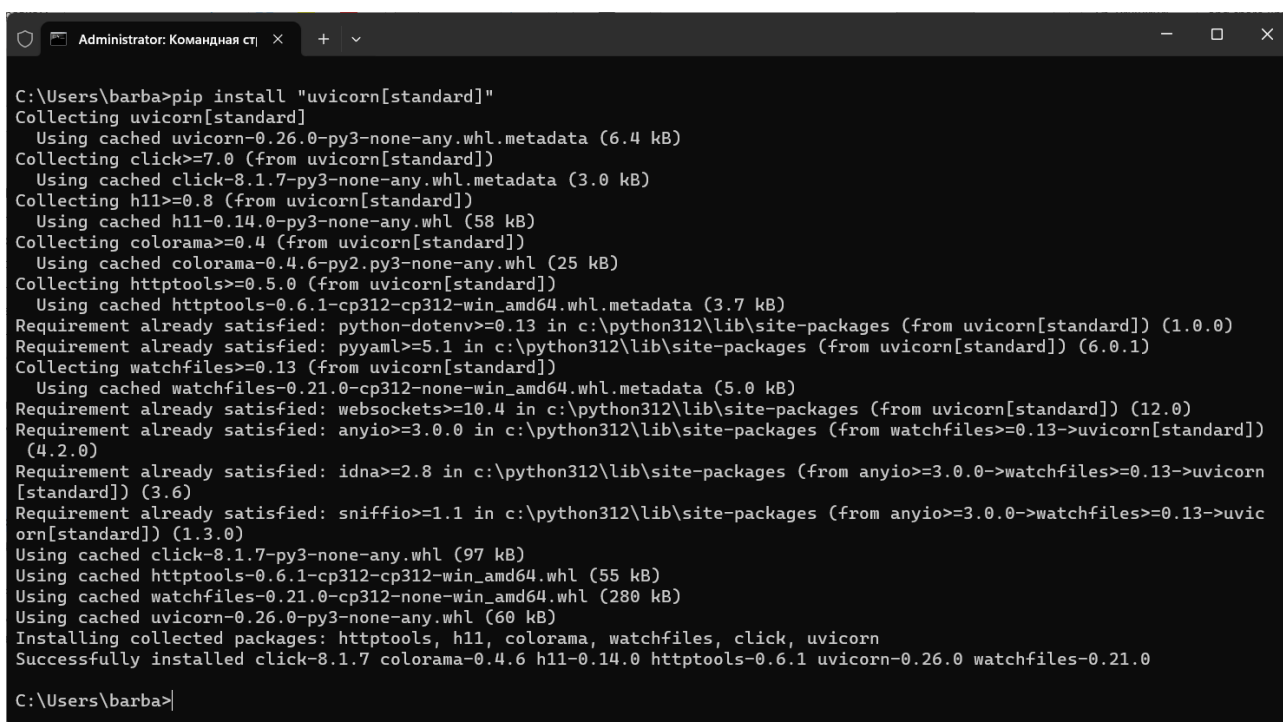
C:\Users\barba>

```

Рисунок 1.1 Результат встановлення пакету FastAPI

Також для роботи з FastAPI нам знадобиться ASGI веб-сервер (вебсервер із підтримкою протоколу Asynchronous Server Gateway Interface). В якості такого в Python можна використовувати [Univer](#) або [Hypercorn](#). У цьому випадку будемо використовувати Univer. Також встановимо його пакети за допомогою менеджера *pip* за допомогою такої команди:

```
pip install "uvicorn[standard]"
```

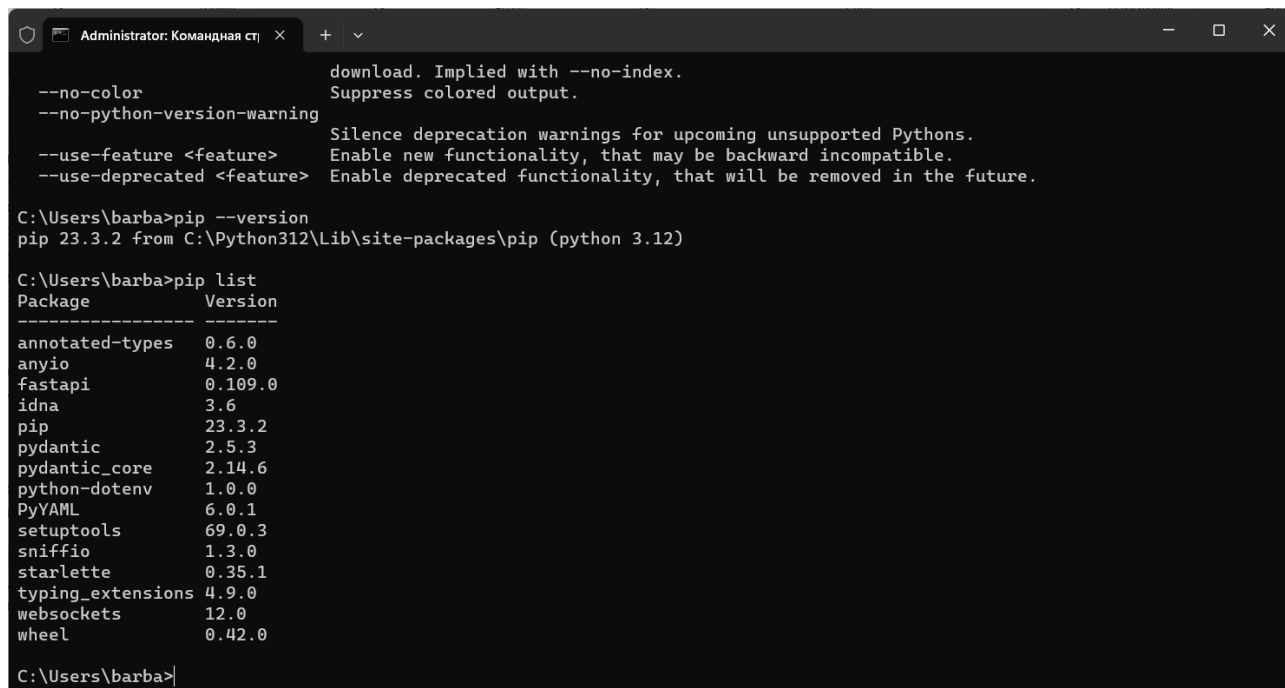


```
C:\Users\barba>pip install "uvicorn[standard]"
Collecting uvicorn[standard]
  Using cached uvicorn-0.26.0-py3-none-any.whl.metadata (6.4 kB)
Collecting click>=7.0 (from uvicorn[standard])
  Using cached click-8.1.7-py3-none-any.whl.metadata (3.0 kB)
Collecting h11>=0.8 (from uvicorn[standard])
  Using cached h11-0.14.0-py3-none-any.whl (58 kB)
Collecting colorama>=0.4 (from uvicorn[standard])
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting httptools>=0.5.0 (from uvicorn[standard])
  Using cached httptools-0.6.1-cp312-cp312-win_amd64.whl.metadata (3.7 kB)
Requirement already satisfied: python-dotenv>=0.13 in c:\python312\lib\site-packages (from uvicorn[standard]) (1.0.0)
Requirement already satisfied: pyyaml>=5.1 in c:\python312\lib\site-packages (from uvicorn[standard]) (6.0.1)
Collecting watchfiles>=0.13 (from uvicorn[standard])
  Using cached watchfiles-0.21.0-cp312-none-win_amd64.whl.metadata (5.0 kB)
Requirement already satisfied: websockets>=10.4 in c:\python312\lib\site-packages (from uvicorn[standard]) (12.0)
Requirement already satisfied: anyio>=3.0.0 in c:\python312\lib\site-packages (from watchfiles>=0.13->uvicorn[standard]) (4.2.0)
Requirement already satisfied: idna>=2.8 in c:\python312\lib\site-packages (from anyio>=3.0.0->watchfiles>=0.13->uvicorn[standard]) (3.6)
Requirement already satisfied: sniffio>=1.1 in c:\python312\lib\site-packages (from anyio>=3.0.0->watchfiles>=0.13->uvicorn[standard]) (1.3.0)
Using cached click-8.1.7-py3-none-any.whl (97 kB)
Using cached httptools-0.6.1-cp312-cp312-win_amd64.whl (55 kB)
Using cached watchfiles-0.21.0-cp312-none-win_amd64.whl (280 kB)
Using cached uvicorn-0.26.0-py3-none-any.whl (60 kB)
Installing collected packages: httptools, h11, colorama, watchfiles, click, uvicorn
Successfully installed click-8.1.7 colorama-0.4.6 h11-0.14.0 httptools-0.6.1 uvicorn-0.26.0 watchfiles-0.21.0

C:\Users\barba>
```

Рисунок 1.2 Результат встановлення пакету Univer

Якщо під час встановлення пакетів виникла помилка, то слід перевірити актуальність пакету *pip*.



```

Administrator: Командная строка
--no-color                download. Implied with --no-index.
--no-python-version-warning Suppress colored output.
--use-feature <feature>   Silence deprecation warnings for upcoming unsupported Pythons.
--use-deprecated <feature> Enable new functionality, that may be backward incompatible.
                           Enable deprecated functionality, that will be removed in the future.

C:\Users\barba>pip --version
pip 23.3.2 from C:\Python312\Lib\site-packages\pip (python 3.12)

C:\Users\barba>pip list
Package            Version
-----
annotated-types    0.6.0
anyio              4.2.0
fastapi            0.109.0
idna               3.6
pip               23.3.2
pydantic           2.5.3
pydantic_core      2.14.6
python-dotenv      1.0.0
PyYAML            6.0.1
setuptools         69.0.3
sniffio           1.3.0
starlette          0.35.1
typing_extensions  4.9.0
websockets         12.0
wheel              0.42.0

C:\Users\barba>

```

Рисунок 1.3 – Результат перевірки пакету `pip`

Про виправлення помилок роботи пакету *pip* можна прочитати <https://phoenixnap.com/kb/install-pip-windows>.

Створення першого додатка

Визначимо на диску папку, де розташовуватимуться файли з вихідним кодом програми. Наприклад, у моєму випадку це папка `C:\webpy\fastapi`. Створимо в цій папці новий файл, який назвемо *main.py* і який матиме такий код:

```

from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.get("/")
def read_root():
    html_content = "<h2>Hello WebPython!</h2>"
    return HTMLResponse(content=html_content)

```

Для обробки запитів до додатка спочатку необхідно створити об'єкт додатка за допомогою конструктора FastAPI з пакета *fastapi* (рядок 4).

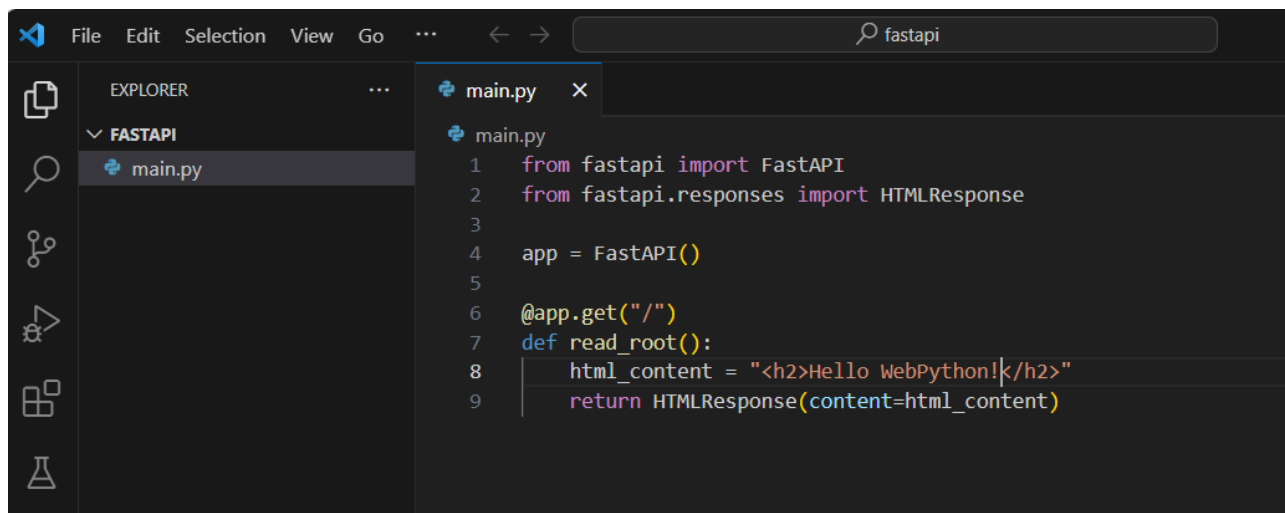


Рисунок 1.4 – Код *main.py*

Потім визначаємо функцію, яка оброблятиме запити. До цієї функції застосовується спеціальний декоратор у вигляді методу `app.get()` – рядок 6.

У цей метод передається шаблон маршруту, за яким функція оброблятиме запити. У цьому випадку це рядок `"/"`, який означає, що функція оброблятиме запити за шляхом `"/"`, тобто запити до кореня веб-додатка.

Після декоратора `app.get` йде власне визначення функції, яка обробляє запит – рядки 7-9.

Це звичайна функція python. Вона називається `read_root` (ім'я довільне). Для надсилання відповіді вона використовує клас *HTMLResponse* з пакета *fastapi.responses*. Клас *HTMLResponse* дає змогу надіслати у відповідь деякий вміст у вигляді коду html.

Для встановлення вмісту, що надсилається, у конструкторі *HTMLResponse* застосовується параметр *content*, якому в цьому випадку передається рядок `"<h2>Hello WebPython!</h2>"` зі значенням `"Hello WebPython!"`. Тобто коли клієнт звернеться до веб-додатку за шляхом `"/"`, йому буде надіслано html-код `"<h2>Hello WebPython!</h2>"`.

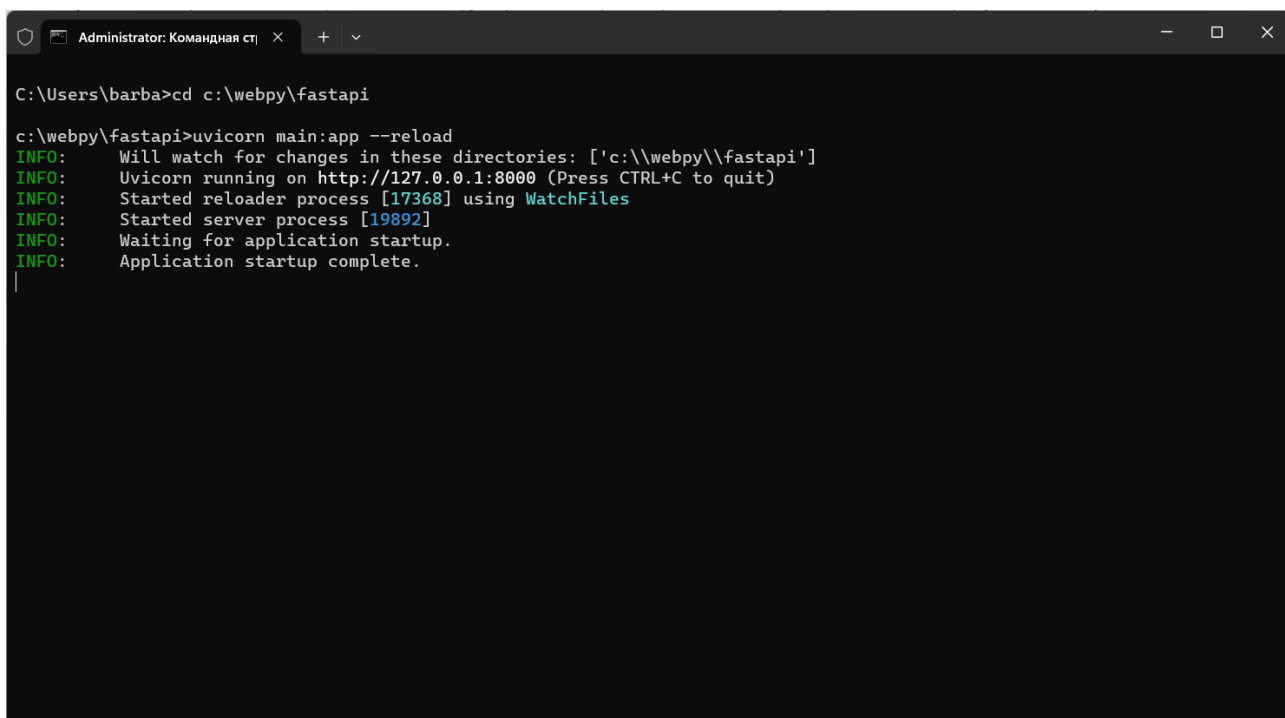
Запуск програми

Тепер запусимо додаток. Для цього перейдемо в терміналі до папки, де розміщено файл `main.py` і потім виконаємо команду

```
uvicorn main:app --reload
```

У цьому випадку ми запускаємо сервер `uvicorn` і передаємо йому низку параметрів:

- `main` вказує на назву модуля, яка за замовчуванням збігається з назвою файлу - `main`
- `app` вказує на об'єкт додатка, створений у рядку `app = FastAPI()`
- `--reload` дає змогу відстежувати зміни у файлах вихідного коду й автоматично перезапускати проєкт.



```
Administrator: Командная строка
C:\Users\barba>cd c:\webpy\fastapi

c:\webpy\fastapi>uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['c:\\webpy\\fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17368] using WatchFiles
INFO: Started server process [19892]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Рисунок 1.5 – Результат запуску сервера `uvicorn`

Під час запуску консоль відображає адресу, за якою запущено додаток. Зазвичай це адреса `http://127.0.0.1:8000`. І якщо ми звернемося за цією адресою в браузері, то він відобразить нам відправлені сервером дані:

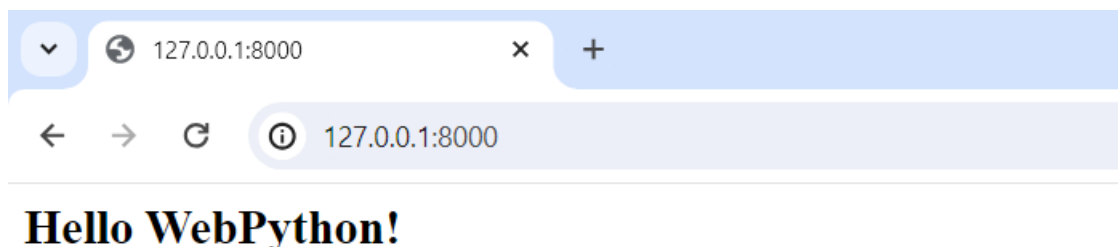


Рисунок 1.6 – Відповідь від серверу

1.2 Клас FastAPI та обробка запиту

У центрі додатка FastAPI знаходиться однойменний клас FastAPI з пакета *fastapi*. Цей клас фактично і представляє додаток *FastAPI*. Цей клас успадковується від класу *starlette.applications.Starlette*. Starlette (<https://www.starlette.io/>) представляє інший легкий ASGI-фреймворк для створення асинхронних веб-сервісів на Python. Власне *fastAPI* працює поверх *Starlette*, використовуючи і доповнюючи його функціональність. Це стосується не тільки самого класу FastAPI, а й інших класів фреймворку - багато хто з них використовує функціонал *Starlette*.

Конструктор класу *FastAPI* має близько трьох десятків різних параметрів, які дають змогу налаштувати роботу застосунку. Але в загальному випадку для створення функціонуючого об'єкта класу можна не передавати в конструктор жодних аргументів, тоді параметри отримують значення за замовчуванням:

```
from fastapi import FastAPI

app = FastAPI()
```

Однією з переваг FastAPI є те, що фреймворк дає змогу швидко та легко побудувати веб-сервіс у стилі REST. Архітектура REST передбачає застосування таких методів або типів запитів HTTP для взаємодії із сервером, де кожен тип запиту відповідає за певну дію:

- *GET* (отримання даних)
- *POST* (додавання даних)
- *PUT* (зміна даних)
- *DELETE* (видалення даних)

Крім цих типів запитів HTTP підтримує ще низку, зокрема:

- *OPTIONS*
- *HEAD*
- *PATCH*
- *TRACE*

У класі FastAPI для кожного з цих типів запитів визначено однойменні методи:

- *get()*
- *post()*
- *put()*
- *delete()*
- *options()*
- *head()*
- *patch()*
- *trace()*

Наприклад, якщо нам треба обробити HTTP-запит типу *GET*, то застосовується метод *get()*.

Усі ці методи мають безліч параметрів, але всі вони як обов'язковий параметр приймають шлях, запит за яким має оброблятися.

Причому ці методи сам запит не обробляють - вони застосовуються як декоратор до функцій, які безпосередньо обробляють запит. Наприклад:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello WebPython!"}
```

У цьому випадку метод *app.get()* застосовується як декоратор до функції *root()* (символ *@* вказує на визначення декоратора). Цей декоратор визначає шлях, запити за яким оброблятиме функція *root()*. У даному випадку шлях представляє рядок *"/"*, тобто функція оброблятиме запити до кореня веб-додатка (наприклад, за адресою *http://127.0.0.1:8000/*).

Функція повертає деякий результат. Зазвичай це словник (об'єкт *dict*). Тут словник містить один елемент *"message"*. Під час надсилання ці дані автоматично серіалізуються у формат *JSON* - популярний формат для взаємодії між клієнтом і сервером. А у відповіді для заголовка *content-type* встановлюється значення *application/json*. Взагалі функція може повертати різні дані - словники (*dict*), списки (*list*), поодинокі значення типу рядків, чисел тощо, які потім серіалізуються в *json*.

Відповідно, якщо ми запустимо додаток і звернемося за адресою *http://127.0.0.1:8000/*, наприклад, у браузері, то ми отримаємо відповідь сервера у форматі *json*:

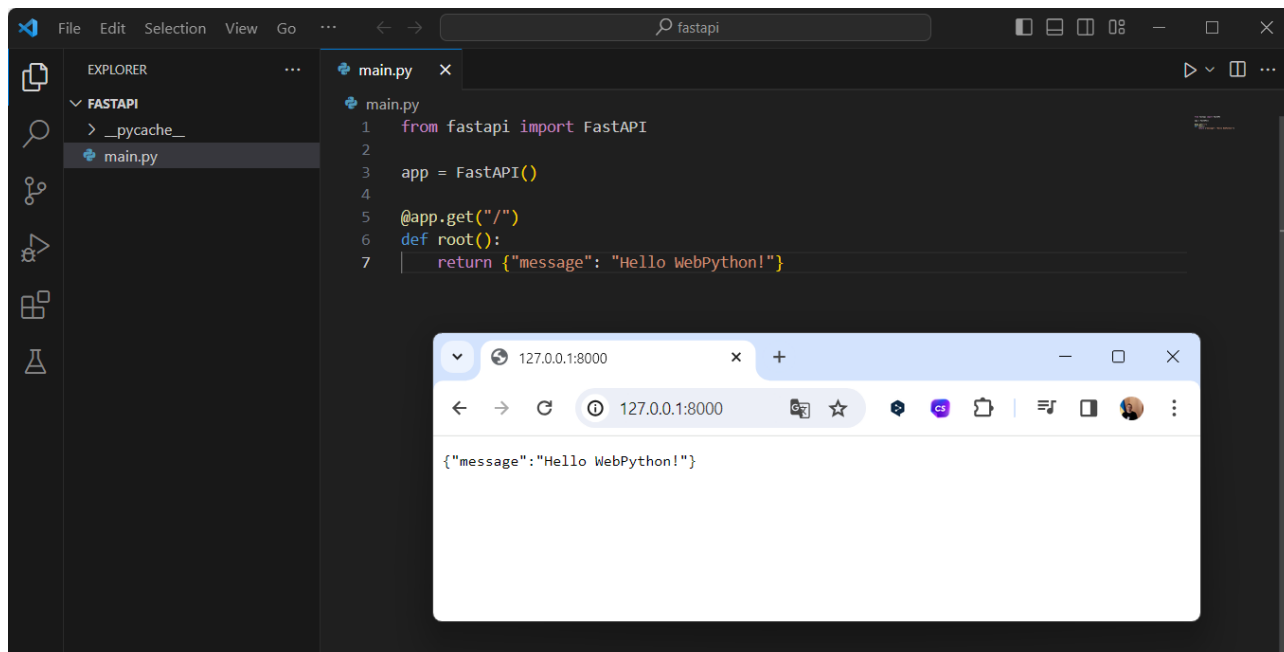


Рисунок 1.7 – Код main.py та відповідь сервера

Подібним чином можна визначати й інші функції, які оброблятимуть запити за іншими шляхами. Наприклад:

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello WebPython!"}

@app.get("/about")
def about():
    return {"message": "About"}
  
```

Тут додано функцію *about()*, яка обробляє запити за шляхом *"/about"*:

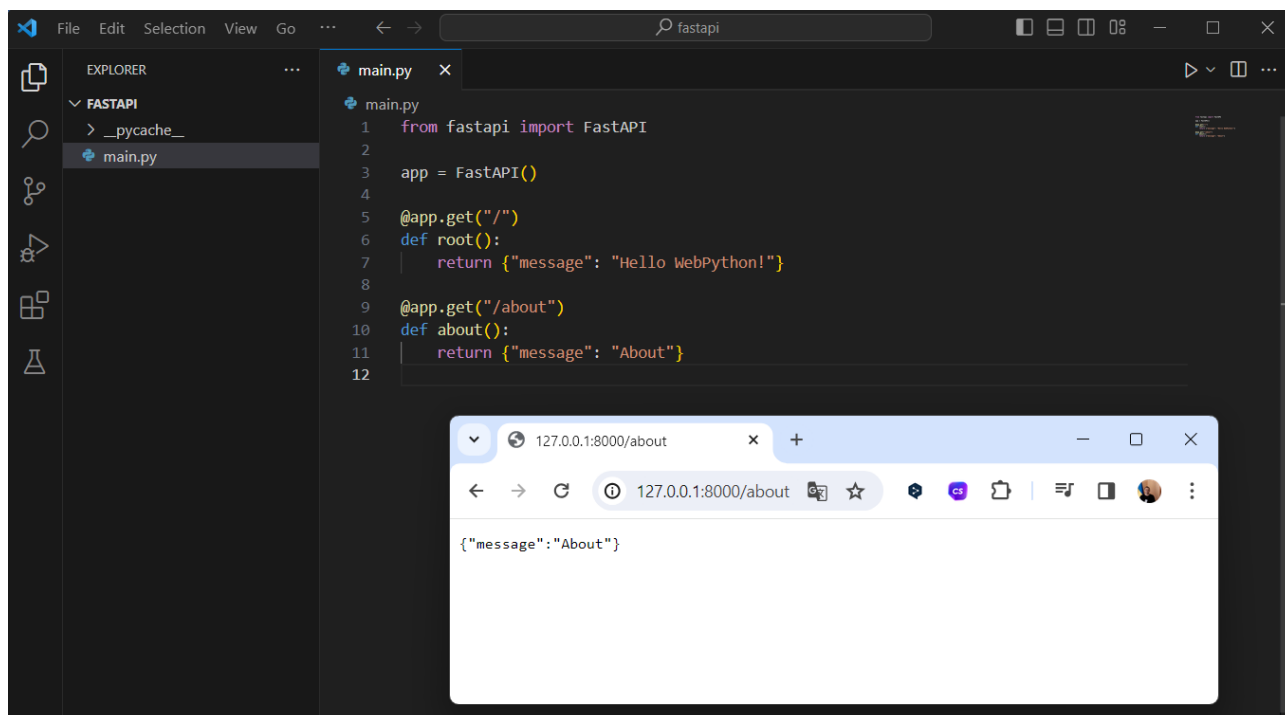


Рисунок 1.8 – Код main.py та відповідь сервера

1.3 Надсилання відповіді

У минулому пункті розглядалося оброблення запиту функціями, наприклад, найпростіший код (рис. 1.7).

Тут функція *root* обробляє запит за шляхом `/` (тобто звернення до кореня веб-додатка) і повертає деякий результат у вигляді словника (об'єкт *dict*), який містить один елемент *"message"*. Під час надсилання ці дані автоматично серіалізуються у формат *JSON* - популярний формат для взаємодії між клієнтом і сервером. А у відповіді для заголовка *content-type* встановлюється значення *application/json*. Взагалі функція може повертати різні дані - словники (*dict*), списки (*list*), поодинокі значення типу рядків, чисел тощо, які потім серіалізуються в *json* за допомогою кодувальника *fastapi.encoders.jsonable_encoder*. А для відправлення відповіді FastAPI за замовчуванням використовує клас *fastapi.responses.JSONResponse*. Тобто попередній код у принципі буде еквівалентний наступному:

```

from fastapi import FastAPI
from fastapi.responses import JSONResponse
from fastapi.encoders import jsonable_encoder

app = FastAPI()

@app.get("/")
def root():
    data = {"message": "Hello WebPython!"}
    json_data = jsonable_encoder(data)
    return JSONResponse(content=json_data)

```

Параметр *content* задає дані, що надсилаються. Або можна без явної серіалізації передати дані в *JSONResponse*:

```

from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/")
def root():
    return JSONResponse(content={"message": "Hello WebPython!"})

```

І за потреби надіслати якісь нестандартні несеріалізовані дані або, якщо нас не влаштовує серіалізація за замовчуванням, ми можемо визначити свій серіалізатор *json*.

Однак функція необов'язково повинна повертати саме дані у форматі *json*. У реальності ми можемо повернути об'єкт класу *Response* або одного з його підкласів (яким також є *JSONResponse*), що дають змогу надсилати клієнту відповідь у різних видах і формах. Розглянемо деякі з цих класів.

Response

Клас `fastapi.Response` є базовим для решти класів відповіді. Його перевагою є те, що він дає змогу також надіслати відповідь, яка не покривається вбудованими класами, наприклад, у якомусь нестандартному форматі. Для визначення відповіді конструктор класу приймає такі параметри:

- *content*: задає вміст, що відправляється
- *status_code*: задає статусний код відповіді
- *media_type*: задає MIME-тип відповіді
- *headers*: задає заголовки відповіді

Розглянемо найпростіший приклад:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/")
def root():
    data = "Hello WebPython!"
    return Response(content=data, media_type="text/plain")
```

У цьому випадку клієнту відправляє звичайний рядок "Hello WebPython!". А MIME-тип *"text/plain"* вказує, що тип відповіді - простий текст.

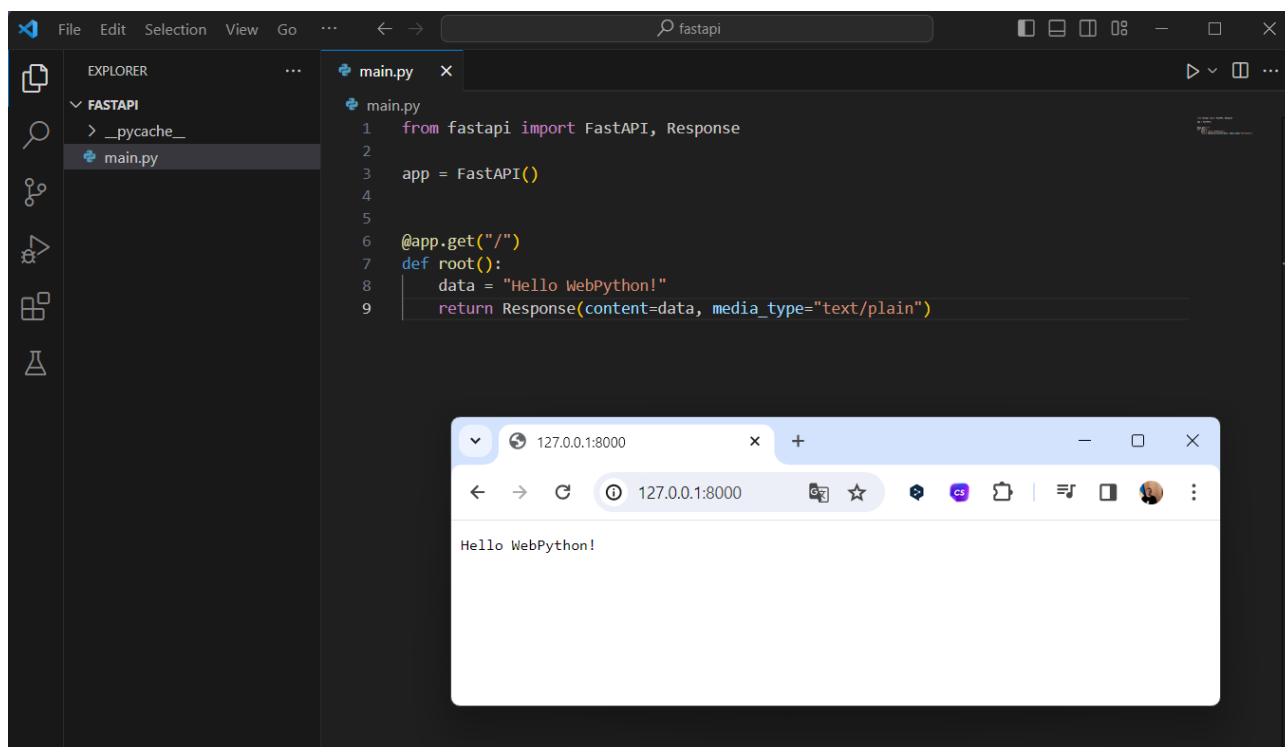


Рисунок 1.9 – Код main.py та відповідь сервера

PlainTextResponse

Для надсилання простого тексту також можна використовувати клас-спадкоємець `PlainTextResponse`:

```
from fastapi import FastAPI
from fastapi.responses import PlainTextResponse

app = FastAPI()

@app.get("/")
def root():
    data = "Hello WebPython!"
    return PlainTextResponse(content=data)
```

HTMLResponse

Для спрощення надсилання коду *html* призначений клас *HTMLResponse*. Він встановлює для заголовка *Content-Type* значення *text/html*:

```
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()

@app.get("/")
def root():
    data = "<h2>Hello WebPython!</h2>"
    return HTMLResponse(content=data)
```

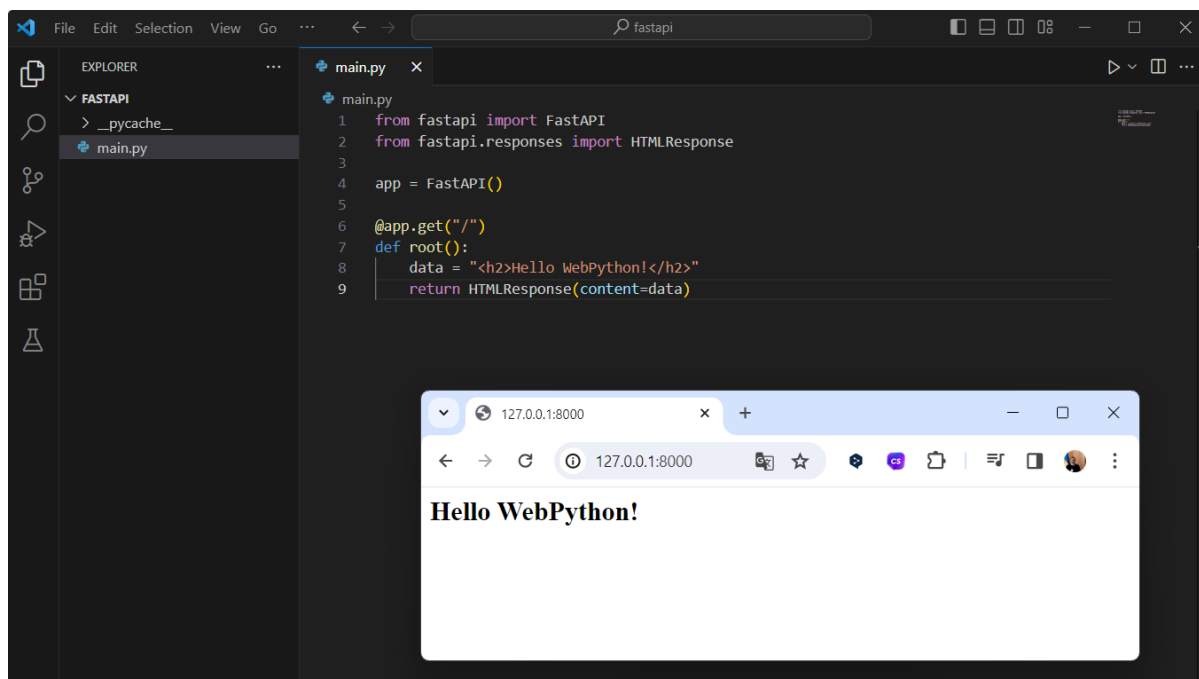



Рисунок 1.10 – Код main.py та відповідь сервера

Встановлення типу відповіді через методи FastAPI

Методи FastAPI, такі як *get()*, *post()* тощо, дають змогу задати тип відповіді за допомогою параметра *response_class*:

```

from fastapi import FastAPI
from fastapi.responses import PlainTextResponse, JSONResponse, HTMLResponse

app = FastAPI()

@app.get("/text", response_class = PlainTextResponse)
def root_text():
    return "Hello WebPython!"

@app.get("/html", response_class = HTMLResponse)
def root_html():
    return "<h2>Hello WebPython!</h2>"
  
```

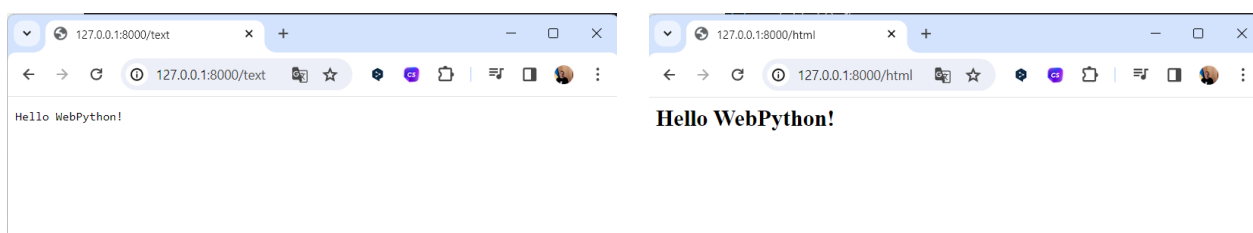


Рисунок 1.11 – Відповіді сервера

1.4 Надсилання файлів із сервера

Для надсилання файлів із програми *FastAPI* застосовується клас *FileResponse* - спадкоємець класу *Response*.

Припустімо, у нас є такий проєкт і нехай у проєкті в папці *public* є файл *index.html* з таким кодом:

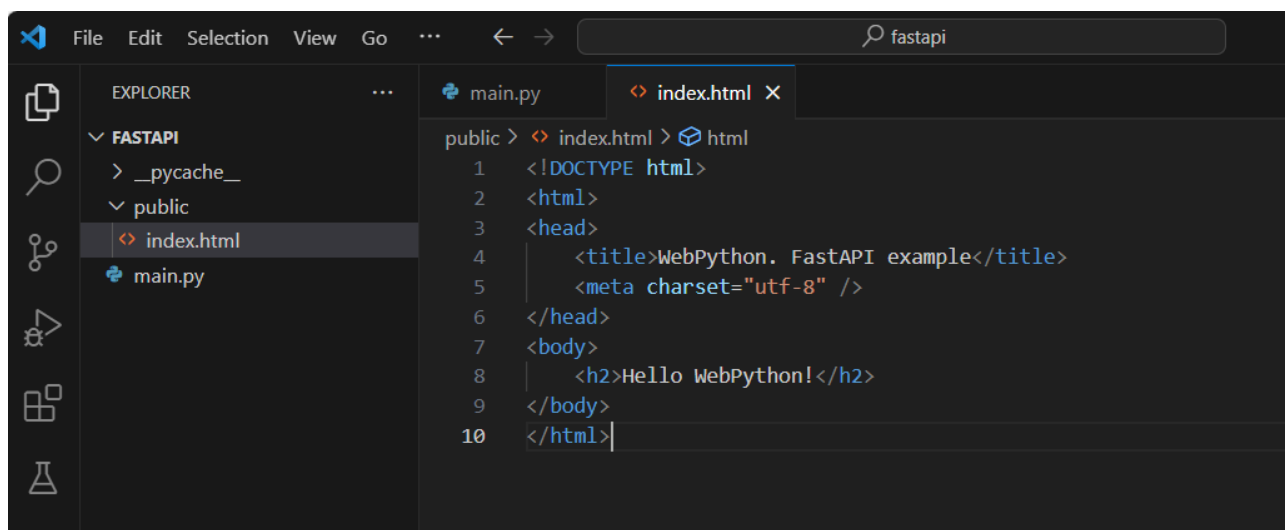


Рисунок 1.12 – Структура проєкту

У файлі *main.py* визначимо код для надсилання цього файлу клієнту:

```

from fastapi import FastAPI
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

# альтернативний варіант
@app.get("/file", response_class = FileResponse)
def root_html():
    return "public/index.html"

```

Як обов'язковий параметр конструктор *FileResponse* приймає шлях до файлу.

Під час надсилання файлу браузер зможе інтерпретувати його як деякий код *html*, і відповідно ми зможемо споглядати цю сторінку в браузері:

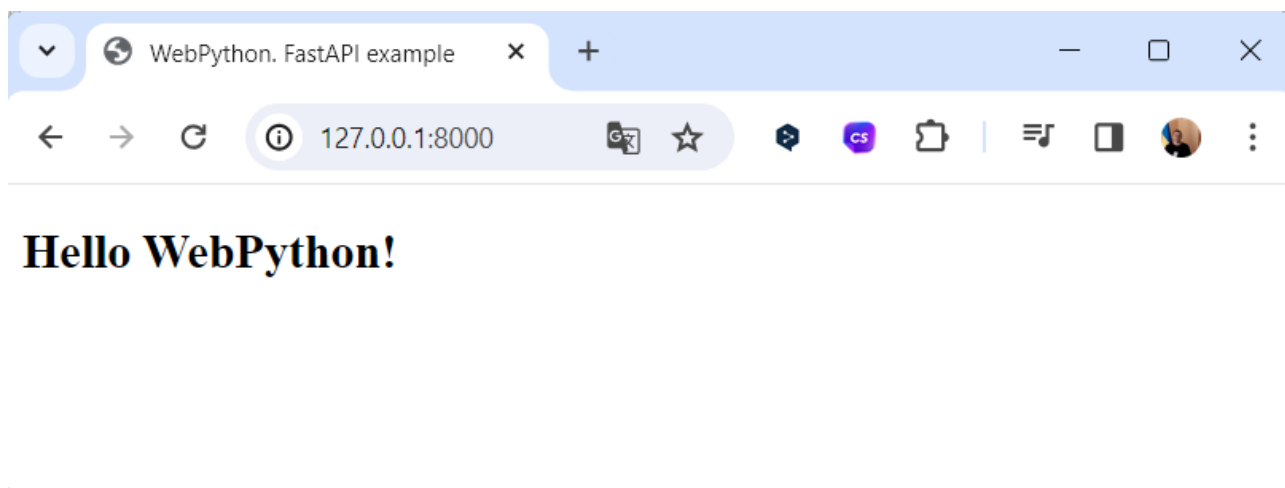


Рисунок 1.13 – Відповідь сервера

Браузер за замовчуванням намагається інтерпретувати і відобразити всі файли, які може, наприклад, текстові файли, файли зображень, якісь інші мультимедіа-файли. Якщо файл не може бути інтерпретований браузером, то він завантажується. Однак може виникнути необхідність автоматично завантажити без відображення якісь інтерпретовані файли, наприклад, ті ж самі файли *html*. У цьому разі ми можемо встановити для параметра *media_type* значення *application/octet-stream*. Крім того, за допомогою параметра *filename* для завантажуваного файлу можна задати ім'я:

```
import mimetypes
from fastapi import FastAPI
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html",
                        filename="mainpage.html",
                        media_type="application/octet-stream")
```

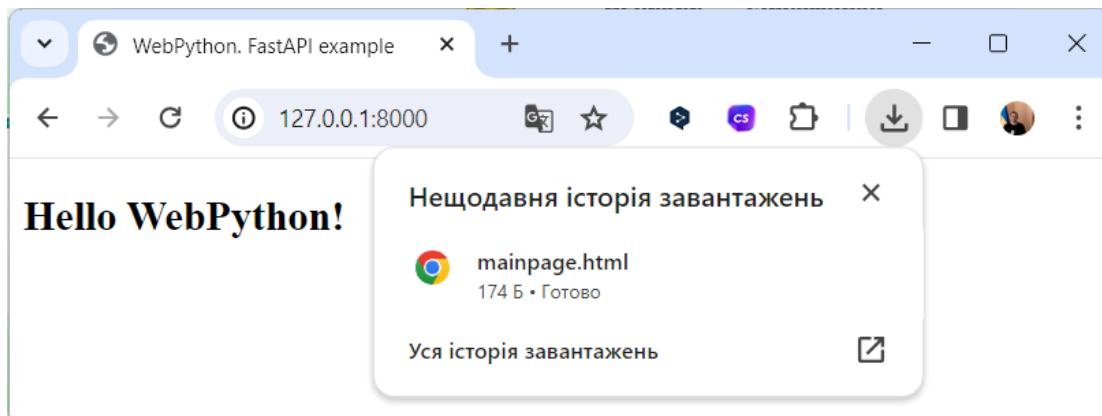


Рисунок 1.14 – Відповідь сервера на запит завантаження файлу

1.5 Параметри шляху

Шлях запиту може містити спеціальні значення, які фреймворк *FastAPI* може пов'язати з параметрами функції обробника запиту. Це так звані параметри шляху (*path parameter*). Завдяки параметрам шляху можна передавати в додатки деякі значення.

Параметри шляху визначаються в шаблоні шляху всередині фігурних дужок: {назва_параметра}. Наприклад, визначимо такий додаток:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{id}")
def users(id):
    return {"user_id": id}
```

Рисунок 1.15 – Код використання параметрів у шляху

у цьому випадку декоратор *app.get* використовує шаблон шляху *"/users/{id}"*. Тут *id* представляє параметр шляху.

Функція *users()*, яка обробляє запит за цим шляхом, приймає однойменний параметр. Через цей параметр у функцію передаватиметься значення зі шляху, яке фреймворк визначить як параметр *id*.

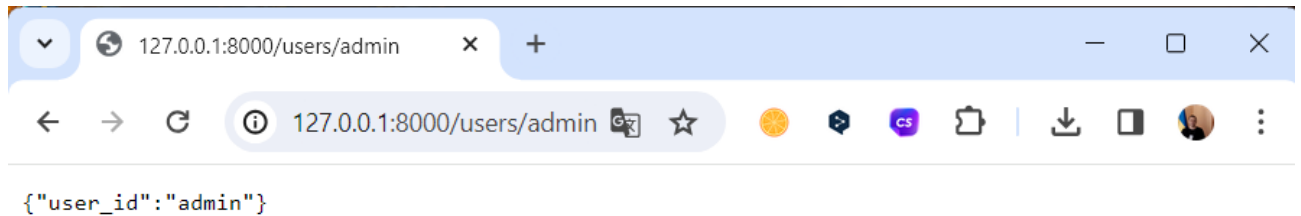


Рисунок 1.16 – Відповідь сервера у вигляді передачі значень параметру

Так, на скріншоті видно, що браузер надсилає запит за адресою `http://127.0.0.1:8000/users/admin`, тобто шлях запиту в даному випадку представляє `users/admin`. Фреймворк FastAPI бачить, що цей шлях відповідає шаблону `"/users/{id}"`, тому цей запит буде оброблятися функцією `users`.

Параметр `id` у шаблоні шляху становить другий сегмент адреси. Відповідно фреймворк зможе зіставити сегмент `"admin"` з параметром `id`. Тому при зверненні за адресою `http://127.0.0.1:8000/users/admin` параметр `id` у функції `users` отримає значення `admin`.

Подібним чином можна використовувати і більшу кількість параметрів:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{name}/{age}")
def users(name, age):
    return {"user_name": name, "user_age": age}
```

У цьому випадку функція `users` обробляє запити, шлях яких відповідає шаблону `"/users/{name}/{age}"`. Подібний шлях має складатися з трьох сегментів, де другий сегмент представляє значення параметра `name`, а третій сегмент - значення параметра `age`.

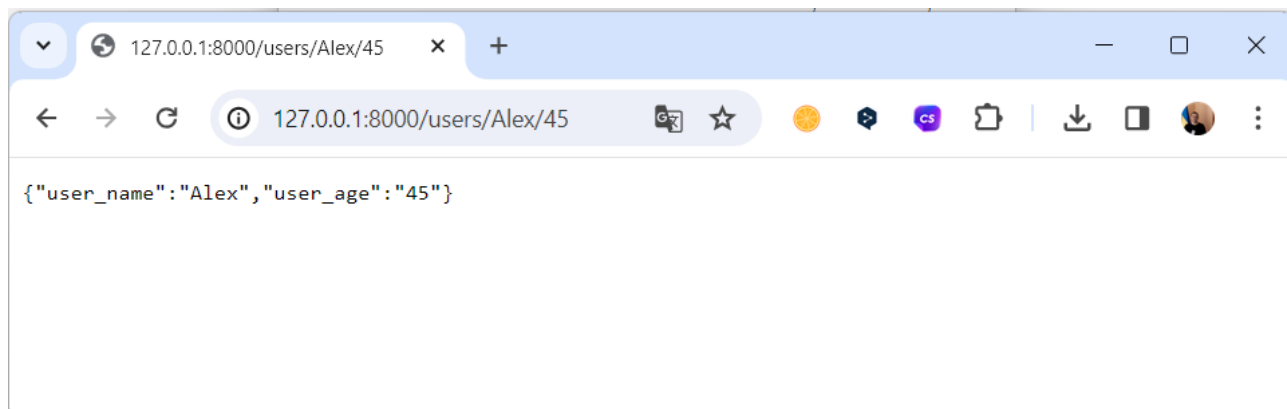


Рисунок 1.17 – Відповідь сервера у вигляді передачі значень параметрів

Зазвичай якщо в шляху передається кілька параметрів, то роздільником між ними, як правило, служить слеш, який закриває сегмент, як у прикладі вище. Однак це необов'язково, наприклад, ми можемо розділити параметри за допомогою дефіса:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{name}-{age}")
def users(name, age):
    return {"user_name": name, "user_age": age}
```

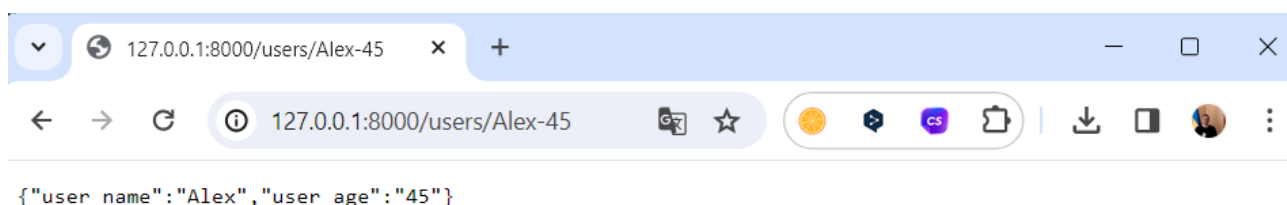


Рисунок 1.18 – Відповідь сервера у вигляді передачі значень параметрів

Такому шаблону відповідав би запит за шляхом `http://127.0.0.1:8000/users/Alex-45`, і фреймворк FastAPI автоматично розпарував би запитаний шлях і розділив би його останній сегмент на параметри *name* і *age*.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/admin")
def admin():
    return {"message": "Hello admin"}

@app.get("/users/{name}")
def users(name):
    return {"user_name": name}
```

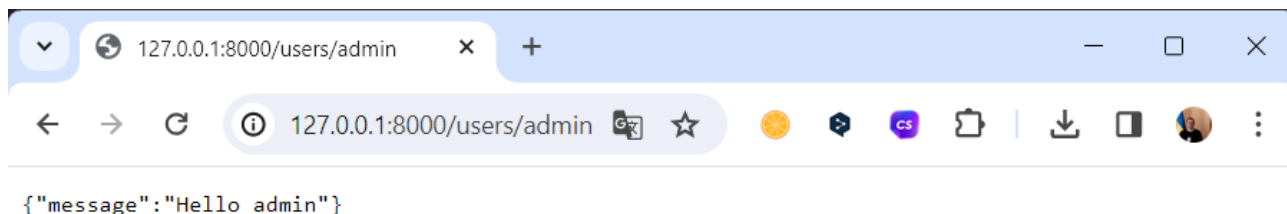


Рисунок 1.19 – Відповідь сервера у вигляді передачі значень параметрів

Черговість визначення шляхів

Однак під час визначення шаблонів шляхів слід враховувати, що між різними шаблонами може виникнути двоїстість, коли запит відповідає кільком певним шаблонам. І у зв'язку з цим слід враховувати черговість визначення шаблонів шляхів. Наприклад:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{name}")
def users(name):
    return {"user_name": name}

@app.get("/users/admin")
def admin():
    return {"message": "Hello admin"}
```

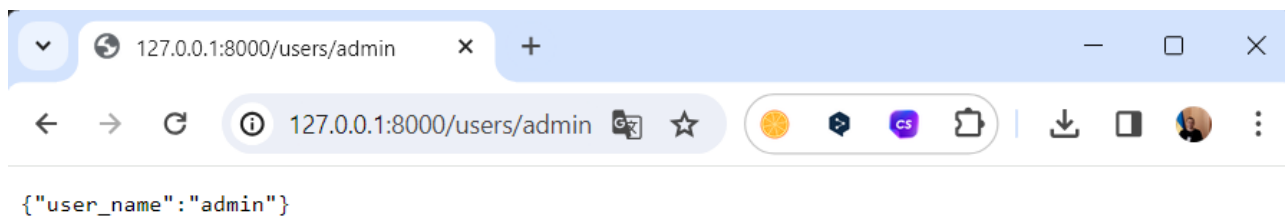


Рисунок 1.20 – Відповідь сервера

Тому що функція *users* визначена до функції *admin*, відповідно функція *users* і буде обробляти цей запит. Щоб домогтися потрібного результату, нам треба поміняти визначення функцій місцями:

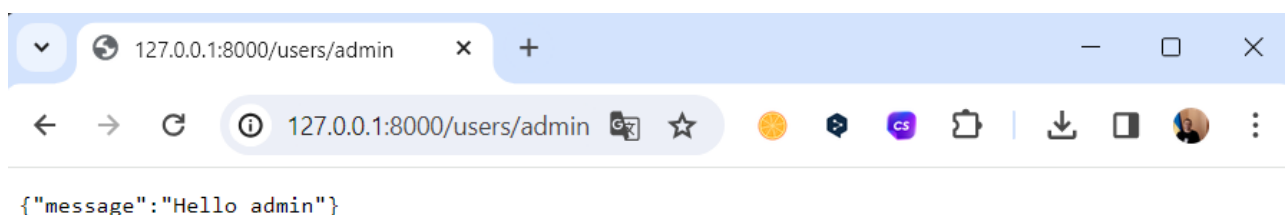


Рисунок 1.21 – Відповідь сервера

Обмеження типу параметрів

FastAPI дає змогу обмежити тип параметрів і відповідно набір використовуваних значень. Наприклад, ми хочемо, щоб параметр *id* у коді, наведено на рис.1.15, представляв тільки ціле число. Щоб вказати, що параметр - ціле число, у параметра функції явно задається тип *int*.


```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{id}")
def users(id: int):
    return {"user_id": id}
```

І якщо ми спробуємо передати цьому параметру не цілочисельне значення, то сервер надішле повідомлення про помилку:

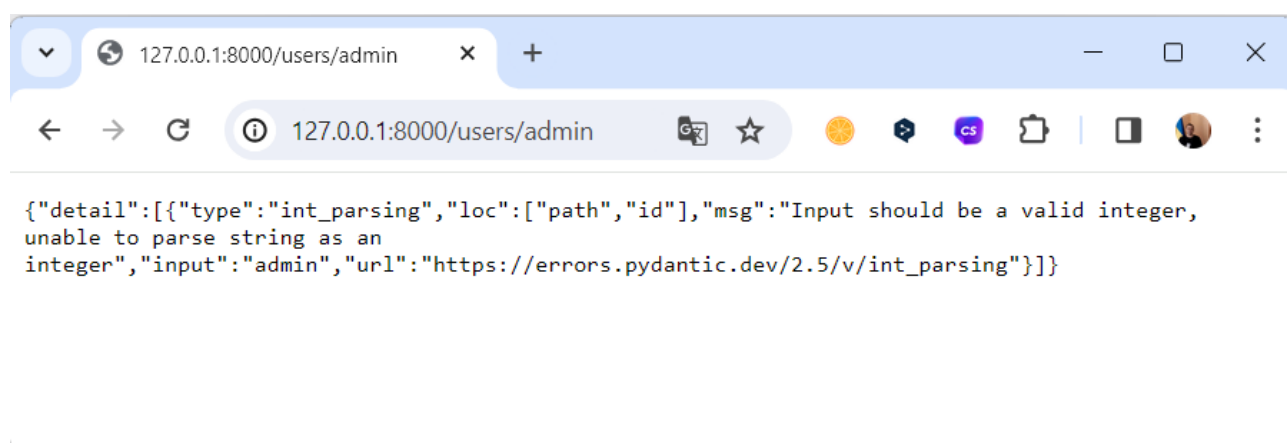


Рисунок 1.22 – Відповідь сервера про помилку

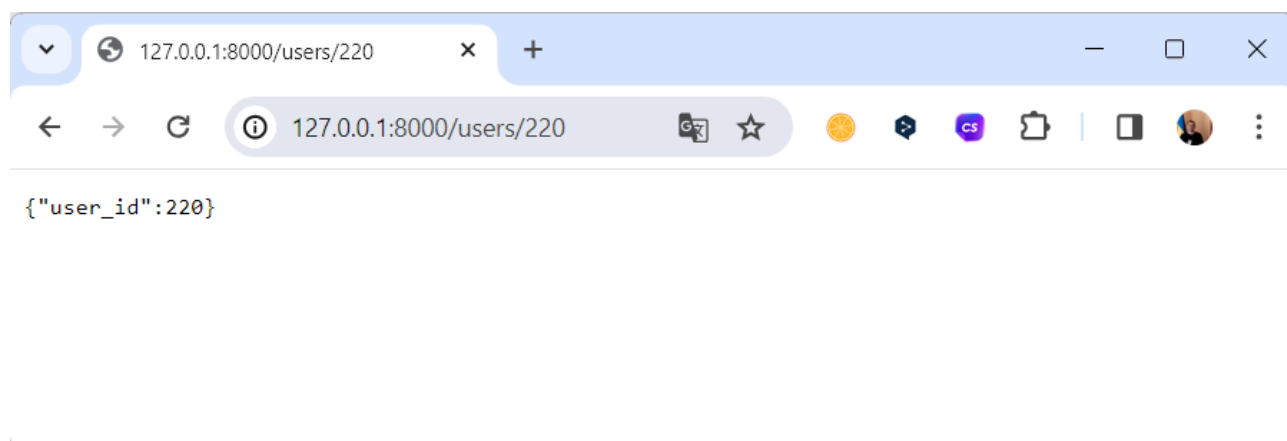


Рисунок 1.23 – Відповідь сервера з типизованими параметрами

Подібним чином як обмеження можна використовувати й інші типи: *str*, *float*, *bool* і низку інших.

Крім відправки повідомлень до клієнта, усі дії, у тому числі і помилкові фіксуються у протоколі роботи сервера та відображаються у термінал в режимі реального часу.

```

Administrator: Командная строка
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [12656]
INFO: Started server process [11036]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:30597 - "GET /users/admin HTTP/1.1" 200 OK
INFO: 127.0.0.1:30598 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:30606 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:30606 - "GET /openapi.json HTTP/1.1" 200 OK
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [11036]
INFO: Started server process [18472]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:30708 - "GET /users/admin HTTP/1.1" 422 Unprocessable Entity
INFO: 127.0.0.1:30709 - "GET /users/220 HTTP/1.1" 200 OK
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [18472]
INFO: Started server process [3108]
INFO: Waiting for application startup.
INFO: Application startup complete.

```

Рисунок 1.24 – Лог роботи серверу

Фреймворк FastAPI надає нам інтерактивну документацію з API. Для цього необхідно перейти на <http://127.0.0.1:8000/docs>. Ви побачите автоматичну інтерактивну документацію API (надану [Swagger UI](#)).



Рисунок 1.25 – Документація додатка

У документації ми і можемо побачити обмеження на типи параметрів шляху.

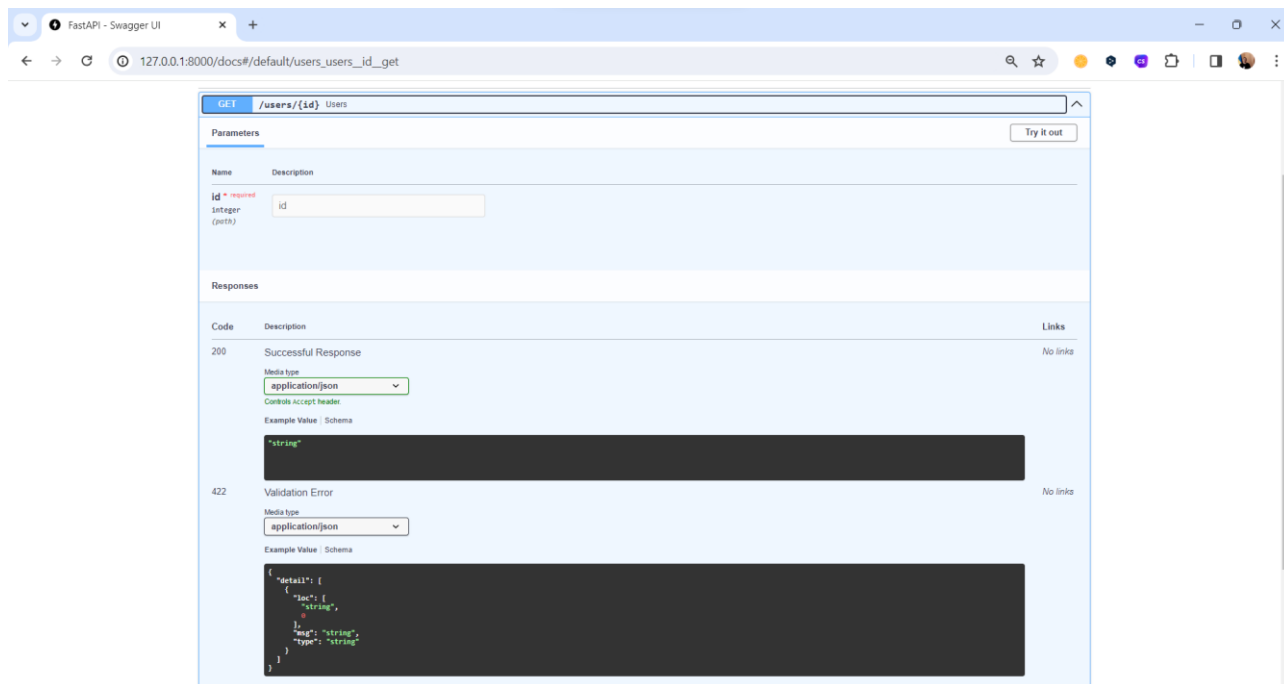


Рисунок 1.26 – Розгорнута гілка документації

Так само нам доступна альтернативна документація з API за посиланням <http://127.0.0.1:8000/redoc>, надану [ReDoc](#).

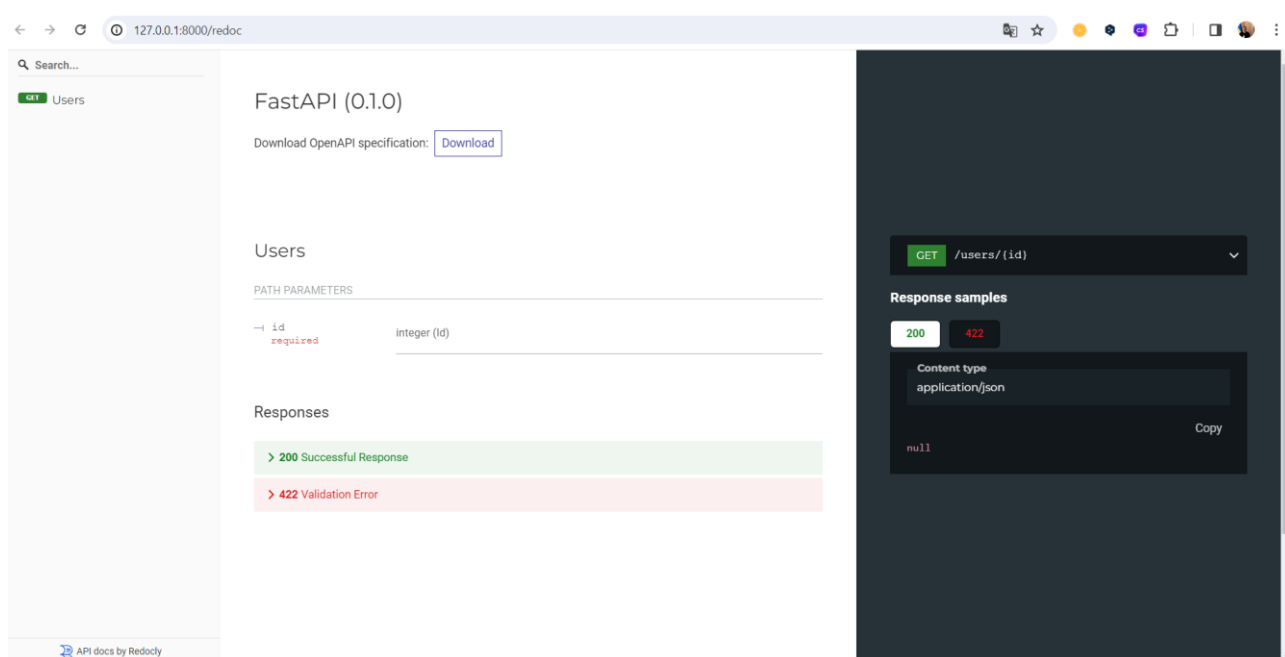


Рисунок 1.27 – Альтернативна документація проєкта

Path

Додатково для роботи з параметрами шляху фреймворк *FastAPI* надає клас *Path* з пакета *fastapi*. Клас *Path* дає змогу валідувати значення параметрів. Зокрема, через конструктор *Path* можна встановити такі параметри для валідації значень:

- *min_length*: встановлює мінімальну кількість символів у значенні параметра
- *max_length*: встановлює максимальну кількість символів у значенні параметра
- *regex*: встановлює регулярний вираз, якому має відповідати значення параметра

- *lt*: значення параметра має бути меншим за певне значення
- *le*: значення параметра має бути меншим або дорівнювати певному значенню
- *gt*: значення параметра має бути більшим за певне значення
- *ge*: значення параметра має бути більшим або дорівнювати певному значенню

Застосуємо деякі параметри:

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/users/{name}")
def users(name:str = Path(min_length=3, max_length=20)):
    return {"name": name}
```

У цьому випадку отримуємо параметр *name*. Причому його значення повинно мати не менше 3 і не більше 20 символів.

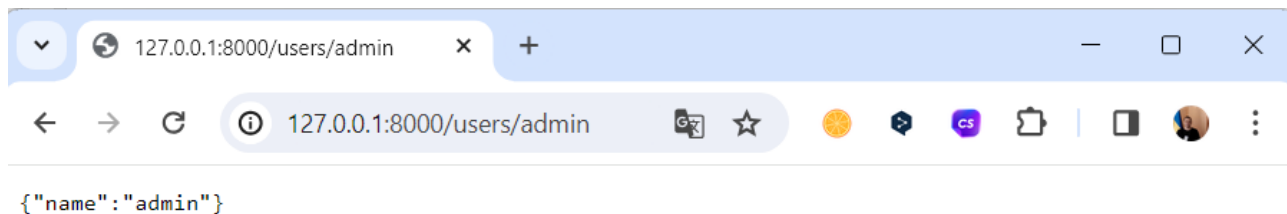


Рисунок 1.28 – Коректна відповідь сервера

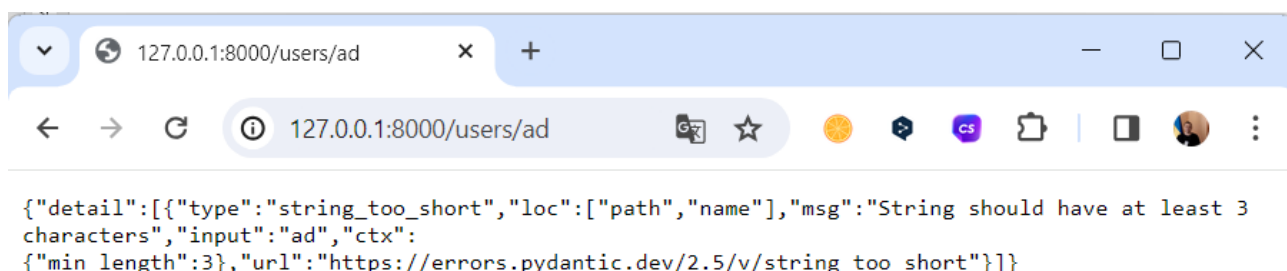


Рисунок 1.29 – Відповідь з текстом про помилку валідації

Подібним чином можна використовувати інші параметри валідації:

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/users/{name}/{age}")
def users(name: str = Path(min_length=3, max_length=20),
          age: int = Path(ge=18, lt=101)):
    return {"name": name, "age": age}
```

В даному випадку додається параметр "age", який повинен представляти число в діапазоні від 18 (включно) до 101 (не включаючи)

Валідація за допомогою регулярного значення виглядає наступним чином:

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/users/{phone}")
def users(phone:str = Path(regex="^\d{11}$")):
    return {"phone": phone}
```

Тут параметр phone має складатися з 11 цифр.

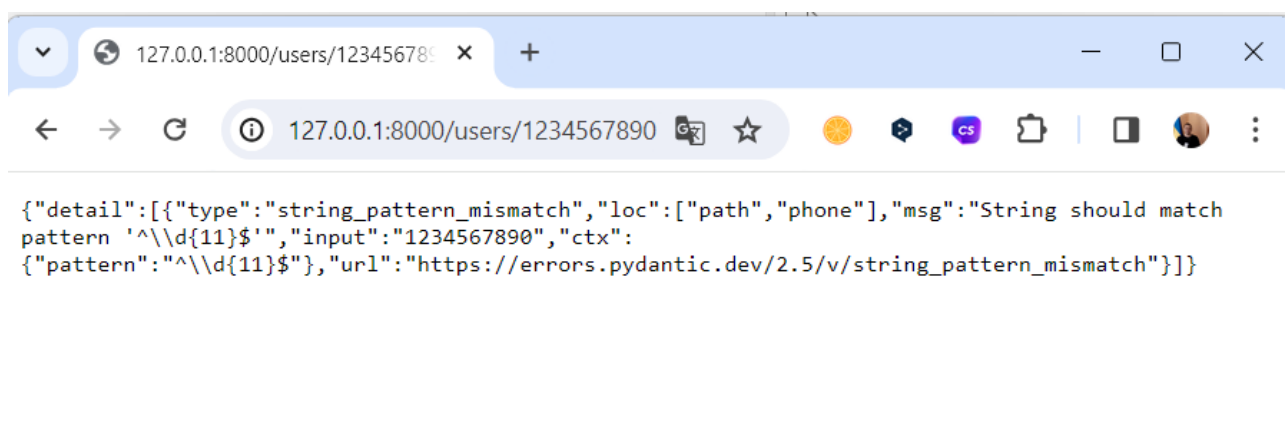


Рисунок 1.30 – Відповідь з текстом про помилку валідації

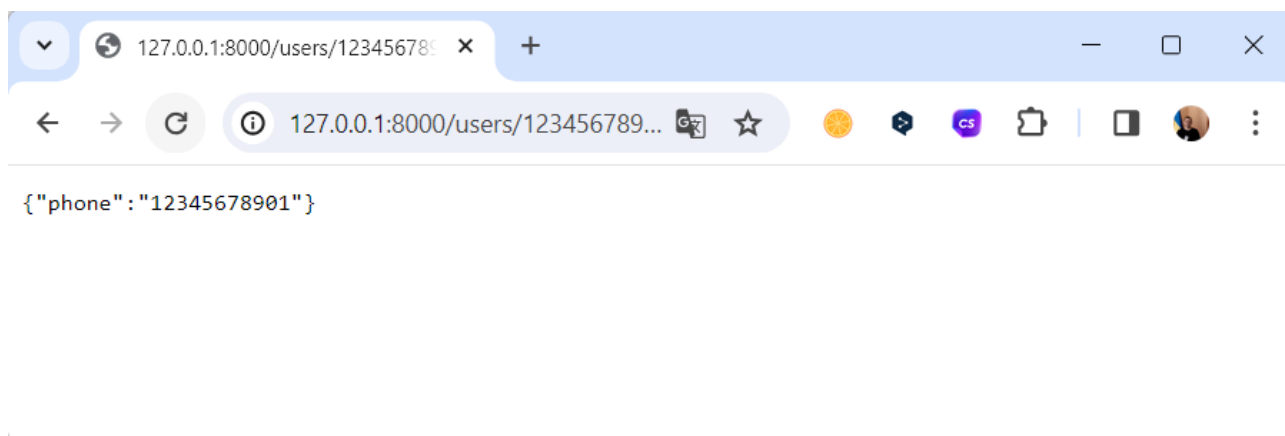


Рисунок 1.31 – Відповідь сервера

1.6 Параметри рядка запиту

Параметри рядка запиту представляють ще один спосіб передати в застосунок деякі значення в запиті типу *GET*. Для початку треба розуміти, що таке рядок запиту. Наприклад, візьмемо таку адресу

```
http://127.0.0.1:8000/users/add?name=Alex&age=45
```

Тут та частина, яка йде після адреси сервера і порту та до знака питання ?, тобто *users/add*, представляє шлях запиту (*path*). А та частина, яка йде після знака питання, тобто *name=Alex&age=45*, представляє рядок запиту (*query string*). У цій статті нас цікавитиме насамперед рядок запиту.

Рядок запиту складається з параметрів. Кожен параметр визначається у формі

```
ім'я_параметра=значення_параметра
```

Якщо рядок запиту містить кілька параметрів, то вони відокремлюються один від одного знаком амперсанду *&*. Так, у прикладі з адресою *http://127.0.0.1:8000/users/add?name=Alex&age=45* рядок запиту складається з двох параметрів: параметр *name* має значення "Alex", а параметр *age* має значення 45.

Для отримання значень параметрів рядка запиту ми можемо у функції визначити однойменні параметри:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
def get_model(name, age):
    return {"user name": name, "user age": age}
```

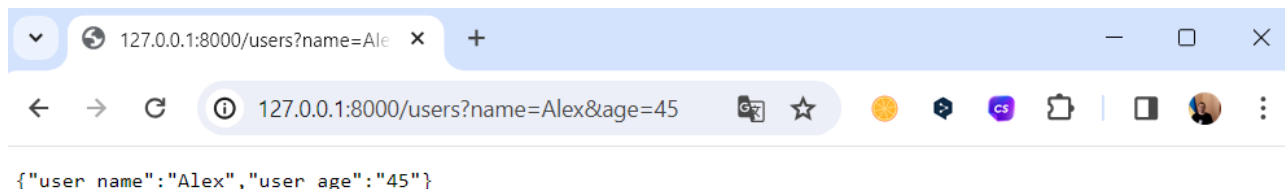


Рисунок 1.32 – Відповідь серверу на параметризований рядок запиту

Значення за замовчуванням

Цілком може бути, що під час звернення до застосунку користувач не передасть значення для будь-якого параметра або навіть для всіх параметрів рядка запиту. У прикладі вище всі параметри рядка запиту є обов'язковими. І якщо ми не передамо хоча б один із параметрів, то ми отримаємо помилку.

Щоб помилки не було, можна задати для параметрів значення за замовчуванням:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
def get_model(name = "Undefined", age = 18):
    return {"user_name": name, "user_age": age}
```

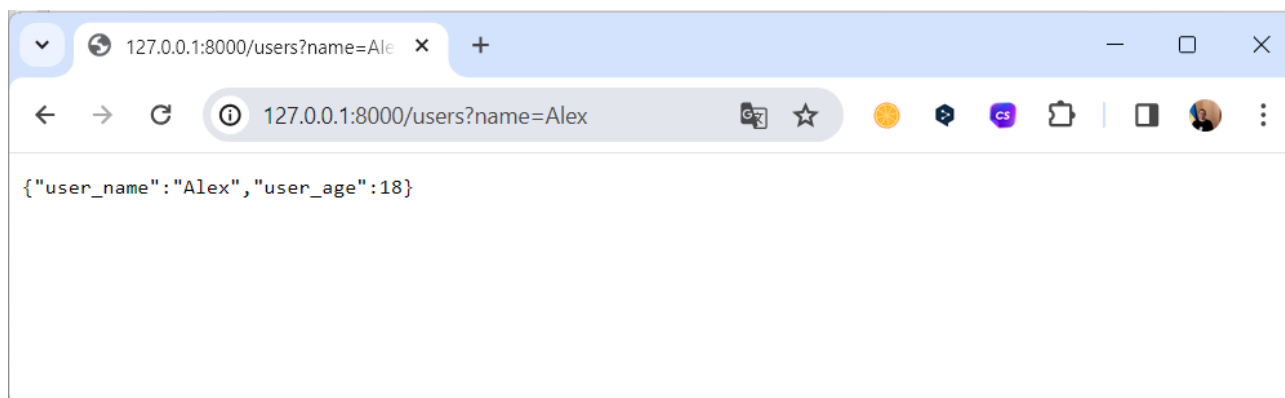


Рисунок 1.33 – Відповідь сервера з доповненими даними

Параметри зі значенням за замовчуванням повинні йти після обов'язкових параметрів.

Обмеження за типом

Також для параметрів рядка запиту можна задати обмеження за типом:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
def get_model(name: str, age: int = 18):
    return {"user_name": name, "user_age": age}
```

У цьому випадку параметр *name* має представляти тип *str*, тобто рядок, а параметр *age* - ціле число. Якщо параметру *age* передати нечислове значення, то ми отримаємо помилку.

Query

Додатково для роботи з параметрами рядка запиту фреймворк надає клас *Query* з пакета *fastapi*. Клас *Query* дає змогу насамперед валідувати значення параметрів рядка запиту. Зокрема, через конструктор *Query* можна встановити такі параметри для валідації значень:

- *min_length*: встановлює мінімальну кількість символів у значенні параметра
- *max_length*: встановлює максимальну кількість символів у значенні параметра
- *regex*: встановлює регулярний вираз, якому має відповідати значення параметра
- *lt*: значення параметра має бути меншим за певне значення
- *le*: значення параметра має бути меншим або дорівнювати певному значенню
- *gt*: значення параметра має бути більшим за певне значення

– *ge*: значення параметра має бути більшим або дорівнювати певному значенню

Застосуємо деякі параметри:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(name:str = Query(min_length=3, max_length=20)):
    return {"name": name}
```

У цьому випадку через рядок запиту отримуємо параметр `name`. Причому його значення повинно мати не менше 3 і не більше 20 символів.

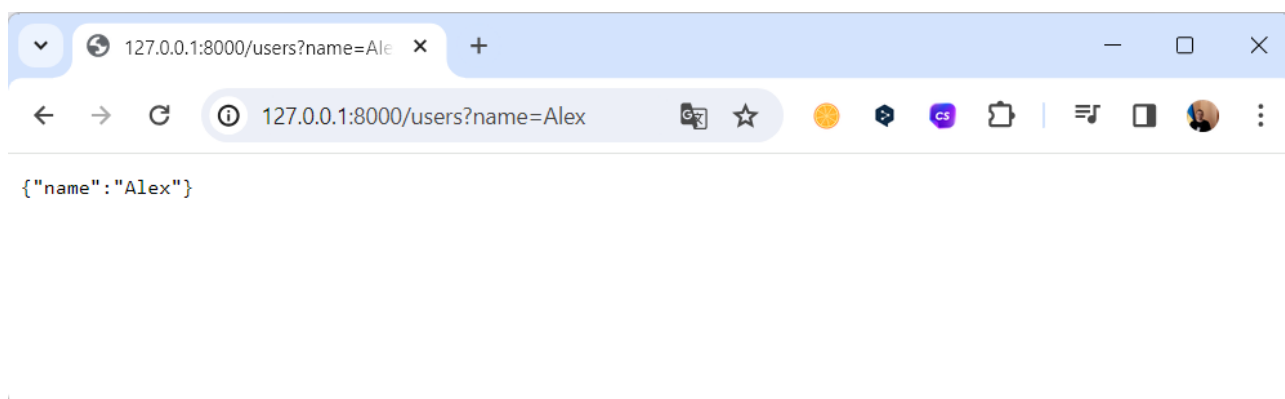


Рисунок 1.34 – Відповідь сервера на запит, що пройшов валідацію

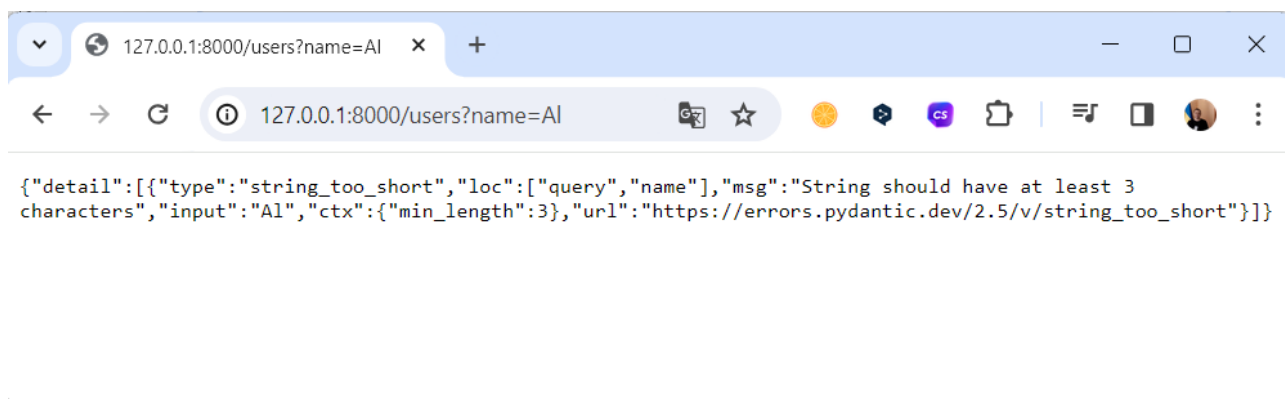


Рисунок 1.35 – Відповідь сервера на запит, що не пройшов валідацію

Подібним чином можна використовувати інші параметри валідації:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(name:str = Query(min_length=3, max_length=20),
          age: int = Query(ge=18, lt=101)):
    return {"name": name, "age": age}
```

В даному випадку додається параметр "age", який повинен представляти число в діапазоні від 18 (включно) до 101 (не включаючи)

Валідація за допомогою регулярного виразу має вигляд:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(phone:str = Query(regex="^\d{11}$")):
    return {"phone": phone}
```

Тут параметр phone має складатися з 11 цифр.

Query дає змогу встановити значення за замовчуванням за допомогою параметра default:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(name: str = Query(default="Undefined", min_length=2)):
    return {"name": name}
```

Тут, якщо в запитаній адресі відсутній параметр name, то за замовчуванням він дорівнюватиме рядку "Undefined".

Якщо параметри мають бути необов'язковими, то параметру *default* передається значення *None*:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(name:str | None = Query(default=None, min_length=2)):
    if name==None:
        return {"name": "Undefined"}
    else:
        return {"name": name}
```

Отримання списків значень

Використання класу Query дає змогу отримувати через рядок запиту списки. У загальному випадку списки значень передаються, коли в рядку запиту одному параметру кілька разів передаються різні значення. Наприклад, як у запиті за такою адресою:

```
http://127.0.0.1:8000/users?people=tom&people=Alex&people=Petr
```

Тут параметру `people` передаються три різні значення, відповідно ми очікуємо, що список `people` міститиме три елементи.

Визначимо такий додаток:

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def users(people: list[str] = Query()):
    return {"people": people}
```

Тут функція `users` має один параметр - *people*, який має представляти список рядків - тип `list[str]`.

Передамо через рядок запиту список значень у вигляді параметра *people*, наприклад, за допомогою запиту:

<http://127.0.0.1:8000/users?people=Petro&people=Alex>

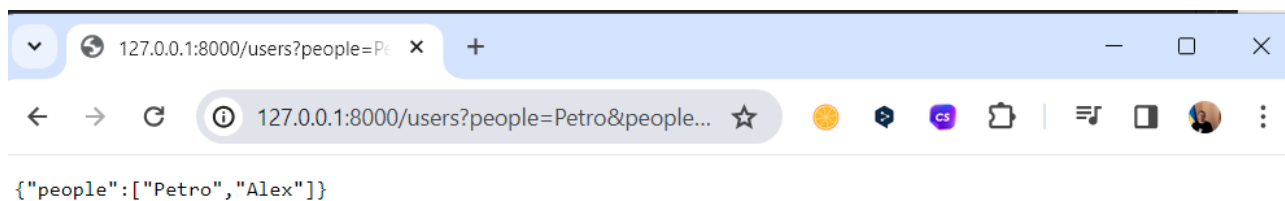


Рисунок 1.36 – Відповідь серверу у вигляді списку

Поєднання параметрів шляху та рядка запиту

За потреби можна поєднувати параметри шляху та рядка запиту:

```
from fastapi import FastAPI, Path, Query

app = FastAPI()

@app.get("/users/{name}")
def users(name:str = Path(min_length=3, max_length=20),
          age: int = Query(ge=18, lt=101)):
    return {"name": name, "age": age}
```

У цьому разі параметр *name* представляє параметр шляху, а *age* - параметр рядка запиту. І в цьому випадку ми могли б звернутися до функції *users*, наприклад, за допомогою адреси <http://127.0.0.1:8000/users/Alex?age=45>

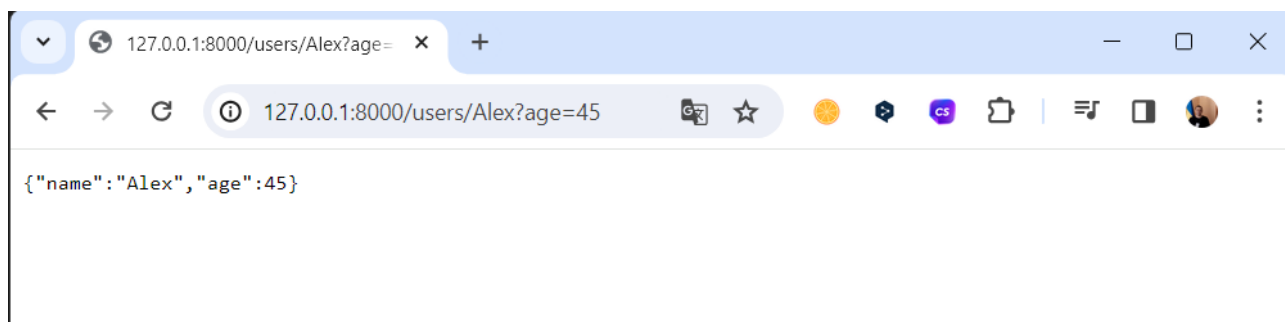


Рисунок 1.37 – Відповідь сервера

1.7 Надсилання статусних кодів

Однією з поширених задач у веб-додатку є надсилання статусних кодів, які вказують на статус виконання операції на сервері.

1xx: призначені для інформації. Відповідь із таким кодом не може мати вмісту

2xx: вказує на успішне виконання операції

3xx: призначені для переадресації

4xx: призначені для надсилання інформації про помилки клієнта

5xx: призначені для інформації про помилки сервера

За замовчуванням функції обробки надсилають статусний код 200, але за потреби ми можемо надіслати будь-який статусний код. Для цього у методів *get()*, *post()*, *put()*, *delete()*, *options()*, *head()*, *patch()*, *trace()* у класі *FastAPI* застосовується параметр *status_code*, який приймає числовий код статусу *HTTP*. Наприклад:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/notfound", status_code=404)
def notfound():
    return {"message": "Resource Not Found"}
```

У цьому разі під час звернення за шляхом *"/notfound"* клієнту надсилається статусний код помилки *404*, який свідчить про те, що ресурс не знайдено.

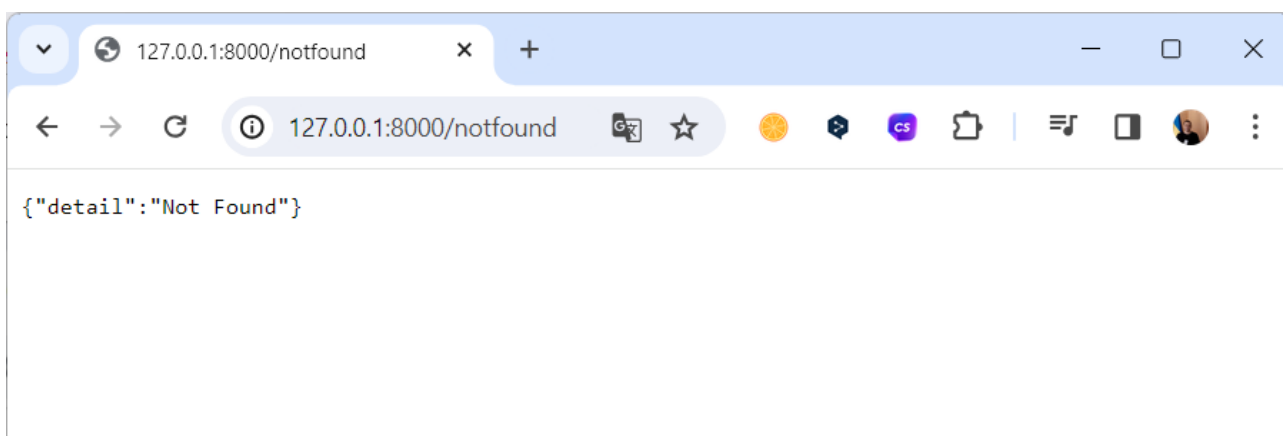


Рисунок 1.38 – Отримання статус коду

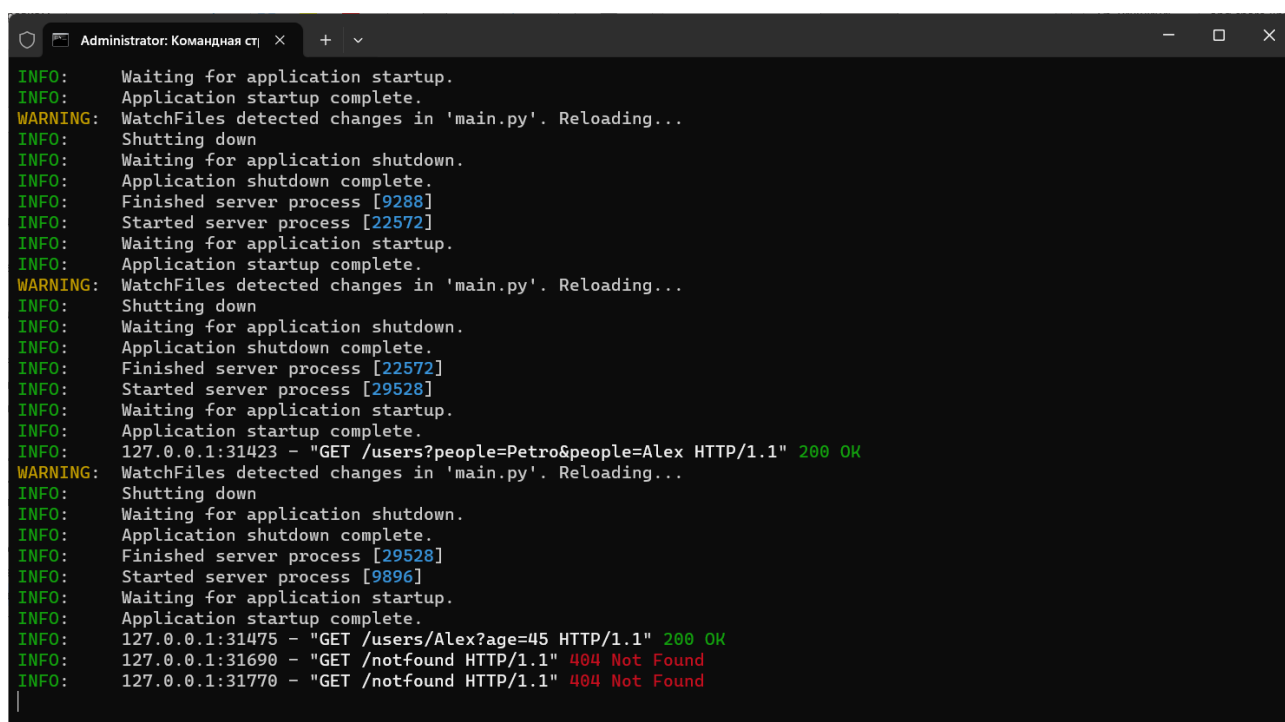
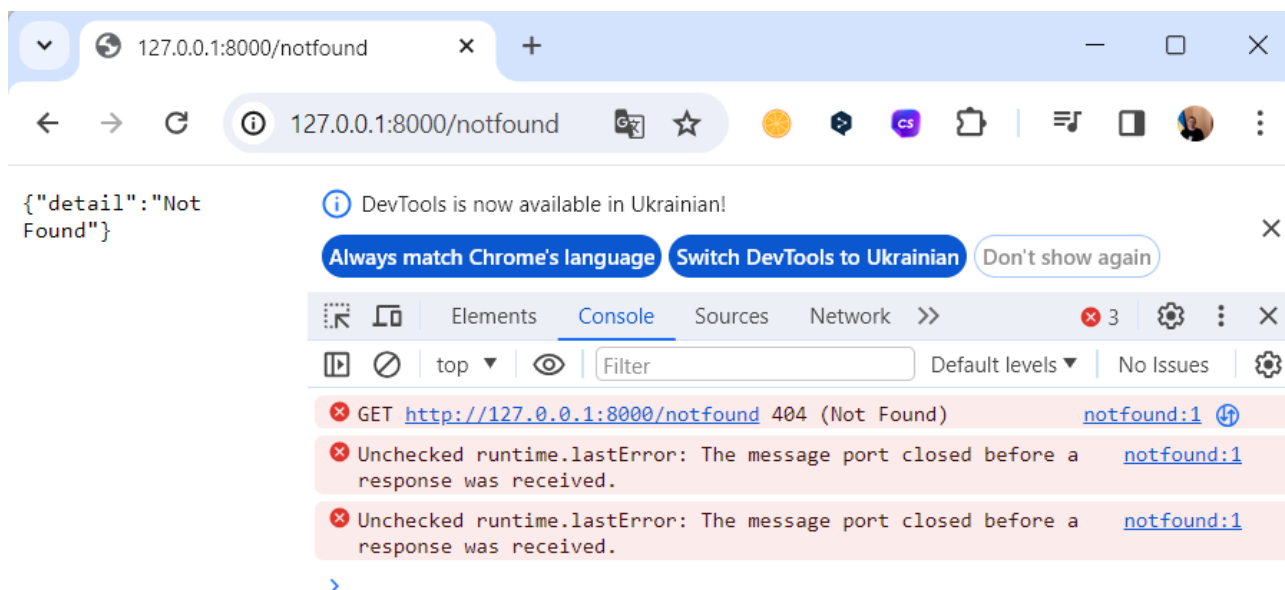


Рисунок 1.39 – Лог роботи серверу

Надсилання статусних кодів у FastAPI і Python

Для спрощення в FastAPI є модуль `status`, в якому визначено константи для представлення статусних кодів:

`HTTP_100_CONTINUE` (код 100)

`HTTP_101_SWITCHING_PROTOCOLS` (код 101)

`HTTP_102_PROCESSING` (код 102)

`HTTP_103_EARLY_HINTS` (код 103)

`HTTP_200_OK` (код 200)

`HTTP_201_CREATED` (код 201)

`HTTP_202_ACCEPTED` (код 202)

`HTTP_203_NON_AUTHORITATIVE_INFORMATION` (код 203)

`HTTP_204_NO_CONTENT` (код 204)

`HTTP_205_RESET_CONTENT` (код 205)

`HTTP_206_PARTIAL_CONTENT` (код 206)

`HTTP_207_MULTI_STATUS` (код 207)

`HTTP_208_ALREADY_REPORTED` (код 208)

`HTTP_226_IM_USED` (код 226)

`HTTP_300_MULTIPLE_CHOICES` (код 300)

`HTTP_301_MOVED_PERMANENTLY` (код 301)

`HTTP_302_FOUND` (код 302)

`HTTP_303_SEE_OTHER` (код 303)

`HTTP_304_NOT_MODIFIED` (код 304)

`HTTP_305_USE_PROXY` (код 305)

`HTTP_306_RESERVED` (код 306)

`HTTP_307_TEMPORARY_REDIRECT` (код 307)

`HTTP_308_PERMANENT_REDIRECT` (код 308)

`HTTP_400_BAD_REQUEST` (код 400)

HTTP_401_UNAUTHORIZED (код 401)
HTTP_402_PAYMENT_REQUIRED (код 402)
HTTP_403_FORBIDDEN (код 403)
HTTP_404_NOT_FOUND (код 404)
HTTP_405_METHOD_NOT_ALLOWED (код 405)
HTTP_406_NOT_ACCEPTABLE (код 406)
HTTP_407_PROXY_AUTHENTICATION_REQUIRED (код 407)
HTTP_408_REQUEST_TIMEOUT (код 408)
HTTP_409_CONFLICT (код 409)
HTTP_410_GONE (код 410)
HTTP_411_LENGTH_REQUIRED (код 411)
HTTP_412_PRECONDITION_FAILED (код 412)
HTTP_413_REQUEST_ENTITY_TOO_LARGE (код 413)
HTTP_414_REQUEST_URI_TOO_LONG (код 414)
HTTP_415_UNSUPPORTED_MEDIA_TYPE (код 415)
HTTP_416_REQUESTED_RANGE_NOT_SATISFIABLE (код 416)
HTTP_417_EXPECTATION_FAILED (код 417)
HTTP_418_IM_A_TEAPOT (код 418)
HTTP_421_MISDIRECTED_REQUEST (код 421)
HTTP_422_UNPROCESSABLE_ENTITY (код 422)
HTTP_423_LOCKED (код 423)
HTTP_424_FAILED_DEPENDENCY (код 424)
HTTP_425_TOO_EARLY (код 425)
HTTP_426_UPGRADE_REQUIRED (код 426)
HTTP_428_PRECONDITION_REQUIRED (код 428)
HTTP_429_TOO_MANY_REQUESTS (код 429)
HTTP_431_REQUEST_HEADER_FIELDS_TOO_LARGE (код 431)
HTTP_451_UNAVAILABLE_FOR_LEGAL_REASONS (код 451)
HTTP_500_INTERNAL_SERVER_ERROR (код 500)

HTTP_501_NOT_IMPLEMENTED (код 501)

HTTP_502_BAD_GATEWAY (код 502)

HTTP_503_SERVICE_UNAVAILABLE (код 503)

HTTP_504_GATEWAY_TIMEOUT (код 504)

HTTP_505_HTTP_VERSION_NOT_SUPPORTED (код 505)

HTTP_506_VARIANT_ALSO_NEGOTIATES (код 506)

HTTP_507_INSUFFICIENT_STORAGE (код 507)

HTTP_508_LOOP_DETECTED (код 508)

HTTP_510_NOT_EXTENDED (код 510)

HTTP_511_NETWORK_AUTHENTICATION_REQUIRED (код 511)

Приклад використання

```
from fastapi import FastAPI, status

app = FastAPI()

@app.get("/notfound", status_code=status.HTTP_404_NOT_FOUND)
def notfound():
    return {"message": "Resource Not Found"}
```

Визначення статусного коду у відповіді

У прикладі вище функція незалежно від даних запиту або якихось інших умов у будь-якому разі повертала статусний код 404. Однак частіше буває необхідно повертати статусний код залежно від деяких умов. У цьому разі ми можемо використовувати параметр *status_code* конструктора класу *Response* або його спадкоємців:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/notfound")
def notfound():
    return JSONResponse(content={"message": "Resource Not Found"}, status_code=404)
```

Зміна статусного коду

Можна комбінувати обидва підходи:

```
from fastapi import FastAPI, Response, Path

app = FastAPI()

@app.get("/users/{id}", status_code=200)
def users(response: Response, id: int = Path()):
    if id < 1:
        response.status_code = 400
        return {"message": "Неправильні дані"}
    return {"message": f"Id = {id}"}
```

У цьому випадку якщо параметр шляху менше 1, то умовно вважаємо, що передані некоректні дані, і відправляємо у відповідь статусний код 400 (Bad Request)

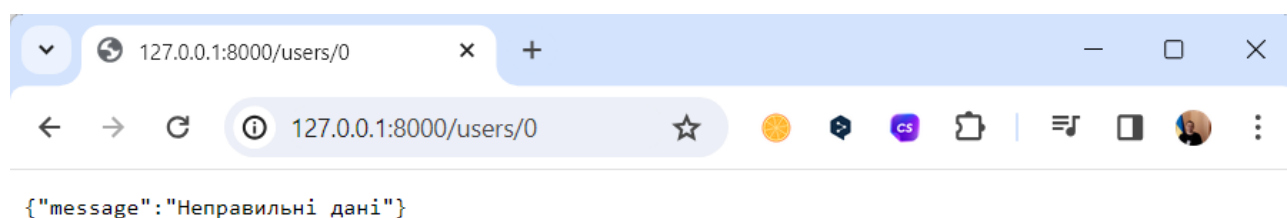


Рисунок 1.40 – Відповідь серверу зі статус кодом 400

```

Administrator: Командная строка
INFO: 127.0.0.1:31690 - "GET /notfound HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:31770 - "GET /notfound HTTP/1.1" 404 Not Found
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [9896]
INFO: Started server process [21552]
INFO: Waiting for application startup.
INFO: Application startup complete.
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [21552]
INFO: Started server process [32012]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:31871 - "GET /notfound HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:31872 - "GET /notfound HTTP/1.1" 404 Not Found
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [32012]
INFO: Started server process [19856]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:31892 - "GET /users/10 HTTP/1.1" 200 OK
INFO: 127.0.0.1:31893 - "GET /users/0 HTTP/1.1" 400 Bad Request

```

Рисунок 1.41 – Лог роботи серверу

1.8 Переадресація

Для переадресації в застосунку *FastAPI* застосовується клас *RedirectResponse* (клас-спадкоємець від *Response*). Як обов'язковий параметр конструктор *RedirectResponse* приймає адресу для перенаправлення:

```

import mimetypes
from fastapi import FastAPI
from fastapi.responses import RedirectResponse, PlainTextResponse

app = FastAPI()

@app.get("/old")
def old():
    return RedirectResponse("/new")

@app.get("/new")
def new():
    return PlainTextResponse("New page")

```

У даному випадку при зверненні за шляхом *"/old"* відбувається перенаправлення за шляхом *"/new"*. Альтернативний варіант:

```
import mimetypes
from fastapi import FastAPI
from fastapi.responses import RedirectResponse, PlainTextResponse

app = FastAPI()

@app.get("/old", response_class= RedirectResponse)
def old():
    return "/new"

@app.get("/new")
def new():
    return PlainTextResponse("New page")
```

Також можна перенаправляти за абсолютною адресою:

```
@app.get("/old")
def old():
    return RedirectResponse("https://server.com/webpython/fastapi")
```

За замовчуванням *RedirectResponse* відправляє статусний код 307 (тимчасова переадресація).

```
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: Started server process [11204]
INFO: Waiting for application startup.
INFO: Application startup complete.
WARNING: WatchFiles detected changes in 'main.py'. Reloading...
INFO: 127.0.0.1:32063 - "GET /new HTTP/1.1" 200 OK
INFO: 127.0.0.1:32063 - "GET /old HTTP/1.1" 307 Temporary Redirect
INFO: 127.0.0.1:32063 - "GET /new HTTP/1.1" 200 OK
```

Якщо такий стан не влаштовує, то можна задати статусний код переадресації за допомогою параметра *status_code*:

```
@app.get("/old")
def old():
    return RedirectResponse("/new", status_code=302)
```

або так

```
@app.get("/old", response_class= RedirectResponse, status_code=302)
def old():
    return "/new"
```

1.9 Статичні файли

Для визначення інтерфейсу для роботи з сервером нерідко використовуються *html*-сторінки, тобто статичні файли з кодом *html*, які можуть використовувати якісь інші статичні файли - файли стилів *css*, зображень, скриптів *JavaScript* і т.д. Для роботи зі статичними файлами *FastAPI* надає зручний та компактний функціонал, який розміщується у пакеті *fastapi.staticfiles*. Зокрема, для обслуговування статичних файлів у певному каталозі застосовується клас *StaticFiles*, конструктор якого має таку сигнатуру:

```
StaticFiles(directory=None, packages=None, html=False,
check_dir=True)
```

Використовувані параметри:

- *directory*: шлях до каталогу зі статичними файлами
- *packages*: список пакетів python у вигляді списку рядків або кортежів рядків
- *html*: встановлює запуск у HTML-режимі, коли при зверненні до кореня каталогу автоматично завантажується файл *index.html* (за наявності такого файлу)
- *check_dir*: гарантує, що каталог із статичними файлами існує

Розглянемо невеликий приклад. Нехай у нас буде наступний проект (рис.1.12) у якому визначено каталог *public*, призначений для зберігання статичних файлів. І визначимо в цьому каталозі найпростіший файл *index.html*.

У файлі *main.py* обслуговування статичних файлів з каталогу *public* визначимо наступний код:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

app.mount("/static", StaticFiles(directory="public"))
```

Для роботи зі статичними файлами спочатку імпортуємо клас *StaticFiles*. Потім створюємо об'єкт програми *FastAPI* і викликаємо метод *mount()*. Метод *mount()* встановлює об'єкт *ASGIApp0* - обробник запитів певним шляхом. В даному випадку для запитів шляхом *"/static"* як обробка запитів виступає об'єкт *StaticFiles*, в якому за допомогою параметра *directory* як каталог статичних файлів встановлюється каталог *"/public"* (назва каталогу довільна).

Тобто при зверненні шляхом *"/static"* додаток буде посилати у відповідь файли з каталогу *"public"*.

Запустимо додаток і звернемося шляхом *http://127.0.0.1:8000/static/index.html*, і додаток у відповідь надішле нам файл *index.html*:

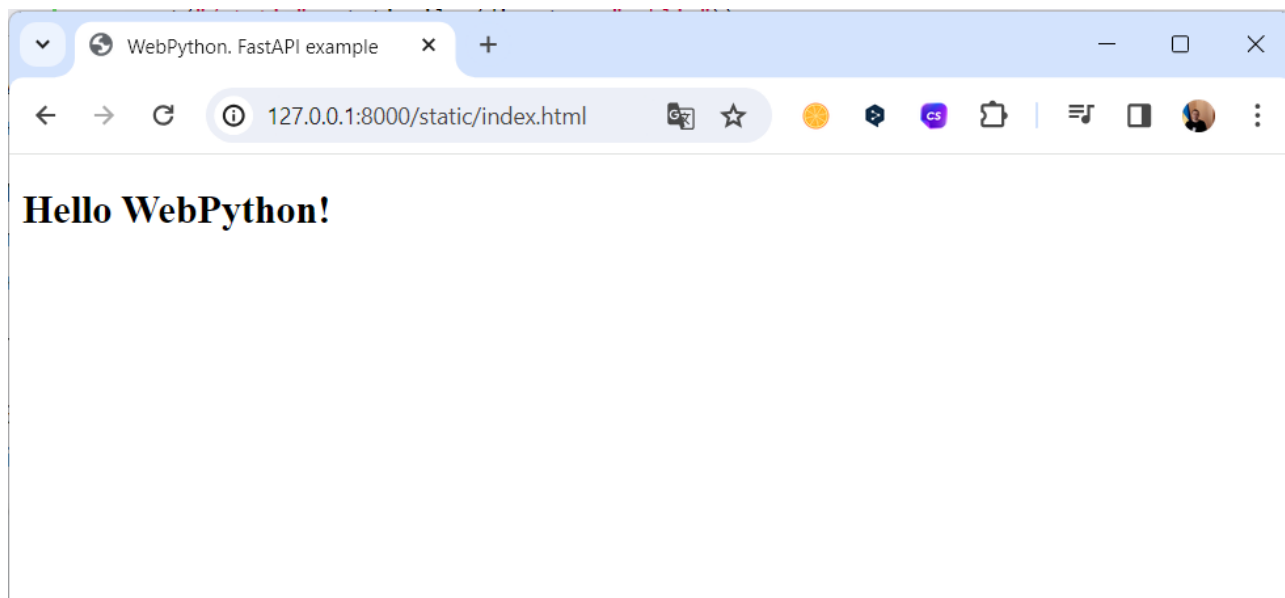


Рисунок 1.42 – Відповідь серверу на *http://127.0.0.1:8000/static/index.html*

Подібним чином ми можемо додавати до каталогу *public* та інші статичні файли.

Встановлення головної сторінки

У прикладі вище для звернення до файлу *index.html* нерідко веб-додаток має певну головну сторінку. Наприклад, коли ми звертаємось до кореня деяких сайтів або до кореня окремих каталогів, веб-сервер надсилає головну сторінку цього сайту або каталогу. Клас *StaticFiles* також дозволяє зробити це за допомогою передачі параметру

html значення *True* (значення за замовчуванням - *False*). У цьому випадку, якщо в дорозі не вказується ім'я файлу, сервер автоматично відправляє файл *index.html* (за його наявності). Наприклад, змінимо код *main.py* наступним чином:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

app.mount("/static", StaticFiles(directory="public", html=True))
```

Тепер ми можемо звернутися шляхом <http://127.0.0.1:8000/static/>, і сервер також надішле нам сторінку *index.html*:

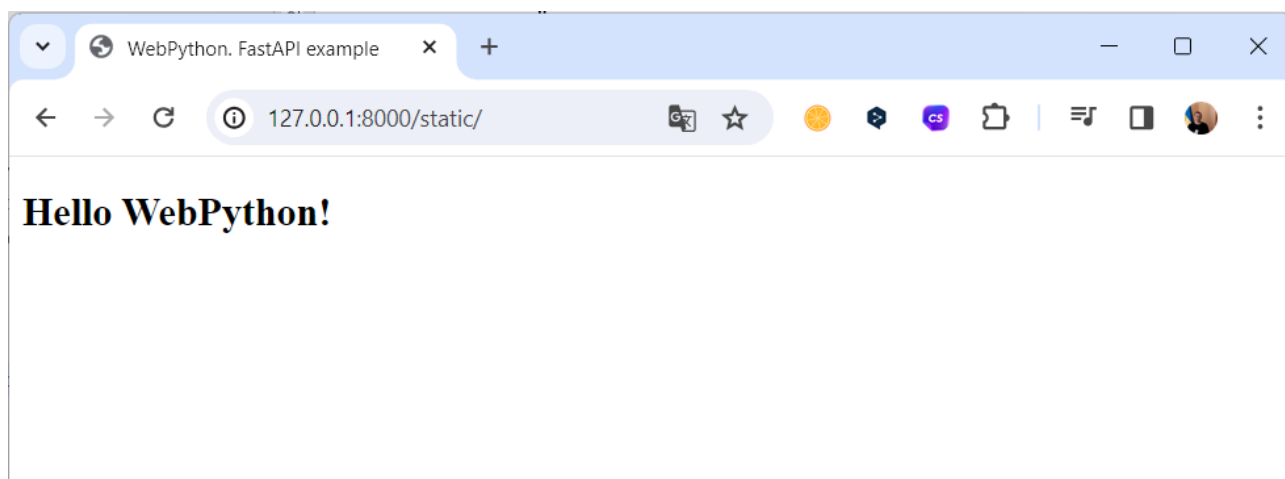


Рисунок 1.43 – Відповідь серверу на <http://127.0.0.1:8000/static>

Подібним чином можна встановити головну сторінку і для всієї веб-програми:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

app.mount("/", StaticFiles(directory="public", html=True))
```

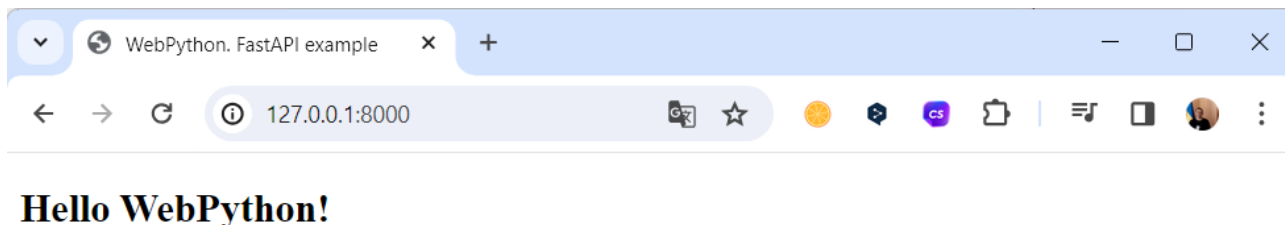



Рисунок 1.44 – Відповідь серверу на *http://127.0.0.1:8000*

1.10 Отримання даних запиту

У запиті можуть передаватися різні дані, наприклад, через надсилання якихось значень у форматі *json*. Розглянемо, як отримувати подібні дані.

Body

Для отримання даних з тіла запиту можна використовувати клас *Body* з пакета *fastapi*. Цей клас дає змогу пов'язати з параметром функції-обробника запиту або все тіло запиту, або якісь окремі його значення. Для прикладу для спрощення надсилання даних визначимо в проєкті папку *public*, у якій створимо новий файл *index-1.html* із таким кодом:

```
<!DOCTYPE html>
<html>
<head>
  <title>WebPython. FastAPI example #2</title>
  <meta charset="utf-8" />
</head>
<body>
  <div id="message"></div>
  <p>
    Input name: <br />
    <input name="username" id="username" />
  </p>
  <p>
    Input age: <br />
    <input name="userage" id="userage" type="number" />
  </p>
</body>
</html>
```

```

    <button onclick="send()">Send</button>
<script>
    async function send(){

        // отримуємо введені в поле ім'я та вік
        const username = document.getElementById("username").value;
        const userage = document.getElementById("userage").value;

        // надсилаємо запит
        const response = await fetch("/hello", {
            method: "POST",
            headers: { "Accept": "application/json", "Content-Type":
"application/json" },
            body: JSON.stringify({
                name: username,
                age: userage
            })
        });
        if (response.ok) {
            const data = await response.json();
            document.getElementById("message").textContent = data.message;
        }
        else
            console.log(response);
    }
</script>
</body>
</html>

```

У цьому випадку для введення даних на веб-сторінці визначено текстове поле, у яке користувач вводить своє ім'я. Після натискання на кнопку спрацьовує функція *send()*, визначена в коді *javascript*. Ця функція отримує введені значення і за допомогою функції *fetch* відправляє його за адресою *"/hello"* у запиті типу *POST*. При цьому в тілі запиту надсилаються серіалізовані в *JSON* дані. Зокрема, надсилається об'єкт, у якому властивість *name* зберігає введені ім'я.

Після надсилання запиту ми очікуємо, що сервер у відповідь надішле нам деякі дані - деякий об'єкт *JSON*, у якому буде властивість *"message"*. І значення цієї властивості виводимо на веб-сторінку в елемент з *id=message*.

Потім визначимо основний файл додатка *main.py*, який буде отримувати й обробляти запити:

```

from fastapi import FastAPI, Body
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
#def hello(name = Body(embed=True)):
def hello(data = Body()):
    name = data["name"]
    age = data["age"]
    return {"message": f"{name}, your age - {age}"}

```

Тут під час звернення за шляхом "/" клієнту надсилатиметься сторінка index-1.html для введення даних.

Для обробки отриманих у *POST*-запиті даних за адресою */hello* визначено функцію *hello()*. Ця функція має один параметр - *data*, який отримує вміст тіла запиту - рядок 11.

Тобто тут *data* представлятиме весь об'єкт, який надсилається з веб-сторінки і який має властивості *"name"* та *"age"*. Цей об'єкт у *python* представлятиме словник. Відповідно, щоб отримати значення властивості *"name"*, звертаємося за однойменним ключем – рядок 12.

Подібним чином отримуємо значення властивості *"age"*. Потім у відповідь клієнту надсилається словник з елементом *"message"*.

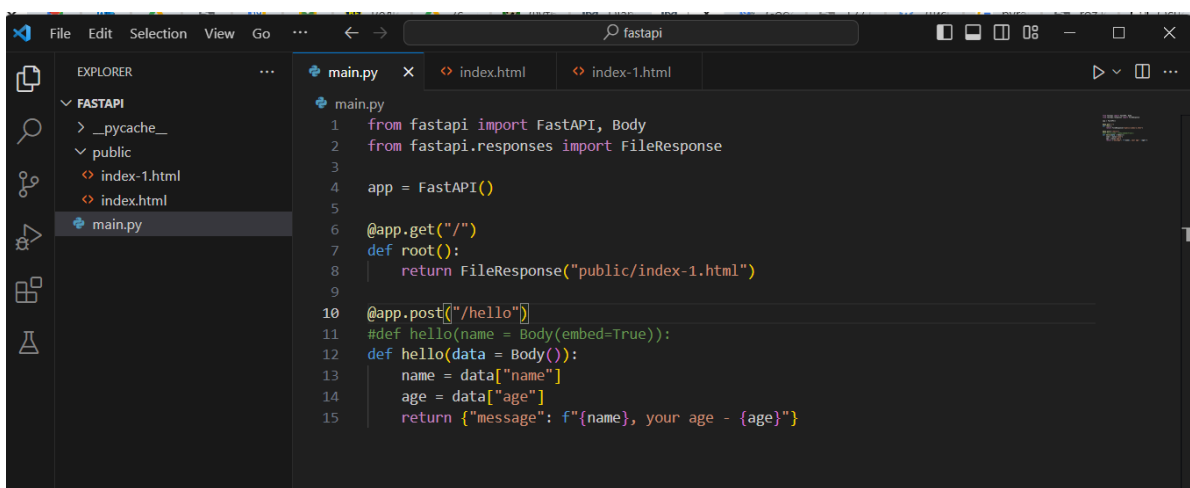


Рисунок 1.45 – Структура проекту та зміст файлу main.py

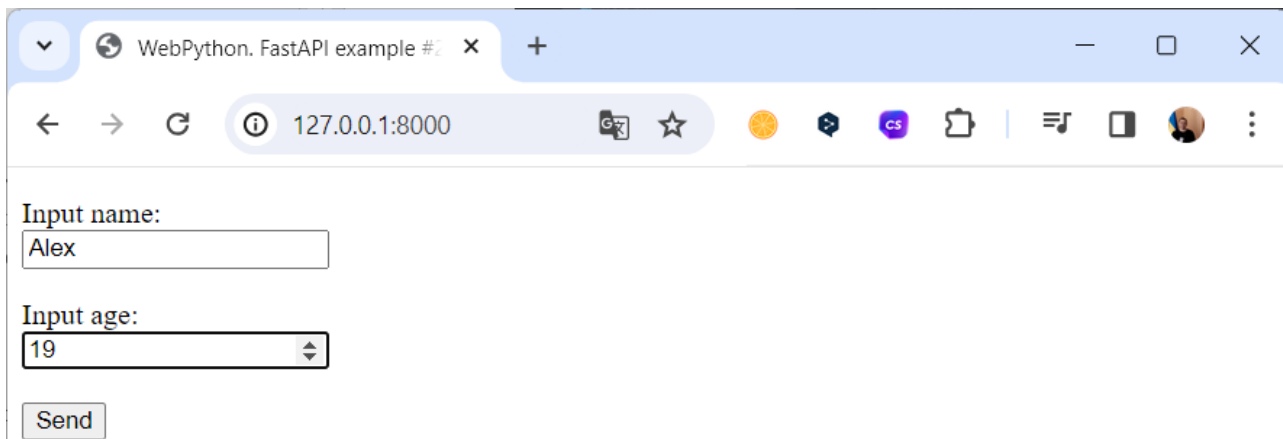


Рисунок 1.46 – Заповнення даних та відправка на сервер

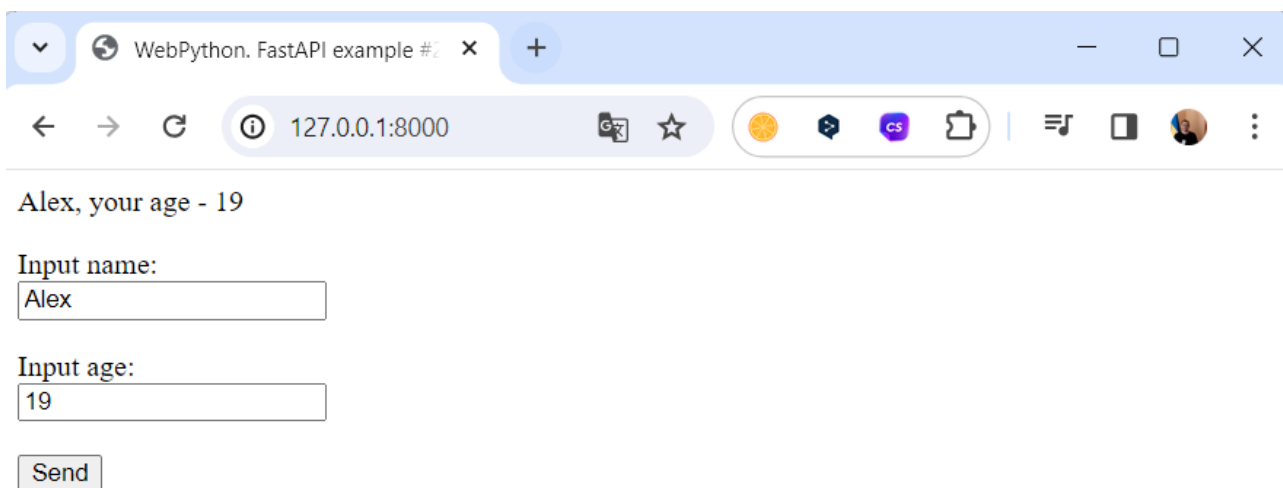


Рисунок 1.47 – Відповідь сервера

Отримання окремих значень

У прикладі вище ми отримували всі дані з тіла запиту в один параметр. Однак, встановивши параметр `embed=True`, можна отримувати окремі значення:

```
from fastapi import FastAPI, Body
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
def hello(name = Body(embed=True), age = Body(embed=True)):
    return {"message": f"{name}, ваш вік - {age}"}
```

Валідація

Клас `Body` дає змогу валідувати значення з тіла запиту. Зокрема, через конструктор `Body` можна встановити такі параметри для валідації значень:

- *min_length*: встановлює мінімальну кількість символів у значенні параметра
- *max_length*: встановлює максимальну кількість символів у значенні параметра
- *regex*: встановлює регулярний вираз, якому має відповідати значення параметра

параметра

- *lt*: значення параметра має бути меншим за певне значення
- *le*: значення параметра має бути меншим або дорівнювати певному значенню
- *gt*: значення параметра має бути більшим за певне значення
- *ge*: значення параметра має бути більшим або дорівнювати певному значенню

Застосуємо деякі параметри:

```
from fastapi import FastAPI, Body
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
def hello(name:str = Body(embed=True, min_length=3, max_length=20),
          age: int = Body(embed=True, ge=18, lt=101)):
    return {"message": f"{name}, ваш вік - {age}"}
```

В даному випадку значення параметра *name* повинно мати не менше 3 і не більше 20 символів, а параметр *"age"* повинен представляти число в діапазоні від 18 (включно) до 101 (не включаючи)

1.11 Отримання даних запиту у вигляді об'єкта класу

У минулому пункті розглядалося отримання даних з тіла запиту за допомогою класу *fastapi.Body* у вигляді словника або окремих його значень. Однак *FastAPI* також дозволяє отримувати дані у вигляді об'єктів своїх класів. Такі класи мають бути успадковані від класу *pydantic.BaseModel*. Такі класи визначаються спеціально під запит, дані якого необхідно отримати.

Наприклад, визначимо у файлі додатка такий код:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
def hello(person: Person):
    return {"message": f"Привіт, {person.name}, твій вік - {person.age}"}
```

Тут у функції *hello* отримуємо дані з тіла запиту в об'єкт класу *Person*. Цей клас успадкований від *BaseModel*. Клас *Person* визначає два атрибути, які відповідають даним з тіла запиту, які ми збираємося отримати. У цьому випадку це атрибут *name*, який представляє рядок, і атрибут *age*, який представляє ціле число.

Отримавши дані, ми зможемо працювати з ними як з даними об'єкта *Person*, наприклад, звернутися до його атрибутів *name* і *age*. Зокрема, в даному випадку, використовуючи ці атрибути, формуємо і відправляємо клієнту деяке повідомлення.

У проекті визначимо папку *public*, у якій визначимо для тестування веб-сторінку *index-1.html* з таким кодом:

```

<!DOCTYPE html>
<html>
<head>
  <title> WebPython. FastAPI приклад #3</title>

  <meta charset="utf-8" />
</head>
<body>
  <div id="message"></div>
  <p>
    Введіть ім'я: <br />
    <input name="username" id="username" />
  </p>
  <p>
    Введіть вік: <br />
    <input name="userage" id="userage" type="number" />
  </p>
  <button onclick="send()">Надіслати</button>
<script>
  async function send(){

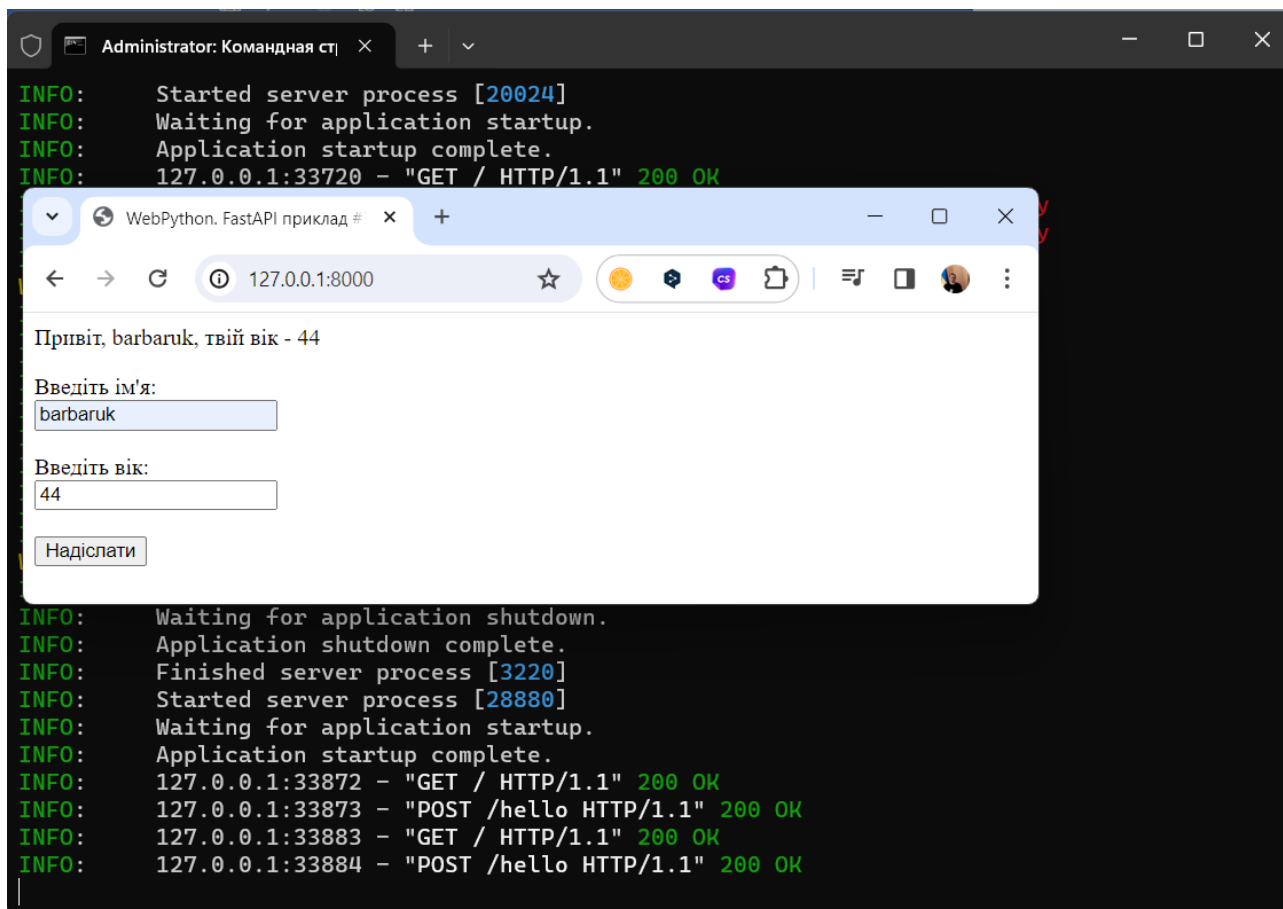
    //отримуємо введені в поля ім'я та вік
    const username = document.getElementById("username").value;
    const userage = document.getElementById("userage").value;

    // надсилаємо запит
    const response = await fetch("/hello", {
      method: "POST",
      headers: { "Accept": "application/json", "Content-Type":
"application/json" },
      body: JSON.stringify({
        name: username,
        age: userage
      })
    });
    if (response.ok) {
      const data = await response.json();
      document.getElementById("message").textContent = data.message;
    }
    else
      console.log(response);
  }
</script>
</body>
</html>

```

Тут на сервер за адресою `/hello` у запиті типу POST надсилаються введені в поля значення. Ці значення в тілі запиту мають ключі `"name"` і `"age"` - як і атрибути моделі *Person*: між ключами даних, що надсилаються, та атрибутами класу має бути

відповідність за іменем. Отримане від сервера повідомлення відображається на веб-сторінці в блоці зверху:



Необов'язкові атрибути

У прикладі вище обидва атрибути: і *name*, і *age* є обов'язковими. А це означає, що якщо в запиті не буде значення хоча б для одного з цих атрибутів, то додаток надішле клієнту помилку. Однак ми також можемо зробити деякі атрибути необов'язковими, присвоївши їм значення *None*:


```

from fastapi import FastAPI, Body
from fastapi.responses import FileResponse
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int | None = None

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
def hello(person: Person):
    if person.age == None:
        return {"message": f"Hi, {person.name}"}
    else:
        return {"message": f"Hi, {person.name}, your age - {person.age}"}

```

У цьому разі атрибут *name* залишається обов'язковим, а атрибут *age* - необов'язковим, тому в запиті необов'язково для нього передавати значення.

Детальне налаштування атрибутів і клас Field

Для більш детального налаштування атрибутів моделі застосовується клас *pydantic.Field*. Наприклад, він дозволяє задати значення за замовчуванням і правила валдації значень за допомогою таких параметрів конструктора:

- *default*: встановлює значення за замовчуванням
- *min_length*: встановлює мінімальну кількість символів у значенні параметра
- *max_length*: встановлює максимальну кількість символів у значенні параметра
- *regex*: встановлює регулярний вираз, якому має відповідати значення параметра
- *lt*: значення параметра має бути меншим за певне значення
- *le*: значення параметра має бути меншим або дорівнювати певному значенню
- *gt*: значення параметра має бути більшим за певне значення
- *ge*: значення параметра має бути більшим або дорівнювати певному значенню

Застосуємо деякі параметри:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

class Person(BaseModel):
    name: str = Field(default="Undefined", min_length=3, max_length=20)
    age: int = Field(default=18, ge=18, lt=101)

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-1.html")

@app.post("/hello")
def hello(person: Person):
    return {"message": f"Hi, {person.name}, your age - {person.age}"}
```

У даному випадку значення параметра `name` повинно мати не менше 3 і не більше 20 символів, а параметр `age` повинен представляти число в діапазоні від 18 (включно) до 101 (не включно). Якщо в запиті не передано значення для атрибутів класу, то атрибути `name` і `age` отримують значення за замовчуванням: рядок `"Undefined"` і число 18 відповідно.

Отримання списків

Подібним чином можна отримувати список об'єктів моделі:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

@app.post("/hello")
def hello(people: list[Person]):
    return {"message": people}
```

У цьому разі для тесту ми могли б надіслати дані з коду javascript у такий спосіб:

```
const response = await fetch("/hello", {
  method: "POST",
  headers: { "Accept": "application/json", "Content-Type":
"application/json" },
  body: JSON.stringify([
    { name: "Tom", age: 38 },
    { name: "Bob", age: 41 },
    { name: "Sam", age: 25 }
  ])
});
const data = await response.json();
console.log(data);
```

Отримання вкладених списків

Модель може містити список. Наприклад, клас *Person* містить список мов, що вивчаються:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    languages: list = []

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

@app.post("/hello")
def hello(person: Person):
    return {"message": f"Name: {person.name}. Languages: {person.languages}"}
```

У цьому випадку для зберігання мов у класі *Person* визначено атрибут *languages*.

У цьому разі надсилання даних із *javascript* мало б такий вигляд:

```
const response = await fetch("/hello", {
  method: "POST",
  headers: { "Accept": "application/json", "Content-Type":
"application/json" },
  body: JSON.stringify({
    name: "Tom",
    languages: ["Python", "JavaScript"]
  })
});
const data = await response.json();
console.log(data);
```

Також у атрибута можна встановити значення за замовчуванням на випадок, якщо в запиті не міститься відповідних даних:

```
class Person(BaseModel):
    name: str
    languages: list = ["Java", "Python", "JavaScript"]
```

Вкладені моделі

Одна модель може містити іншу модель. Наприклад, користувач працює в якій-небудь компанії. І для зберігання даних компанії можна створити окрему модель - *Company*:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel

class Company(BaseModel):
    name: str

class Person(BaseModel):
    name: str
    company: Company

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

@app.post("/hello")
def hello(person: Person):
    return {"message": f"{person.name} ({person.company.name)}"}
```

Для простоти тут клас компанії має тільки один атрибут - назву компанії. Відправлення запиту в коді javascript у цьому випадку могло б виглядати так:

```
const response = await fetch("/hello", {
  method: "POST",
  headers: { "Accept": "application/json", "Content-Type":
"application/json" },
  body: JSON.stringify({
    name: "Tom",
    company: {name: "Google"}
  })
})
```

```
});
const data = await response.json();
console.log(data);
```

1.12 Створення найпростішого API

Розглянутого в минулих темах матеріалу достатньо для створення примітивного додатка. У цій темі спробуємо реалізувати найпростіший додаток Web API у стилі REST. Архітектура REST передбачає застосування таких методів або типів запитів HTTP для взаємодії із сервером, де кожен тип запиту відповідає за певну дію:

- *GET* (отримання даних)
- *POST* (додавання даних)
- *PUT* (зміна даних)
- *DELETE* (видалення даних)

Для кожного з цих типів запитів клас FastAPI надає відповідні методи. Розглянемо, як ми можемо реалізувати за допомогою цих методів найпростіший API. Нехай у нас буде такий навчальний проєкт, котрий містить *main.py* на *index-5.html*.

Створення сервера

У файлі *main.py* визначимо такий код:

```
import uuid
from fastapi import FastAPI, Body, status
from fastapi.responses import JSONResponse, FileResponse

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.id = str(uuid.uuid4())

# умовна база даних - набір об'єктів Person
people = [Person("Sarah", 38), Person("Jessica", 42), Person("Parker", 28)]

# для пошуку користувача в списку people
def find_person(id):
```

```

    for person in people:
        if person.id == id:
            return person
    return None

app = FastAPI()

@app.get("/")
async def main():
    return FileResponse("public/index-5.html")

@app.get("/api/users")
def get_people():
    return people

@app.get("/api/users/{id}")
def get_person(id):
    # отримуємо користувача за id
    person = find_person(id)
    print(person)
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person==None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )
    #якщо користувача знайдено, відправляємо його
    return person

@app.post("/api/users")
def create_person(data = Body()):
    person = Person(data["name"], data["age"])
    # додаємо об'єкт до списку people
    people.append(person)
    return person

@app.put("/api/users")
def edit_person(data = Body()):

    # отримуємо користувача за id
    person = find_person(data["id"])
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )
    # якщо користувач знайдений, змінюємо його дані і відправляємо назад клієнту
    person.age = data["age"]
    person.name = data["name"]
    return person

```

```
@app.delete("/api/users/{id}")
def delete_person(id):
    # отримуємо користувача за id
    person = find_person(id)

    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JsonResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )

    # якщо користувача знайдено, видаляємо його
    people.remove(person)
    return person
```

Розберемо в загальних рисах цей код. Насамперед для представлення даних, з якими ми будемо працювати, визначаємо клас *Person*.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
        self.id = str(uuid.uuid4())
```

Цей клас містить три атрибути. Два атрибути - *name* і *age* представлятимуть ім'я і вік користувача і будуть встановлюватися через конструктор. А третій атрибут - *id* слугуватиме для унікальної ідентифікації цього об'єкта і зберігатиме значення *guid*. Для генерації *guid* застосовується функція *uuid.uuid4()* з пакета *uuid*. У конструкторі *Person* згенерований *guid* перетворюється в рядок і присвоюється атрибуту *id*.

Для зберігання даних у додатку визначимо список *people*, який виконуватиме роль умовної бази даних і зберігатиме об'єкти *Person*.

```
# умовна база даних - набір об'єктів Person
people = [Person("Sarah", 38), Person("Jessica", 42), Person("Parker", 28)]
```

Для пошуку об'єкта *Person* у цьому списку визначено допоміжну функцію *find_person()*.

Під час звернення до кореня веб-додатка, тобто за шляхом "/", він надсилатиме у відповідь файл *index.html*, тобто веб-сторінку, за допомогою якої користувач зможе взаємодіяти із сервером:

```
@app.get("/")
async def main():
    return FileResponse("public/index-5.html")
```

Далі визначаються функції, які власне і представляють *API*. Спочатку визначається функція, яка обробляє запит типу *GET* за шляхом *"api/users"*:

```
@app.get("/api/users")
def get_people():
    return people
```

Запит *GET* передбачає отримання об'єктів, і в цьому випадку відправляємо вище визначений список об'єктів *Person*.

Коли клієнт звертається до додатка для отримання одного об'єкта за *id* у запит типу *GET* за адресою *"api/users/{id}"*, то спрацьовує інша функція:

```
@app.get("/api/users/{id}")
def get_person(id):
    # отримуємо користувача за id
    person = find_person(id)
    print(person)
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person==None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )
    #якщо користувача знайдено, відправляємо його
    return person
```

Тут через параметр *id* отримуємо зі шляху запиту ідентифікатор об'єкта *Person* і за цим ідентифікатором шукаємо потрібний об'єкт у списку *people*. Якщо об'єкт за *id* не був знайдений, то повертаємо за допомогою класу *JSONResponse* статусний код *404* з

деяким повідомленням у форматі *JSON*. Якщо об'єкт знайдено, то відправляємо знайдений об'єкт клієнту.

При отриманні запиту типу *DELETE* за маршрутом *"/api/users/{id}"* спрацьовує інша функція:

```
@app.delete("/api/users/{id}")
def delete_person(id):
    # отримуємо користувача за id
    person = find_person(id)

    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )

    # якщо користувача знайдено, видаляємо його
    people.remove(person)
    return person
```

Тут діє аналогічна логіка - якщо об'єкт за *id* не знайдений, відправляє статусний код *404*. Якщо ж об'єкт знайдено, то видаляємо його зі списку і надсилаємо клієнту.

При отриманні запиту з методом *POST* за адресою *"/api/users"* спрацьовує така функція:

```
@app.post("/api/users")
def create_person(data = Body()):
    person = Person(data["name"], data["age"])
    # додаємо об'єкт до списку people
    people.append(person)
    return person
```

Запит типу *POST* передбачає передачу застосунку даних, що надсилаються. Причому ми очікуємо, що клієнт надішле дані, які містять значення *name* і *age*. Для стислості ми поки що опускаємо валідацію вхідних даних. І для отримання даних з тіла запиту за допомогою класу *Body* отримуємо дані в параметр *data* і потім використовуємо дані з цього параметра для створення об'єкта *Person*. Потім створений об'єкт додається в список *people* і відправляється назад клієнту.

Якщо додатку надходить PUT-запит за адресою `"/api/users"`, то аналогічним чином отримуємо відправлені клієнтом дані у вигляді об'єкта *Person* і намагаємося знайти подібний об'єкт у списку *people*. Якщо об'єкт не знайдено, відправляємо статусний код 404. Якщо об'єкт знайдено, то змінюємо його дані і відправляємо назад клієнту:

```
@app.put("/api/users")
def edit_person(data = Body()):

    # отримуємо користувача за id
    person = find_person(data["id"])
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={ "message": "Користувач не знайдений" }
        )
    # якщо користувач знайдений, змінюємо його дані і відправляємо назад клієнту
    person.age = data["age"]
    person.name = data["name"]
    return person
```

Таким чином, ми визначили найпростіший *API*. Тепер додамо код клієнта.

Визначення клієнта

Тепер у проєкті визначимо папку *public*, у яку додамо новий файл *index-5.html*

Визначимо у файлі *index-5.html* наступним код для взаємодії із сервером *FastAPI*:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Hollywood</title>
</head>
<body>
    <h2>Список користувачів</h2>
    <div>
        <input type="hidden" id="userId" />
```

```

    <p>
      Ім'я:<br/>
      <input id="userName" />
    </p>
    <p>
      Вік:<br />
      <input id="userAge" type="number" />
    </p>
    <p>
      <button id="saveBtn">Зберегти</button>
      <button id="resetBtn">Скинути</button>
    </p>
  </div>
<table>
  <thead><tr><th>Ім'я</th><th>Вік</th><th></th></tr></thead>
  <tbody>
  </tbody>
</table>

<script>
// Отримання всіх користувачів
async function getUsers() {
  // надсилаємо запит і отримуємо відповідь
  const response = await fetch("/api/users", {
    method: "GET",
    headers: { "Accept": "application/json" }
  });
  // якщо запит пройшов нормально
  if (response.ok === true) {
    // отримуємо дані
    const users = await response.json();
    const rows = document.querySelector("tbody");
    // додаємо отримані елементи в таблицю
    users.forEach(user => rows.append(row(user)));
  }
}

// Отримання одного користувача
async function getUser(id) {
  const response = await fetch(`/api/users/${id}`, {
    method: "GET",
    headers: { "Accept": "application/json" }
  });
  if (response.ok === true) {
    const user = await response.json();
    document.getElementById("userId").value = user.id;
    document.getElementById("userName").value = user.name;
    document.getElementById("userAge").value = user.age;
  }
  else {
    // якщо сталася помилка, отримуємо повідомлення про помилку
    const error = await response.json();
    console.log(error.message); // і виводимо його на консоль
  }
}

```

```

    }
    // Додавання користувача
    async function createUser(userName, userAge) {

        const response = await fetch("api/users", {
            method: "POST",
            headers: { "Accept": "application/json", "Content-Type":
"application/json" },
            body: JSON.stringify({
                name: userName,
                age: parseInt(userAge, 10)
            })
        });
        if (response.ok === true) {
            const user = await response.json();
            document.querySelector("tbody").append(row(user));
        }
        else {
            const error = await response.json();
            console.log(error.message);
        }
    }
}

// Зміна користувача
async function editUser(userId, userName, userAge) {
    const response = await fetch("api/users", {
        method: "PUT",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            id: userId,
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector(`tr[data-
rowid='${user.id}']`).replaceWith(row(user));
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}

// Видалення користувача
async function deleteUser(id) {
    const response = await fetch(`/api/users/${id}`, {
        method: "DELETE",
        headers: { "Accept": "application/json" }
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector(`tr[data-rowid='${user.id}']`).remove();
    }
}

```

```

    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}

// скидання даних форми після відправлення
function reset() {
    document.getElementById("userId").value =
    document.getElementById("userName").value =
    document.getElementById("userAge").value = "";
}

// створення рядка для таблиці
function row(user) {

    const tr = document.createElement("tr");
    tr.setAttribute("data-rowid", user.id);

    const nameTd = document.createElement("td");
    nameTd.append(user.name);
    tr.append(nameTd);

    const ageTd = document.createElement("td");
    ageTd.append(user.age);
    tr.append(ageTd);

    const linksTd = document.createElement("td");

    const editLink = document.createElement("button");
    editLink.append("Змінити");
    editLink.addEventListener("click", async () => await getUser(user.id));
    linksTd.append(editLink);

    const removeLink = document.createElement("button");
    removeLink.append("Видалити");
    removeLink.addEventListener("click", async () => await deleteUser(user.id));

    linksTd.append(removeLink);
    tr.appendChild(linksTd);

    return tr;
}

// скидання значень форми
document.getElementById("resetBtn").addEventListener("click", () => reset());

// надсилання форми
document.getElementById("saveBtn").addEventListener("click", async () => {

    const id = document.getElementById("userId").value;
    const name = document.getElementById("userName").value;
    const age = document.getElementById("userAge").value;
    if (id === "")

```

```

        await createUser(name, age);
    else
        await editUser(id, name, age);
    reset();
});

// завантаження користувачів
getUsers();
</script>
</body>
</html>

```

Основна логіка тут міститься в коді *javascript*. Під час завантаження сторінки в браузері отримуємо всі об'єкти з БД за допомогою функції *getUsers()*:

```

// Отримання всіх користувачів
async function getUsers() {
    // надсилаємо запит і отримуємо відповідь
    const response = await fetch("/api/users", {
        method: "GET",
        headers: { "Accept": "application/json" }
    });
    // якщо запит пройшов нормально
    if (response.ok === true) {
        // отримуємо дані
        const users = await response.json();
        const rows = document.querySelector("tbody");
        // додаємо отримані елементи в таблицю
        users.forEach(user => rows.append(row(user)));
    }
}

```

Для додавання рядків у таблицю використовується функція *row()*, яка повертає рядок. У цьому рядку будуть визначені посилання для зміни та видалення користувача.

Посилання для зміни користувача за допомогою функції *getUser()* отримує із сервера виділеного користувача:

```
// Отримання одного користувача
async function getUser(id) {
    const response = await fetch(`/api/users/${id}`, {
        method: "GET",
        headers: { "Accept": "application/json" }
    });
    if (response.ok === true) {
        const user = await response.json();
        document.getElementById("userId").value = user.id;
        document.getElementById("userName").value = user.name;
        document.getElementById("userAge").value = user.age;
    }
    else {
        // якщо сталася помилка, отримуємо повідомлення про помилку
        const error = await response.json();
        console.log(error.message); // і виводимо його на консоль
    }
}
```

І виділений користувач додається у форму над таблицею. Ця ж форма застосовується і для додавання об'єкта. За допомогою прихованого поля, яке зберігає *id* користувача, ми можемо дізнатися, яка дія виконується - додавання або редагування. Якщо *id* не встановлений (дорівнює порожньому рядку), то виконується функція *createUser*, яка відправляє дані в *POST*-запиті:

```
// Додавання користувача
async function createUser(userName, userAge) {

    const response = await fetch("api/users", {
        method: "POST",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector("tbody").append(row(user));
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}
```

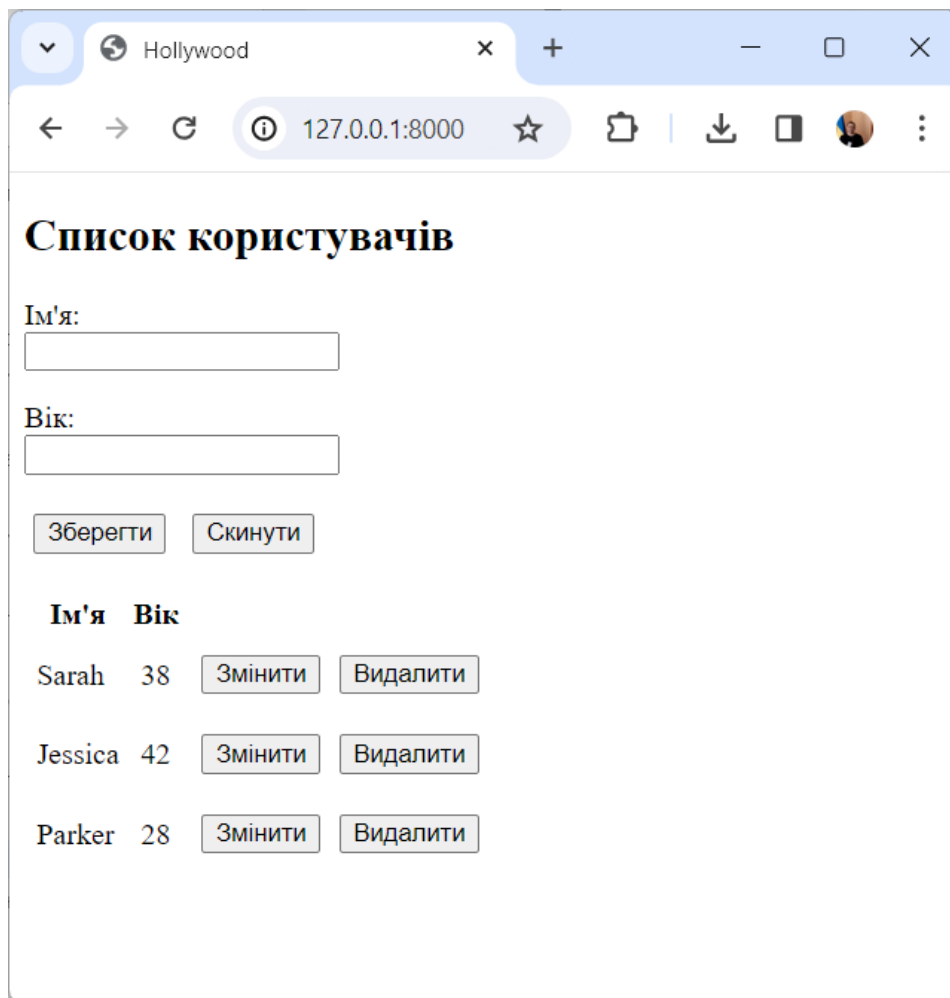

Якщо ж раніше користувача було завантажено на форму, і в прихованому полі зберігся його *id*, то виконується функція *editUser*, яка відправляє *PUT*-запит:

```
// Зміна користувача
async function editUser(userId, userName, userAge) {
  const response = await fetch("api/users", {
    method: "PUT",
    headers: { "Accept": "application/json", "Content-Type":
"application/json" },
    body: JSON.stringify({
      id: userId,
      name: userName,
      age: parseInt(userAge, 10)
    })
  });
  if (response.ok === true) {
    const user = await response.json();
    document.querySelector(`tr[data-rowid='${user.id}']`).replaceWith(row(user));
  }
  else {
    const error = await response.json();
    console.log(error.message);
  }
}
```

І функція *deleteUser()* посилає на сервер запит типу *DELETE* на видалення користувача, і в разі успішного видалення на сервері видаляє об'єкт за *id* зі списку об'єктів *Person*.

```
// Видалення користувача
async function deleteUser(id) {
  const response = await fetch(`/api/users/${id}`, {
    method: "DELETE",
    headers: { "Accept": "application/json" }
  });
  if (response.ok === true) {
    const user = await response.json();
    document.querySelector(`tr[data-rowid='${user.id}']`).remove();
  }
  else {
    const error = await response.json();
    console.log(error.message);
  }
}
```

Тепер запустимо проєкт, і за замовчуванням додаток надішле браузеру веб-сторінку *index-5.html*, яка завантажить список об'єктів:



Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
Sarah	38	Змінити	Видалити
Jessica	42	Змінити	Видалити
Parker	28	Змінити	Видалити

Рисунок 1.48 – Форма додатку

Після цього ми зможемо виконувати всі базові операції з користувачами - отримання, додавання, зміна, видалення.

Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
Sarah	38	Змінити	Видалити
Jessica	42	Змінити	Видалити
Parker	28	Змінити	Видалити

Рисунок 1.49 – Редагування інформації

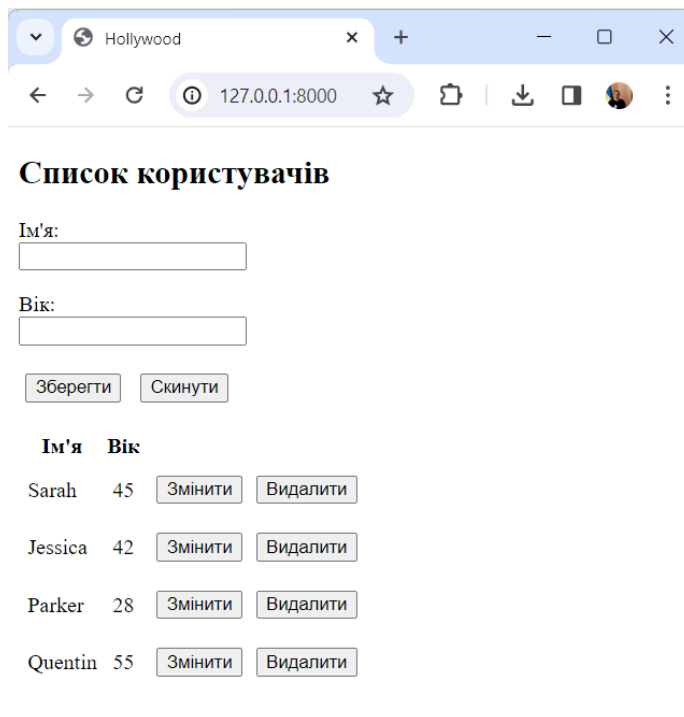
Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
Sarah	45	Змінити	Видалити
Jessica	42	Змінити	Видалити
Parker	28	Змінити	Видалити

Рисунок 1.50 – Додавання нового користувача



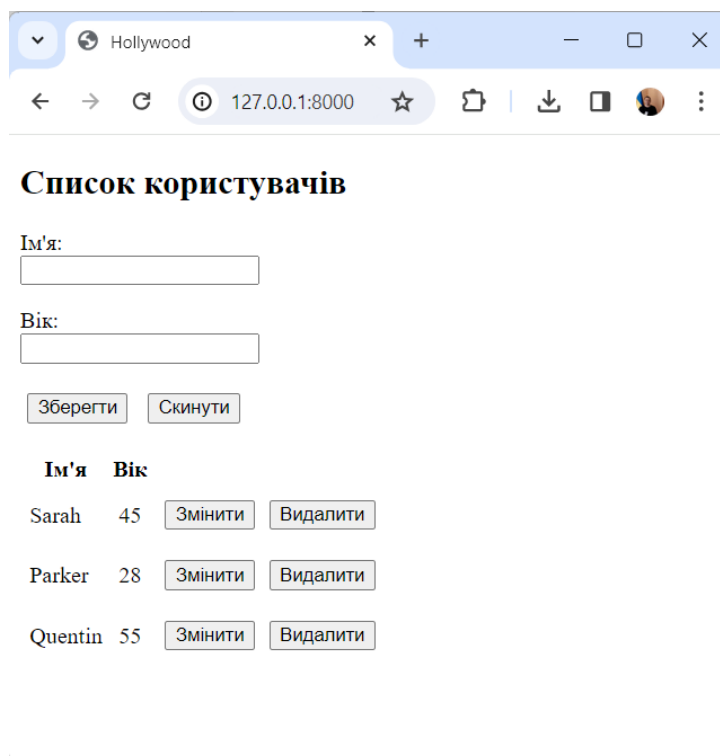
Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
Sarah	45	Змінити	Видалити
Jessica	42	Змінити	Видалити
Parker	28	Змінити	Видалити
Quentin	55	Змінити	Видалити

Рисунок 1.51 – Нового користувача додано



Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
Sarah	45	Змінити	Видалити
Parker	28	Змінити	Видалити
Quentin	55	Змінити	Видалити

Рисунок 1.52 – Видалення користувача

1.13 Надсилання та отримання заголовків

Надсилання заголовків

Для надсилання заголовка в конструктор класу *Response* або його спадкоємців параметру *headers* передають словник, де ключі представляють назви заголовків:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/")
def root():
    data = "Hello WebPython!"
    return Response(content=data, media_type="text/plain", headers={"Secret-Code" :
"password"})
```

Для прикладу в цьому випадку клієнту надсилається кастомний заголовок "Secret-Code" зі значенням "password":

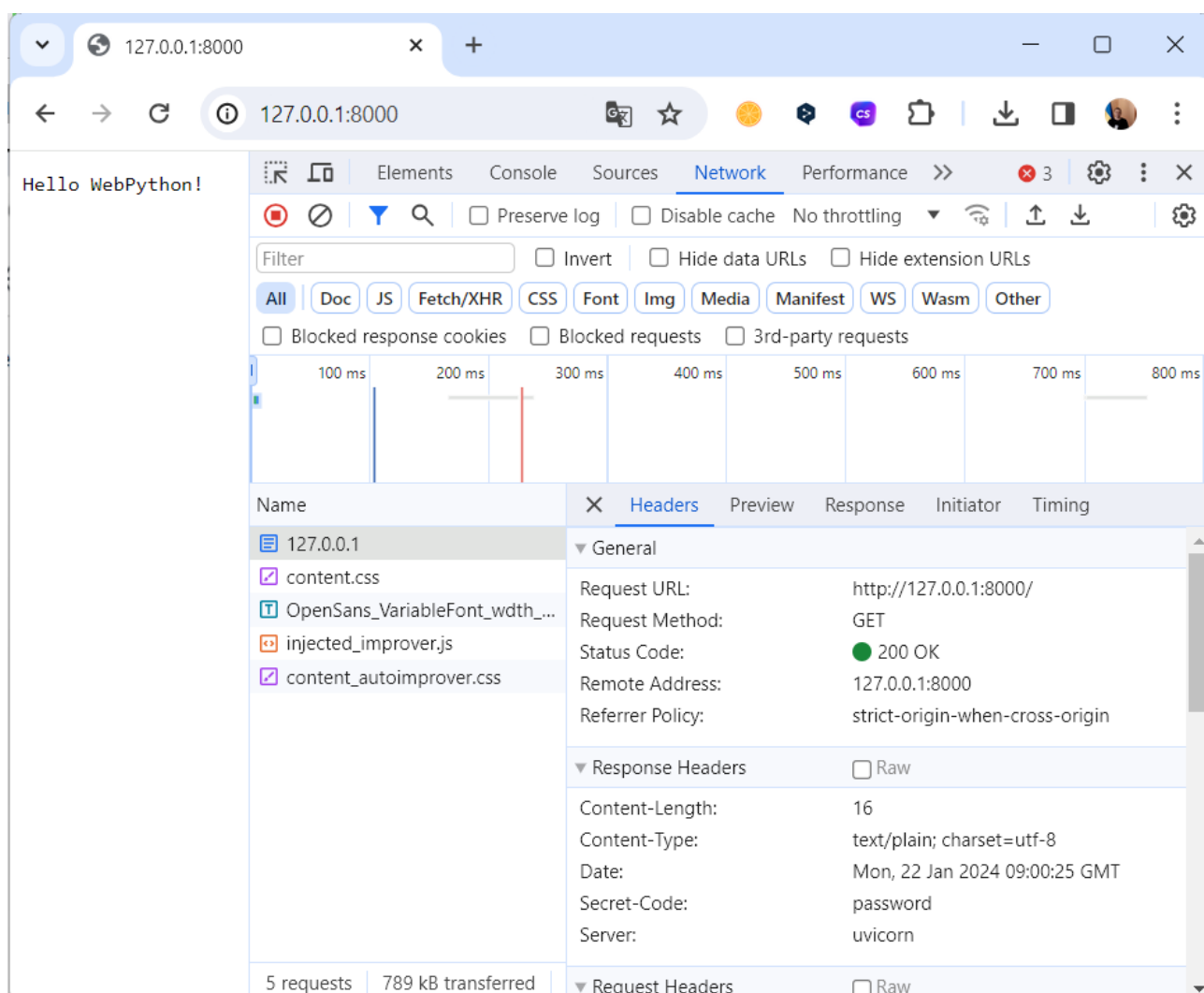


Рисунок 1.53 – Відправлення заголовку

Також можна задати заголовки за допомогою атрибута *headers*, який є у класу *Response* і його спадкоємців. Цей атрибут фактично являє собою словник, де ключі - назви заголовків:

```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/")
def root(response: Response):
    response.headers["password"] = "qwerty"
    return {"message": "Hello WebPython!"}
```

За такого підходу об'єкт *Response* явно передається у функцію-обробник через параметр.

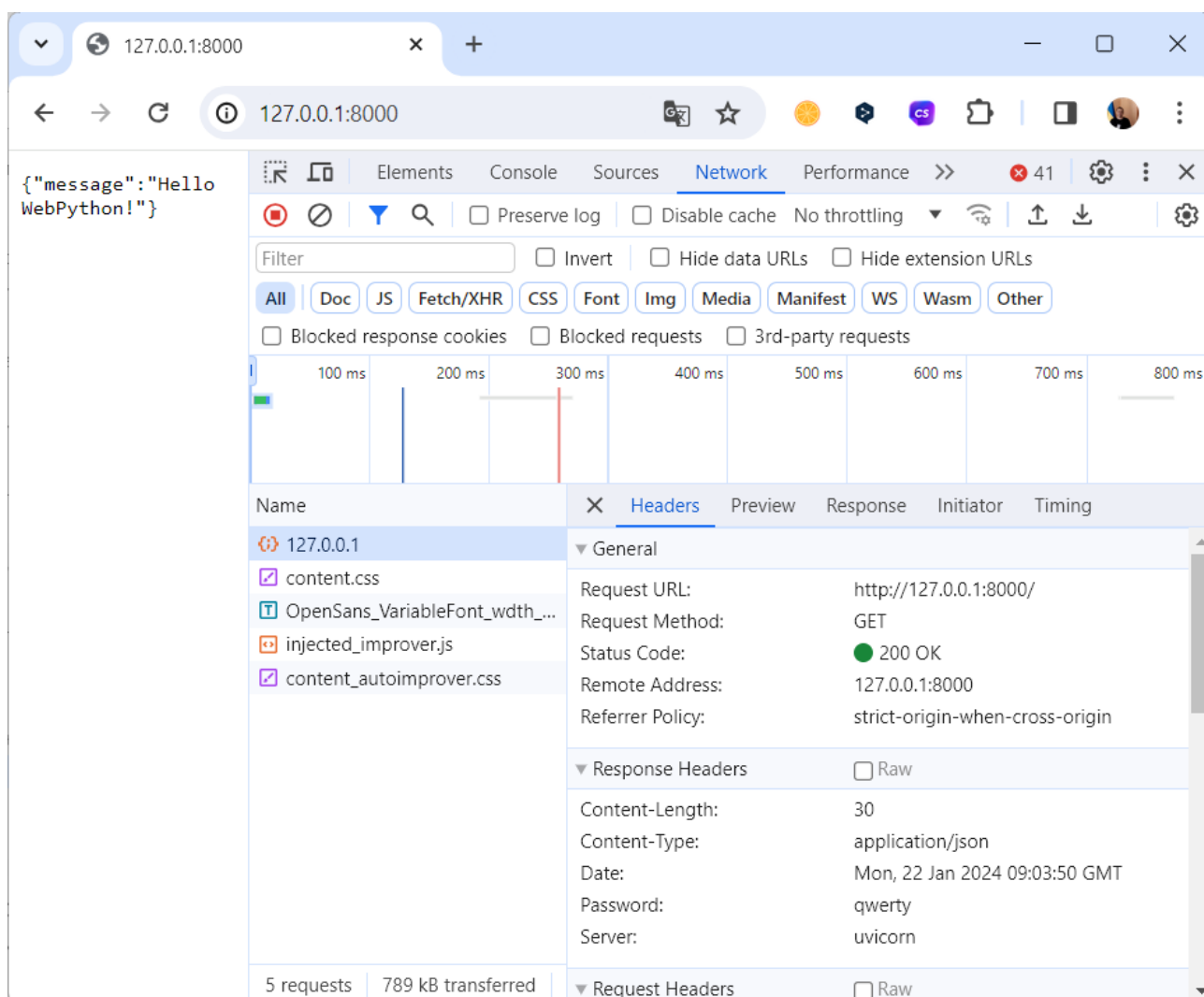


Рисунок 1.54 – Відповідь серверу

Отримання заголовків

Для отримання заголовків запиту застосовується клас *fastapi.Header*. Наприклад, отримаємо заголовок *User-Agent*:

```
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/")
def root(user_agent: str = Header()):
    return {"User-Agent": user_agent}
```

При цьому параметр функції, у який отримуємо значення заголовка, має називатися як і заголовок за тим винятком, що замість дефіса застосовується прочерк: параметр *"user_agent"* відповідає заголовку *"User-Agent"*.

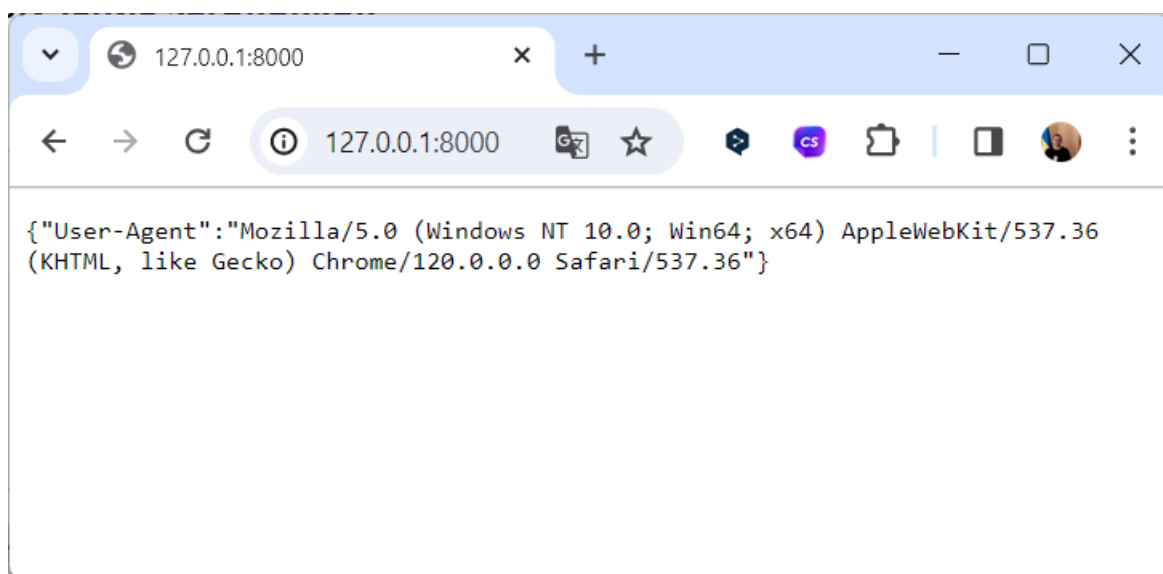


Рисунок 1.55 – Відповідь серверу

Але в запиті може бути відсутній потрібний заголовок. У цьому разі ми можемо встановити для нього значення за замовчуванням, наприклад, *None*

```
from fastapi import FastAPI, Header

app = FastAPI()

@app.get("/")
def root(personal_data: str | None = Header(default=None)):
    return {"Personal-Data": personal_data}
```

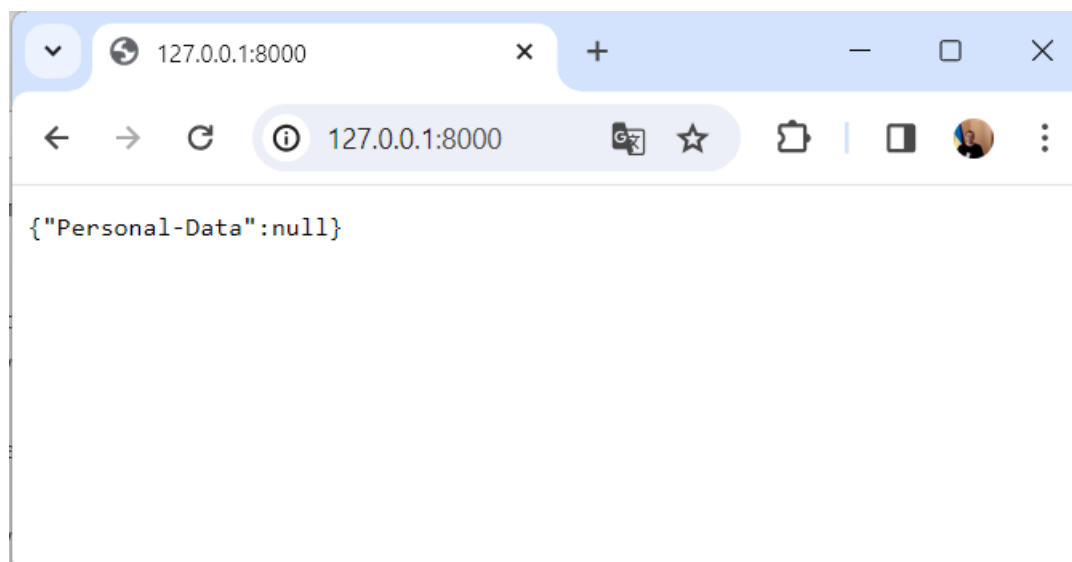


Рисунок 1.56 – Відповідь серверу

1.14 Кукі

Встановлення кукі

Для встановлення кукі на сервері в об'єкта `Response` і його класів-спадкоємців застосовується метод `set_cookie()`. Цей метод приймає низку параметрів:

- *key*: ключ або ім'я куки. Обов'язковий параметр
- *value*: значення куки, значення за замовчуванням - порожній рядок
- *max_age*: максимальний час життя куки в секундах. Це може бути або число, або значення *None* (обмежує час життя куки поточною сесією браузера, є значенням за замовчуванням).
- *expires*: коли закінчується дія куки. Це може бути або число, або значення *None* (обмежує час життя куки поточною сесією браузера, є значенням за замовчуванням).

- *path*: шлях, для якого встановлюються куки. Значення за замовчуванням - корінь веб-додатка `"/"`
- *domain*: домен, до якого застосовуються куки. значення за замовчуванням *None*
- *secure*: встановлює використовуваний протокол. Так, якщо має значення *True*, то куки надсилатимуться на сервер тільки в запиті за протоколом *https*. Значення за замовчуванням *False*
- *httponly*: встановлює доступність для скриптів *javascript* на клієнті. Так, значення *True* запобігає доступу до куки з коду *javascript* на клієнті. Значення за замовчуванням *False*
- *samesite*: встановлює дозволи на надсилання куки в кросдоменних запитах. Так, значення *lax* (значення за замовчуванням) вказують браузеру не надсилати куки в кросдоменних запитах.

Установимо куку, яку назовемо `"last_visit"`:

```
from fastapi import FastAPI, Response
from datetime import datetime

app = FastAPI()

@app.get("/")
def root(response: Response):
    now = datetime.now()    # отримуємо поточну дату та час
    response.set_cookie(key="last_visit", value=now)
    return {"message": "куки встановлено"}
```

У цьому випадку кука `"last_visit"` представляє дату останнього візиту і зберігає поточну дату. Потім у браузері через інструменти розробника ми можемо побачити параметри цієї куки.

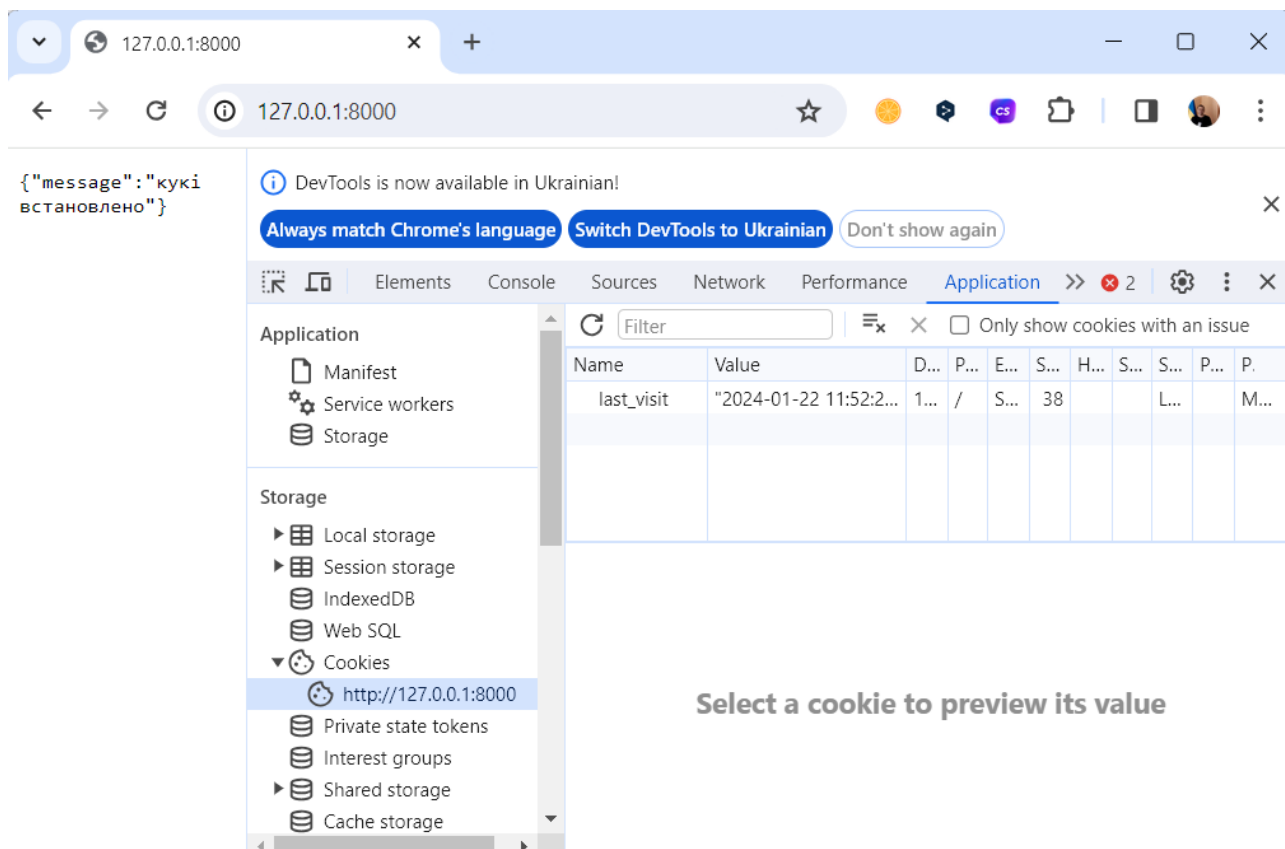


Рисунок 1.57 – Відповідь сервера та встановлення куки

У прикладі вище об'єкт *Response* передавався як параметр. Але також можна встановити куку і за допомогою явно створеного об'єкта *Response*:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
from datetime import datetime

app = FastAPI()

@app.get("/")
def root():
    now = datetime.now() # отримуємо поточну дату та час
    response = JSONResponse(content={"message": "кукі встановлено"})
    response.set_cookie(key="last_visit", value=now)
    return response
```

Отримання куки

Для отримання куки на сервері застосовується клас *fastapi.Cookie*. Наприклад, отримаємо вище встановлену куку "last_visit":

```
from fastapi import FastAPI, Cookie

app = FastAPI()

@app.get("/")
def root(last_visit = Cookie()):
    return {"last_visit": last_visit}
```

Для отримання куки визначається параметр з ім'ям куки, якому присвоюється об'єкт *Cookie*.

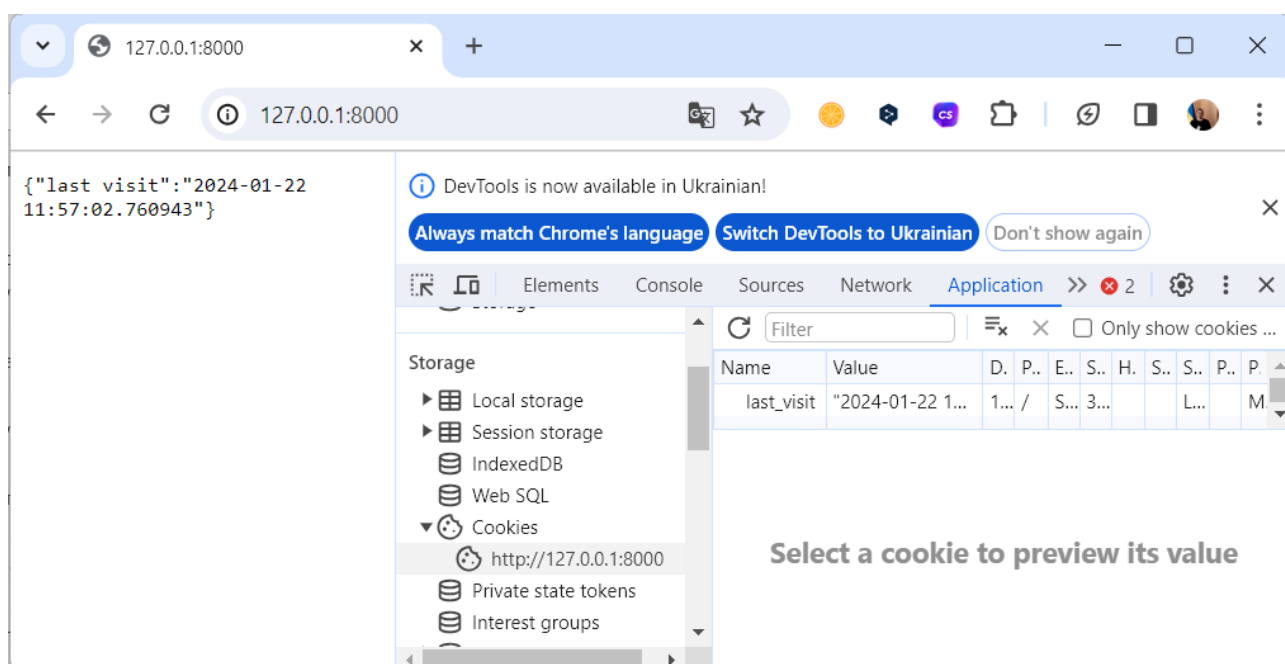


Рисунок 1.58 – Відповідь сервера на отримання куки

У прикладі вище куки була обов'язковою. Якщо її немає в запиті, додаток видав би помилку. Щоб уникнути цього, ми можемо визначити значення за замовчуванням, зокрема `None`:

```

from fastapi import FastAPI, Cookie

app = FastAPI()

@app.get("/")
def root(last_visit: str | None = Cookie(default=None)):
    if last_visit == None:
        return {"message": "Це ваше перше відвідування"}
    else:
        return {"message": f"Ваше останнє відвідування: {last_visit}"}

```

1.15 Надсилання форм

Робота з формами дещо відрізняється від отримання на сервері даних у тілі запиту. Насамперед необхідно встановити модуль *python-multipart* за допомогою команди

```
pip install python-multipart
```

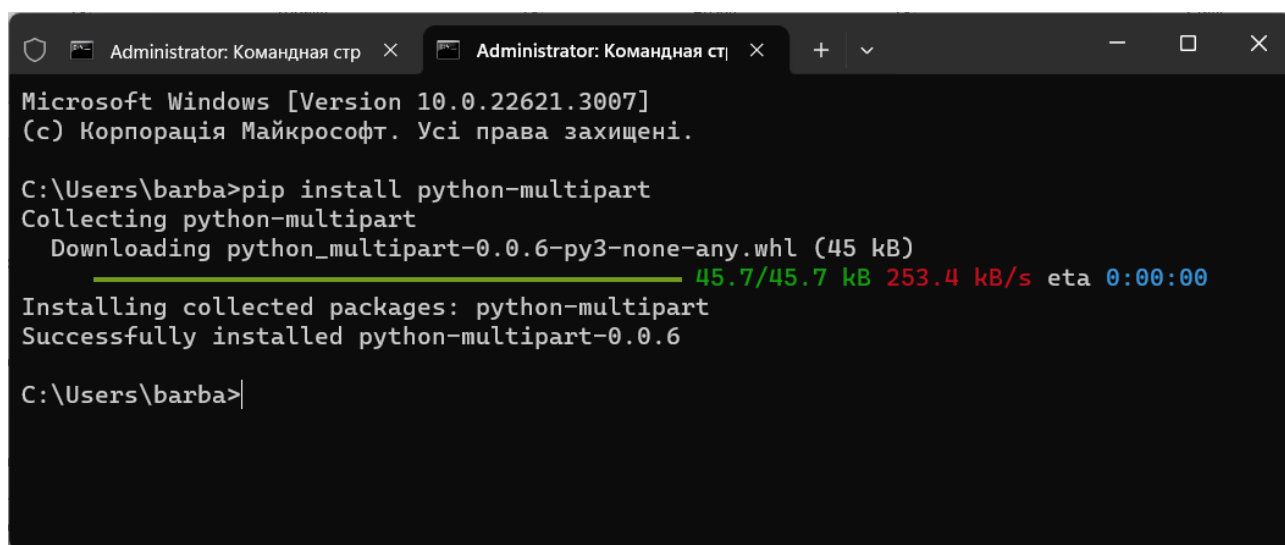


Рисунок 1.59 – Встановлення пакету *python-multipart*

Безпосередньо в самому FastAPI для отримання даних відправлених форм застосовується клас *fastapi.Form*.

Нехай у нас буде навчальний проєкт з типовою структурою: *main-form.py* та *\public\index-6.html*.

У папці *public* визначимо файл *index-6.html* з таким кодом:

```

<!DOCTYPE html>
<html>
<head>
    <title> WebPython. FastAPI example #6</title>

    <meta charset="utf-8" />
</head>
<body>
    <h2>Форма вводу</h2>
    <form action="postdata" method="post">
        <p>
            Name:<br>
            <input name="username" />
        </p>
        <p>
            Age:<br>
            <input name="userage" type="number" />
        </p>
        <input type="submit" value="Send" />
    </form>
</body>
</html>

```

Тут визначена найпростіша форма для введення імені та віку. І після натискання на кнопку дані надсилаються в запиті *POST* за адресою *"postdata"*.

Для отримання даних форм у файлі *main-form.py* визначимо такий код:

```

from fastapi import FastAPI, Form
from fastapi.responses import FileResponse

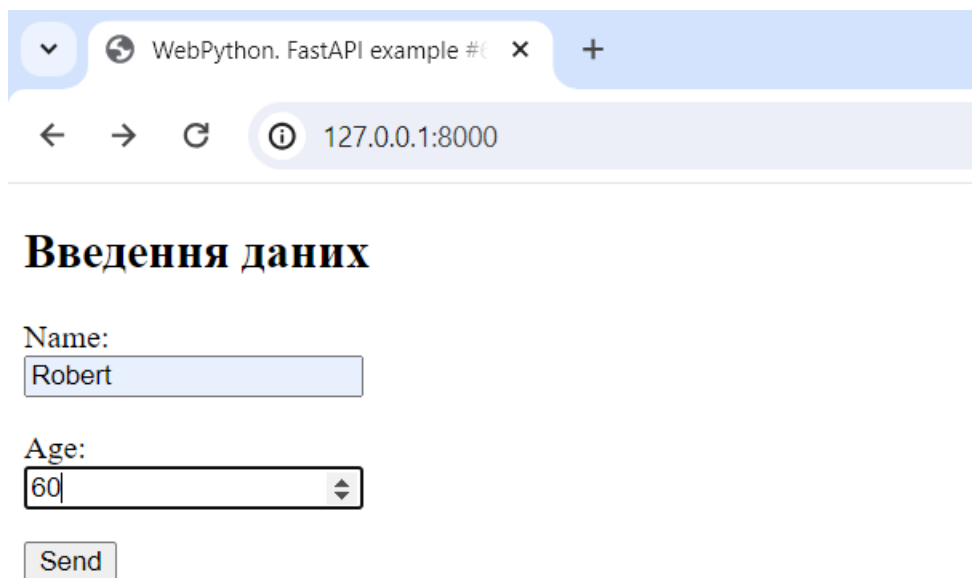
app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-6.html")

@app.post("/postdata")
def postdata(username = Form(), userage=Form()):
    return {"name": username, "age": userage}

```

У функції *postdata*, яка обробляє запит за однойменним шляхом, через параметри отримуємо відправлені дані. Причому параметри називаються так само, як і атрибути *name* у полів форми. А самим параметрам присвоюється об'єкт *Form*.



WebPython. FastAPI example #6

127.0.0.1:8000

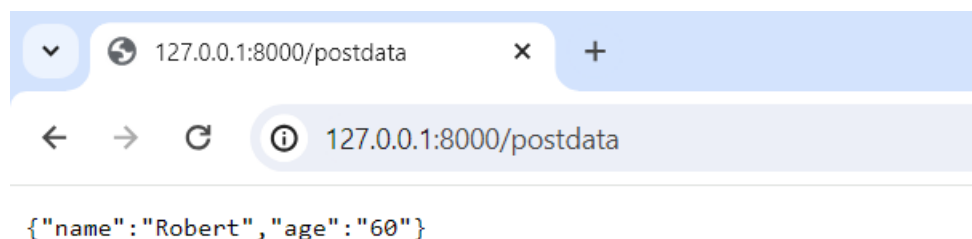
Введення даних

Name:
Robert

Age:
60

Send

Рисунок 1.60 – Відправлення форми



127.0.0.1:8000/postdata

127.0.0.1:8000/postdata

```
{"name": "Robert", "age": "60"}
```

Рисунок 1.61 – Отримання відповіді серверу

Валідація

Клас *Form* з пакета *fastapi* надає можливості валідації значень форм за допомогою деяких параметрів конструктора:

- *min_length*: встановлює мінімальну кількість символів у значенні параметра
- *max_length*: встановлює максимальну кількість символів у значенні параметра

– *regex*: встановлює регулярний вираз, якому має відповідати значення параметра

- *lt*: значення параметра має бути меншим за певне значення
- *le*: значення параметра має бути меншим або дорівнювати певному значенню
- *gt*: значення параметра має бути більшим за певне значення
- *ge*: значення параметра має бути більшим або дорівнювати певному значенню

Застосуємо деякі параметри:

```
from fastapi import FastAPI, Form
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

@app.post("/postdata")
def postdata(username: str = Form(min_length=2, max_length=20),
              usage: int = Form(ge=18, lt=101)):
    return {"name": username, "age": usage}
```

У цьому випадку параметр *username* повинен мати не менше 2 і не більше 20 символів, а параметр *usage* повинен представляти число в діапазоні від 18 (включно) до 101 (не включно)

Значення за замовчуванням

За допомогою параметра *default* конструктора *Form* можна встановити значення за замовчуванням на випадок, якщо у вхідних даних відсутні відповідні значення:

```
from fastapi import FastAPI, Form
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index.html")

@app.post("/postdata")
def postdata(username: str = Form(default="Undefined", min_length=2, max_length=20),
             userage: int = Form(default=18, ge=18, lt=111)):
    return {"name": username, "age": userage}
```

Надсилання списків

За допомогою форм можуть надсилатися набори даних. Наприклад, змінимо файл `index-6.html` таким чином:

```
<!DOCTYPE html>
<html>
<head>
  <title>WebPython. FastAPI example #6</title>
  <meta charset="utf-8" />
</head>
<body>
  <h2>Введення даних</h2>
  <form action="postdata" method="post">
    <p>
      Name:<br>
      <input name="username" />
    </p>
    <p>
      Languages:<br>
      <input name="languages" /><br><br>
      <input name="languages" /><br><br>
      <input name="languages" /><br><br>
    </p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

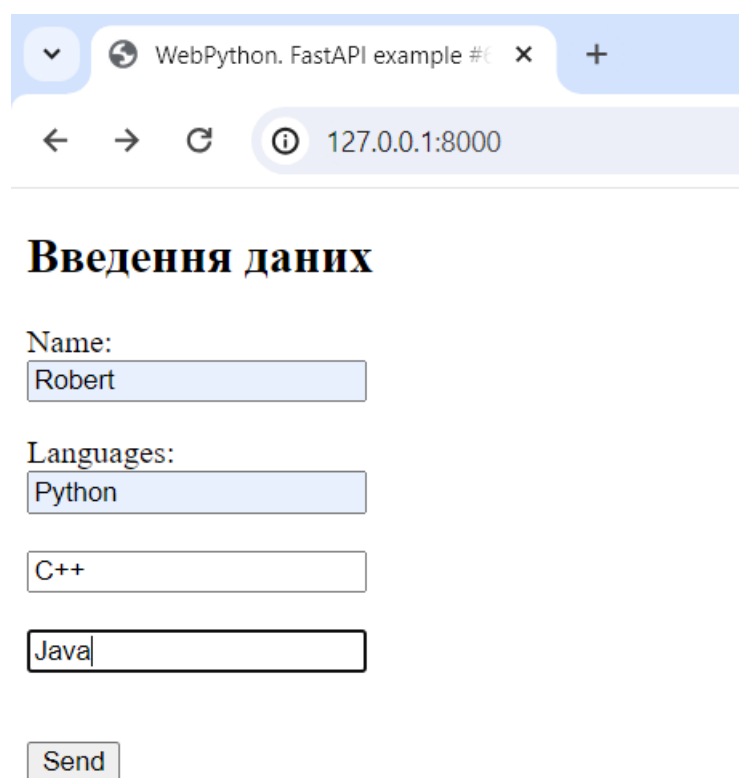

Тут на формі визначено набір елементів введення, які мають одне й те саме ім'я - *"languages"*. Під час надсилання форми зі значень цих елементів формуватиметься набір. Для отримання цього набору в коді сервера визначимо відповідний параметр як параметр типу *list*:

```
from fastapi import FastAPI, Form
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/")
def root():
    return FileResponse("public/index-6.html")

@app.post("/postdata")
def postdata(username: str = Form(),
              languages: list = Form()):
    return {"name": username, "languages": languages}
```



WebPython. FastAPI example #6

127.0.0.1:8000

Введення даних

Name:

Languages:

Рисунок 1.62 – Відправка форми

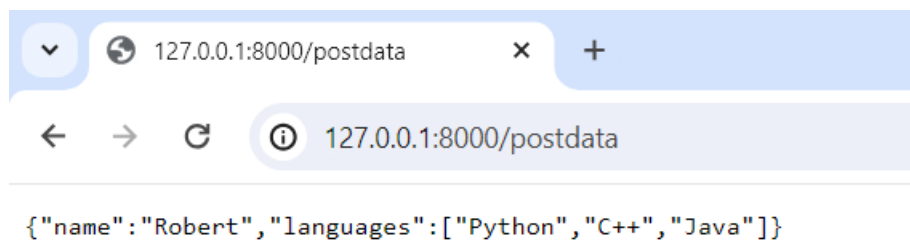


Рисунок 1.63 – Отримання відповіді від серверу

2 Робота з базою даних через SQLAlchemy

2.1 Підключення до бази даних і створення таблиць

FastAPI підтримує роботу з найрізноманітнішими системами баз даних: PostgreSQL, MySQL, SQLite, Oracle, Microsoft SQL Server тощо. Причому ми не обмежені тільки реаліційними базами даних і так само можемо використовувати і нереляційні, так звані NoSQL-системи баз даних.

Під час роботи з базами даних найпростішим рішенням є використання спеціальних інструментів - ORM (Object Relational Mapper), що дають змогу абстрагуватися від будови бази даних у конкретній СУБД і дають змогу автоматично зв'язати сутності в коді Python із таблицями в базі даних. У FastAPI найбільш поширеним ORM-інструментом є SQLAlchemy ORM

Розглянемо роботу з базою даних через SQLAlchemy ORM на прикладі SQLite, оскільки SQLite не потребує встановлення сервера чи якихось додаткових інструментів. Крім того, перейти від однієї СУБД до іншої не складе особливих труднощів і майже не потребуватиме зміни коду взаємодії з БД, за винятком кількох рядків коду.

Насамперед нам треба встановити сам SQLAlchemy за допомогою команди

```
pip install sqlalchemy
```

Визначення рядка підключення до БД

Насамперед для під'єднання до бази даних потрібен рядок під'єднання, що містить адресу бази даних та інші різні параметри, необхідні для під'єднання. Для різних СУБД рядок підключення може відрізнятися. Приклад рядка підключення до SQLite:

```
# рядок підключення  
SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"
```

Для *sqlite* рядок підключення починається з *sqlite://*, потім іде відносний або абсолютний шлях до файлу:

```
sqlite:///відносний_шлях/file.db
sqlite:///абсолютний_шлях/file.db
```

У цьому випадку це *"/sql_app.db"*, який вказує, що файл бази даних називатиметься *"sql_app.db"* і розташовуватиметься в поточній папці.

Залежно від конкретної СУБД рядок підключення буде відрізнятися. Так, якби ми підключилися б до бази даних PostgreSQL, то рядок підключення мав би такий формат:

```
SQLALCHEMY_DATABASE_URL =
"postgresql://user:password@postgresserver/db"
```

де *user* - ім'я користувача на сервері *PostgreSQL*, *password* - пароль, *postgresserver* - адреса сервера, а *db* - ім'я бази даних на сервері.

Під час під'єднання до інших систем баз даних, по суті, це єдиний рядок коду, який потребуватиме змін.

Визначення рушія

Після визначення рядка підключення створюється рушій *SQLAlchemy* за допомогою функції *create_engine()*:

```
from sqlalchemy import create_engine

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"

# створення движка
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread":
False}
)
```

Перший параметр функції вказує на рядок підключення - тобто по суті базу даних, до якої йде підключення.

Другий параметр - додаткові параметри підключення у вигляді словника *connect_args*. Зокрема, елемент *"check_same_thread": False* призначений тільки для *SQLite* і вказує, що для взаємодії з базою даних *SQLite* в рамках одного запиту може використовуватися більше одного потоку. Навіщо це треба? За замовчуванням *SQLite* дозволяє взаємодіяти з БД тільки одному потоку, щоб один потік обробляв окремий запит, що дозволяє запобігти використанню одного потоку для різних запитів. Але в *FastAPI* в рамках одного і того ж запиту з базою даних можуть взаємодіяти більше одного потоку. Власне тому необхідне подібне налаштування для отримання необхідних дозволів. При цьому кожен запит буде отримувати свій власний сеанс підключення до бази даних через механізм впровадження залежностей, тому в механізмі за замовчуванням немає необхідності.

Під час роботи з іншими СУБД достатньо вказати тільки адресу підключення:

```
engine = create_engine(SQLALCHEMY_DATABASE_URL)
```

Визначення моделей

Моделі представляють класи, які відповідають визначенню таблиць у базі даних і об'єкти яких зберігаються в цих таблицях. І однією з переваг *SQLAlchemy* є те, що ми можемо працювати таблицями через ці моделі-класи мови *Python*, не вдаючись до створення запитів мовою SQL.

Для створення моделей необхідна базова модель, від якої потім успадковуються інші моделі. Для створення базової моделі можна використовувати різні способи, але найкоротший - створення класу моделі за допомогою функції *declarative_base()*:

```
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()
```

Тобто в цьому випадку визначається клас *Base* - базова модель. Потім уже можна визначити конкретні моделі, дані яких зберігатимуться в БД. Наприклад:

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Column, Integer, String

Base = declarative_base()

class Person(Base):
    __tablename__ = "people"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer,)
```

Тут визначено модель *Person*, яка представляє користувача. Вона успадковується від *Base* і визначає низку атрибутів.

Атрибут `__tablename__` зберігає ім'я таблиці, з якою буде зіставлятися поточна модель. Тобто дані класу *Person* зберігатимуться в таблиці *"people"*.

Для зберігання даних у класі моделі визначаються атрибути, які зіставляються зі стовпцями таблиці. Такі атрибути як значення отримують об'єкт *Column*. У даному випадку клас *Person* визначає три таких атрибути: *id*, *name* і *age*, для яких у таблиці *people* будуть створюватися однойменні стовпці.

Клас *Column* за допомогою параметрів конструктора визначає налаштування стовпця в таблиці. Тут у конструктор спочатку передається тип стовпця. Так, у *id* і *age* тип стовпця - *Integer*, тобто ціле число. А стовець *name* представляє тип *String*, тобто рядок.

Крім того, в конструктор можна передати низку додаткових параметрів. Так, для стовпця *id* передаються параметри *primary_key* та *index*. Значення *primary_key=True*

вказує, що цей стовпець буде представляти первинний ключ. А параметр *index=True* свідчить, що для цього стовпця створюватиметься індекс.

Створення бази даних і таблиць

Для створення бази даних і таблиць за метаданими моделей застосовується метод *Base.metadata.create_all()*. Його ключовий параметр - *bind* приймає клас, який використовується для підключення до бази даних. Як такий клас застосовується створений раніше рушій *SQLAlchemy*. Якщо база даних і всі необхідні таблиці вже є, то метод не створює заново таблиці.

Подивимося на найпростішому прикладі. Усе об'єднаємо і визначимо у файлі додатка такий код:

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Column, Integer, String

from fastapi import FastAPI

# рядок підключення
SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"

# створюємо рушій SQLAlchemy
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread":
False}
)
#створюємо базовий клас для моделей
Base = declarative_base()

# створюємо модель, об'єкти якої зберігатимуться в бд
class Person(Base):
    __tablename__ = "people"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer,)
```

```
# створюємо таблиці
Base.metadata.create_all(bind=engine)

# додаток, який нічого не робить
app = FastAPI()
```

Після запуску програми в папці проєкту буде створено таблицю `sql_app.db`:

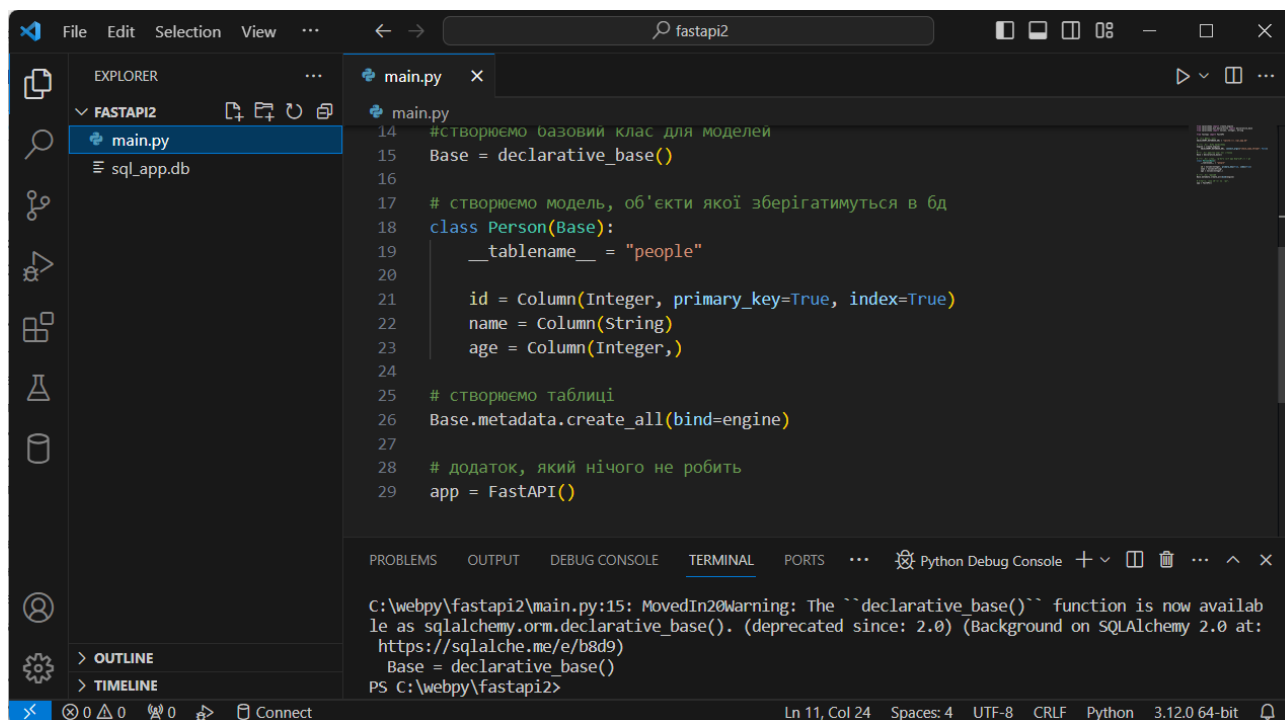


Рисунок 2.1 – Підключення до бази даних SQLite і створення таблиць через SQLAlchemy в FastAPI

Ми можемо відкрити цю базу даних за допомогою будь-якої програми для перегляду баз даних *sqlite* (наприклад, за допомогою *DB Browser for SQLite*) і побачити вміст БД:

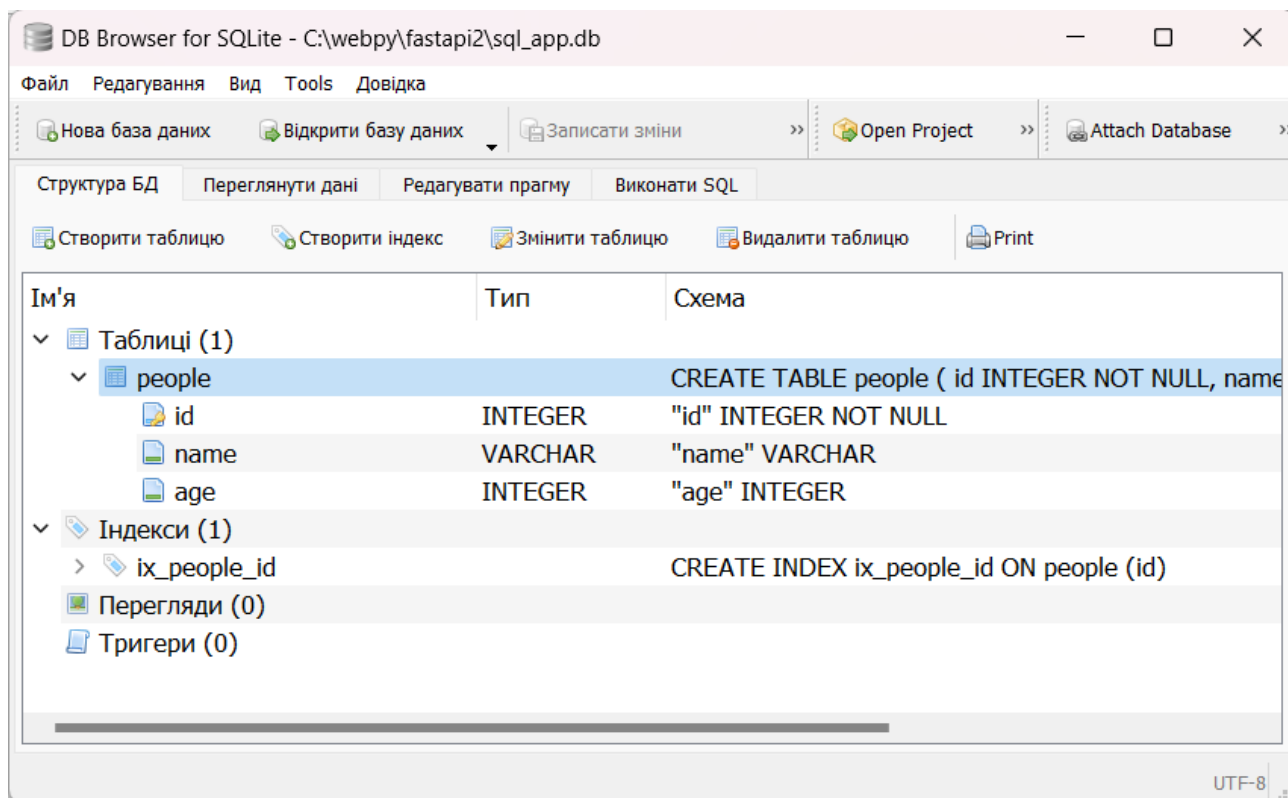


Рисунок 2.2 – Результат створення БД через SQLAlchemy у FastAPI

2.2 Взаємодія з базою даних

Створення сесії бази даних

Для взаємодії з базою даних необхідно створити сесію бази даних, яка представляє об'єкт `sqlalchemy.orm.Session`. Через цей об'єкт йде вся робота з БД. Але для цього спочатку треба створити клас-побудовник `Session` за допомогою функції-фабрики `sessionmaker()`

```
from sqlalchemy.orm import sessionmaker
SessionLocal = sessionmaker(autoflush=False, bind=engine)
```

Ця функція приймає низку параметрів, зокрема, тут застосовується два параметри:

- autoflush: при значенні True (значення за замовчуванням) буде автоматично викликатися метод `Session.flush()`, який записує всі зміни в базу даних
- bind: прив'язує сесію бд до певного рушія, який застосовується для встановлення підключення

Результатом функції є клас *SessionLocal*. Після цього можна створити об'єкт цього класу і через нього взаємодіяти в БД:

```
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(autoflush=False, bind=engine)
db = SessionLocal()
```

Тут `db` якраз представляє об'єкт *Session*.

CRUD

Розглянемо найпростіші операції CRUD (Create-Read-Update-Delete).

Додавання даних

Для додавання в базу даних необхідно спочатку створити об'єкт моделі, який передається в метод `add()` об'єкта *Session*. Після додавання для підтвердження змін у об'єкта *Session* викликається метод `commit()`:

```
# створюємо об'єкт Person для додавання в бд
alice = Person(name="Alice", age=18)
db.add(alice) # додаємо в бд
db.commit() # зберігаємо зміни
```

Наприклад, визначимо у файлі додатка такий код:

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
from sqlalchemy import Column, Integer, String

from fastapi import FastAPI

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})

# створюємо модель, об'єкти якої зберігатимуться в бд
Base = declarative_base()
class Person(Base):
    __tablename__ = "people"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer,)

# створюємо таблиці
Base.metadata.create_all(bind=engine)

# створюємо сесію підключення до бд
SessionLocal = sessionmaker(autoflush=False, bind=engine)
db = SessionLocal()

# створюємо об'єкт Person для додавання в бд
alice = Person(name="Alice", age=38)
db.add(alice) # додаємо в бд
db.commit() # зберігаємо зміни

print(alice.id) # можна отримати встановлений id

# додаток, який нічого не робить
app = FastAPI()

```

У цьому випадку створюється об'єкт Person, який додається в БД. Якщо після цього ми відкриємо БД, то зможемо побачити доданий об'єкт:

Додавання даних у БД

Варто зазначити, що в даному випадку в класі *Person* атрибут *id* виступає первинним ключем і генерується в самій БД під час додавання рядка в таблицю. Але після додавання об'єкта ми можемо отримати значення цього атрибута:

```
print(alice.id)
```

Слід зазначити, що після додавання або оновлення об'єкта, якщо ми хочемо використовувати цей об'єкт, звертатися до його атрибутів, то бажано, а іноді може бути необхідно, використовувати метод *refresh()*, який оновлює стан об'єкта:

```
# створюємо об'єкт Person для додавання в бд
alice = Person(name="Alice", age=18)
db.add(alice) # додаємо в бд
db.commit() # зберігаємо зміни
db.refresh(alice) # оновлюємо стан об'єкта
print(alice.id) # можна отримати встановлений id
```

Подібним чином можна додати кілька об'єктів:

```
rabbit = Person(name="The White Rabbit", age=30)
mouse = Person(name="The Mouse", age=5)
db.add(rabbit)
db.add(mouse)
db.commit()
```

Причому метод *commit()* викликається тільки один раз.

Однак якщо треба додати кілька об'єктів, то простіше застосувати метод *add_all()*, який додає список об'єктів:

```
dodo = Person(name="The Dodo", age=33)
lory = Person(name="The Lory", age=10)
db.add_all([dodo, lory])
db.commit()
```

Отримання даних

Для отримання об'єктів із бази даних спочатку в об'єкта *Session* необхідно викликати метод *query()* - у нього передається тип моделі, дані якої необхідно отримати:

```
db.query(Person)
```

Але цей метод просто створює об'єкт *Query* - деякий запит, який буде виконано в майбутньому під час безпосереднього отримання даних. Далі застосовуючи до об'єкта *Query* різні методи, ми можемо отримати безпосередній результат. Наприклад, якщо треба отримати всі об'єкти, застосовується метод *all()*:

```
people = db.query(Person).all()
```

Метод *all()* повертає список об'єктів моделі. Повний код:

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from sqlalchemy import Column, Integer, String

from fastapi import FastAPI

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})

Base = declarative_base()
class Person(Base):
```

```

__tablename__ = "people"

id = Column(Integer, primary_key=True, index=True)
name = Column(String)
age = Column(Integer,)

SessionLocal = sessionmaker(autoflush=False, bind=engine)
db = SessionLocal()

# отримання всіх об'єктів
people = db.query(Person).all()
for p in people:
    print(f"{p.id}.{p.name} ({p.age})")

# додаток, який нічого не робить
app = FastAPI()

```

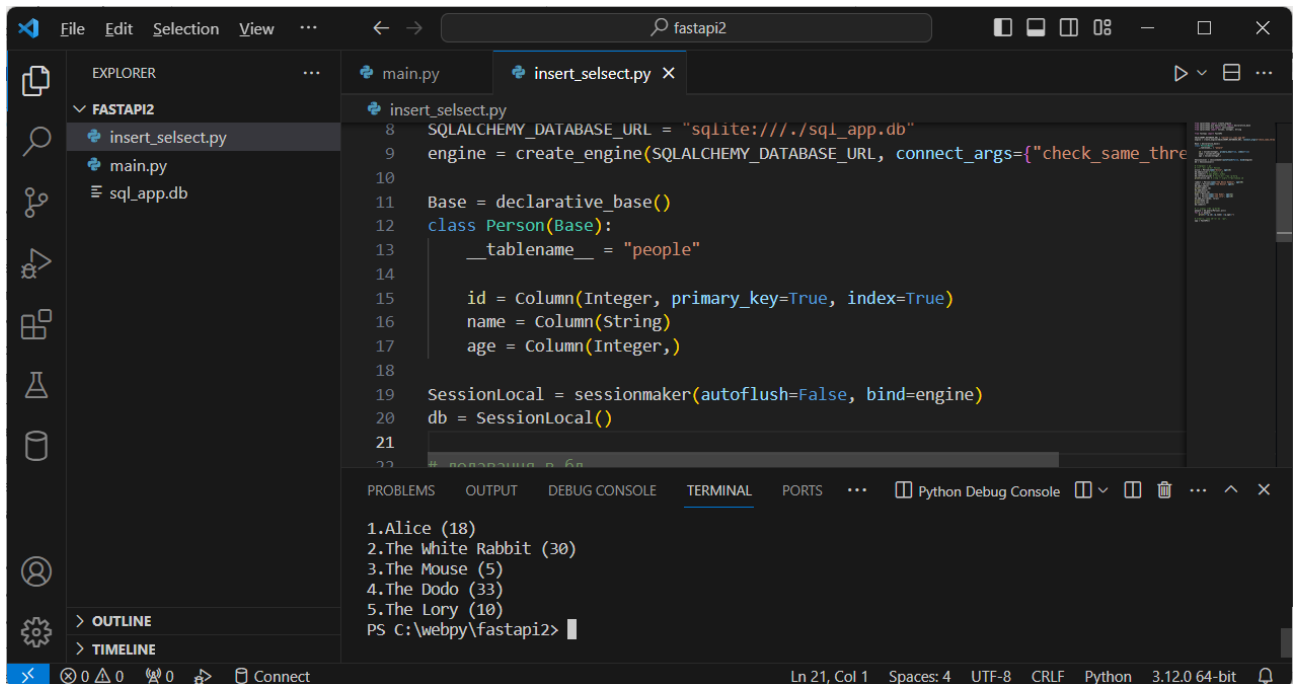


Рисунок 2.3– Отримання даних з бази даних через SQLAlchemy в FastAPI

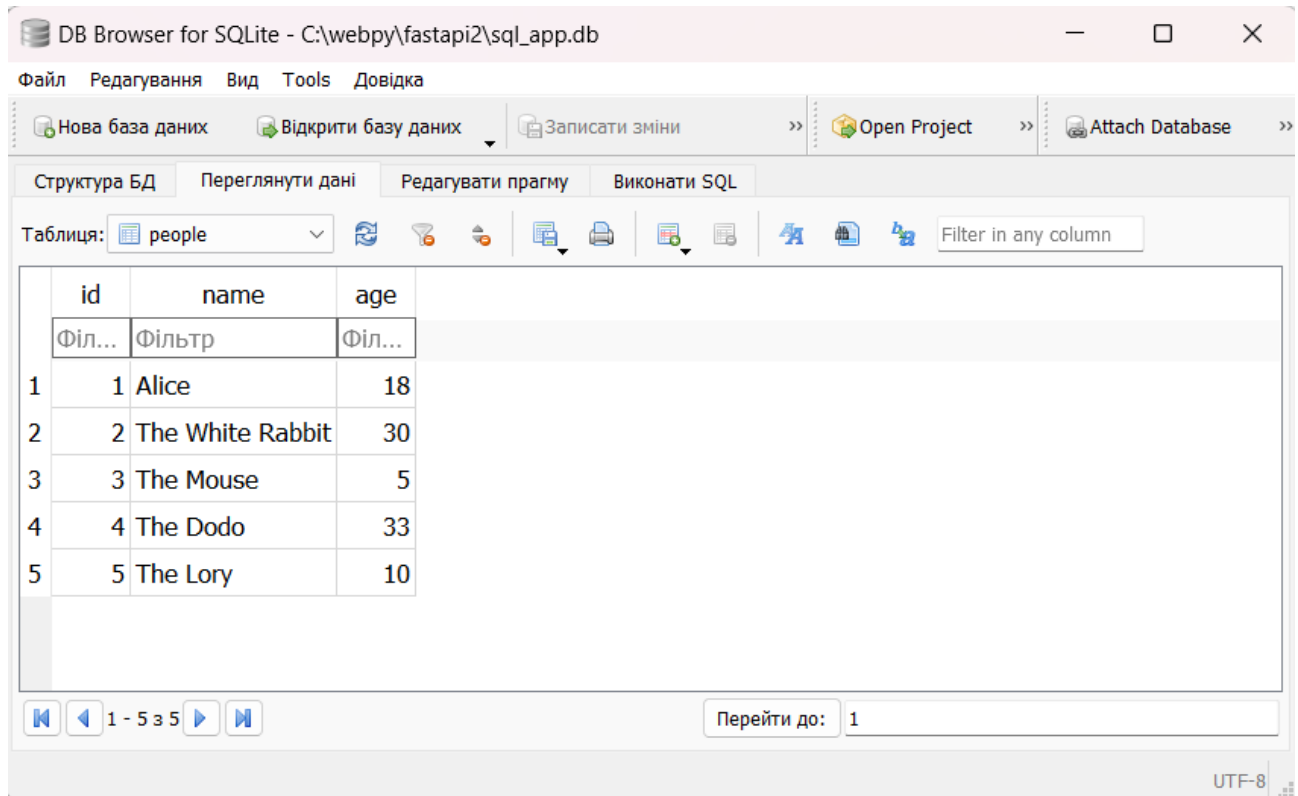


Рисунок 2.4 – Результат додавання даних

Для отримання одного об'єкта за *id* застосовується метод *get()* класу *Session*. Як параметри метод отримує тип моделі та ідентифікатор об'єкта, який треба отримати. Наприклад, отримаємо один об'єкт *Person*, у якого *id = 1*:

```
# отримання одного об'єкта за id
first_person = db.get(Person, 1)
print(f"{first_person.name} - {first_person.age}")
```

Для фільтрації в об'єкта *Query* застосовується метод *filter()*, який приймає умову фільтрації. Наприклад:

```
people = db.query(Person).filter(Person.age > 20).all()
for p in people:
    print(f"{p.id}. {p.name} ({p.age})")
```

Тут отримуємо всі об'єкти *Person*, у яких значення атрибута *age* більше 20. Метод *filter()* також повертає об'єкт *Query*, тому для отримання власне списку об'єктів, які відповідають фільтру, наприкінці по ланцюжку застосовується метод *all()*.

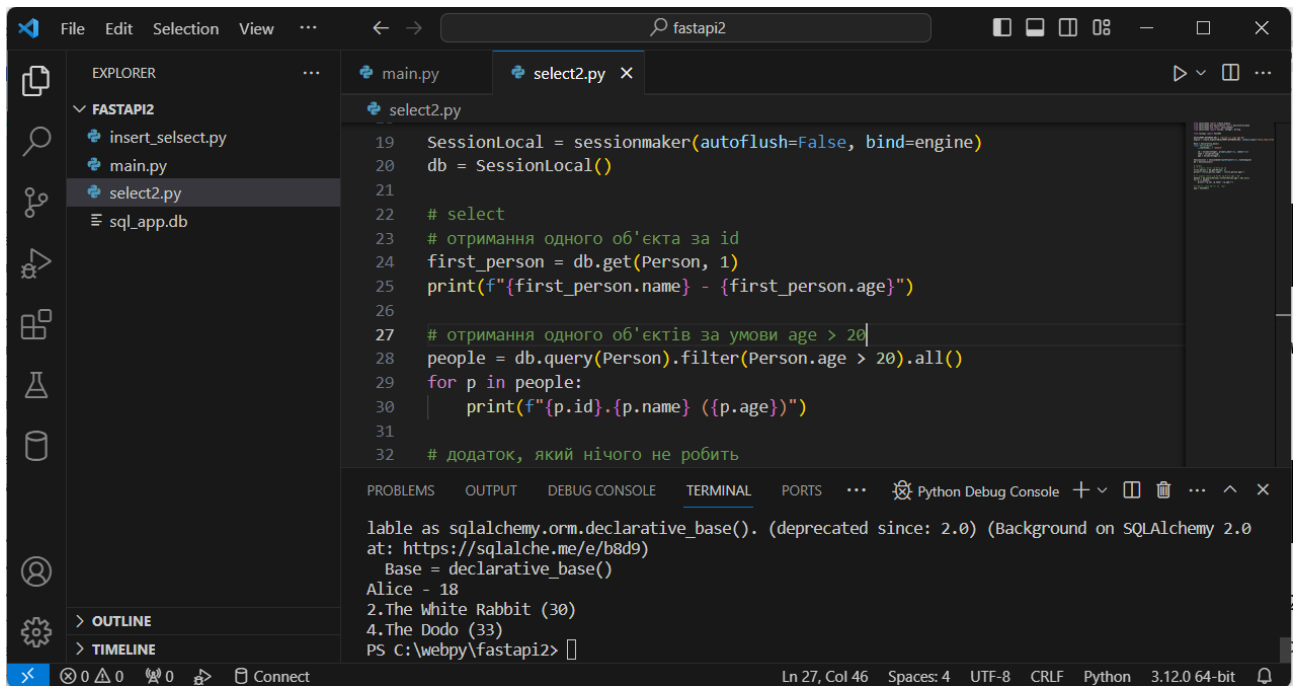


Рисунок 2.5 – Результат виборки даних

Для отримання тільки одного об'єкта застосовується метод *first()* класу *Query*:

```

first = db.query(Person).filter(Person.id==1).first()
print(f"{first.name} ({first.age})")

```

У цьому випадку отримуємо об'єкт *Person*, у якого *id = 1*.

Варто зазначити, що методи *get()* і *first()* повертають *None*, якщо об'єкт не знайдено. Тому при отриманні одиночного об'єкта бажано перевіряти його на значення *None*.

Оновлення

Для оновлення об'єкта достатньо змінити значення його атрибутів і потім викликати в об'єкта *Session* метод *commit()* для застосування змін:

```
# отримуємо один об'єкт, у якого ім'я - Alice
alice = db.query(Person).filter(Person.id==1).first()
print(f"{ alice.id}.{ alice.name} ({alice.age})")

# змінюю значення
alice.name = "ALICE"
alice.age +=1

db.commit() # зберігаємо зміни
```

перевіряємо, що зміни застосовані в бд - отримуємо один об'єкт, у якого id =1

```
alice = db.query(Person).filter(Person.id == 1).first()
print(f"{ alice.id}.{ alice.name} ({alice.age})")
```

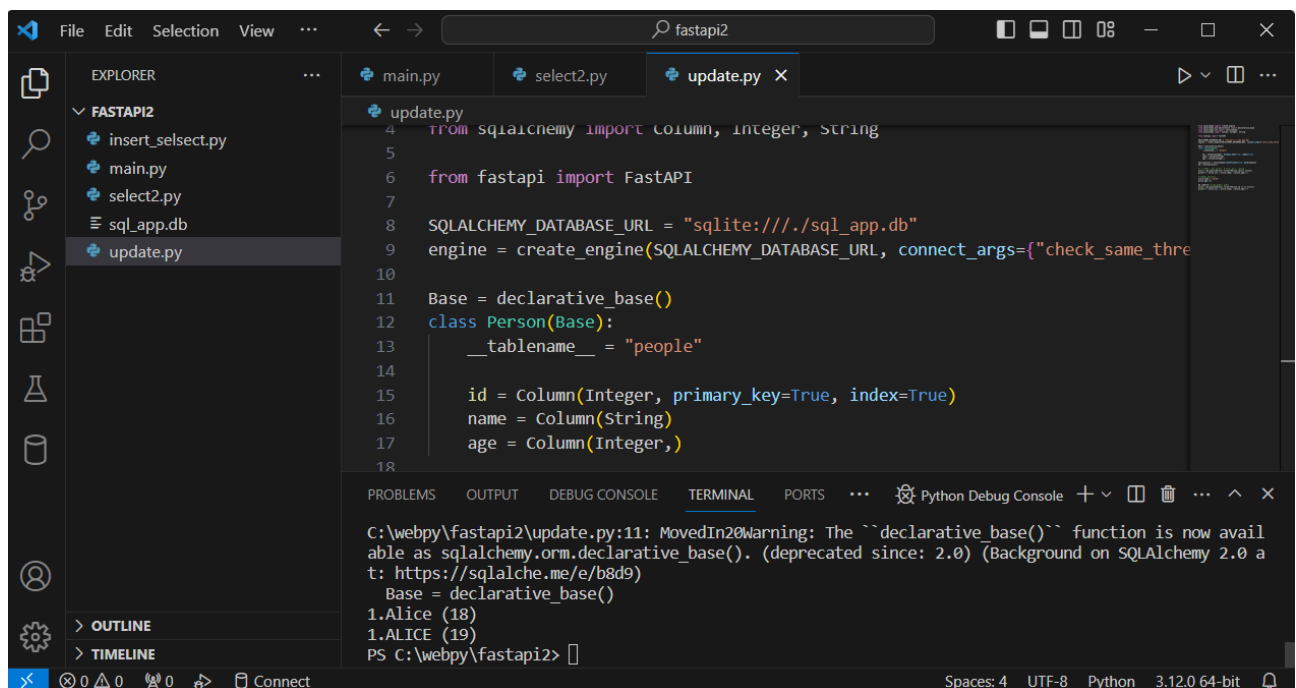


Рисунок 2.6 – Результат оновлення даних

Видалення

Для видалення в об'єкта *Session* застосовується метод *delete()*, у який передається об'єкт, що видаляється:

```
mouse = db.query(Person).filter(Person.id==3).first()
db.delete(mouse) # видаляємо об'єкт
db.commit() # зберігаємо зміни
```

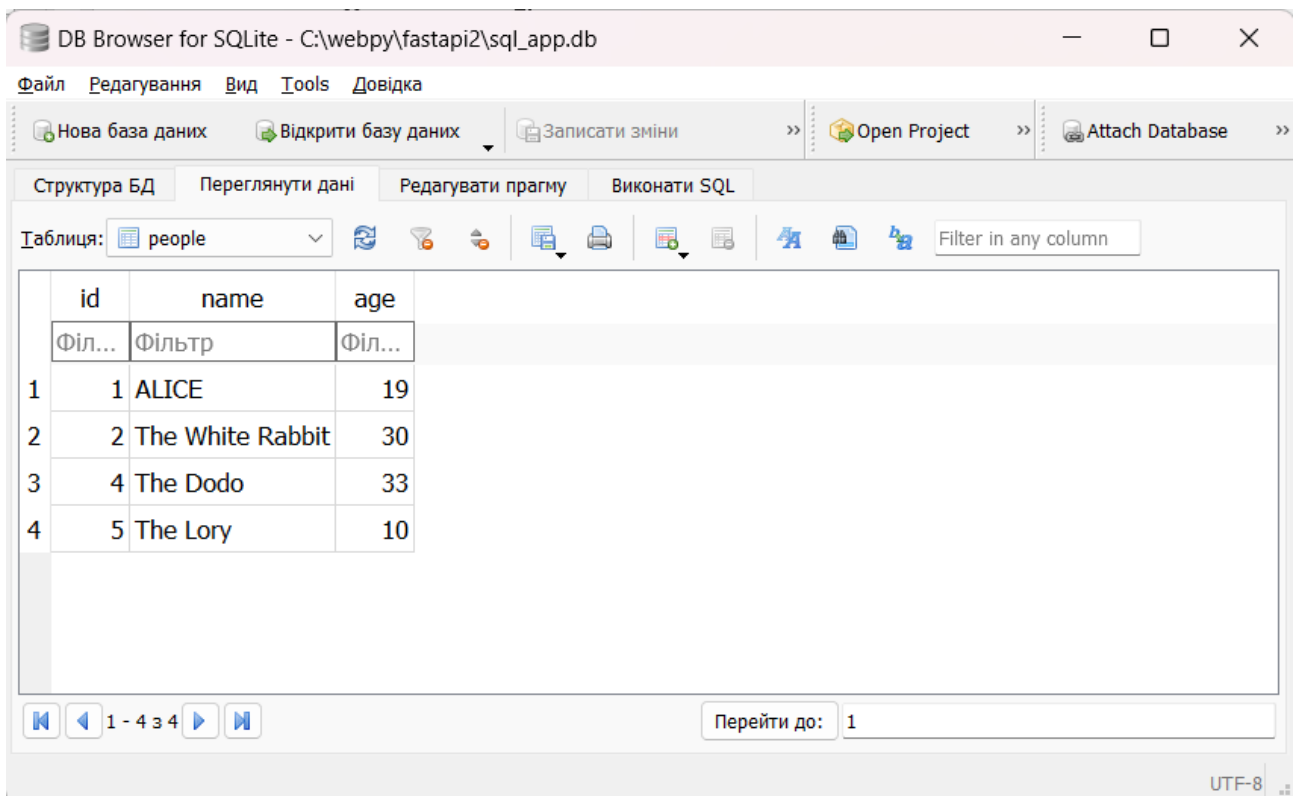


Рисунок 2.7 – Результат видалення даних

2.3 Базові операції з базою даних у веб-додатку

У минулому пункті було розглянуто базові операції з базою даних. Розглянемо, як це буде виглядати в рамках веб-додатка на найпростішому прикладі.

Визначення підключення до бази даних і моделей

Для зберігання всієї інфраструктури для роботи з базою даних у папці проєкту визначимо файл *database.py* з таким кодом:

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from sqlalchemy import Column, Integer, String

from fastapi import FastAPI

SQLALCHEMY_DATABASE_URL = "sqlite:///./sql_app.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread":
False})

Base = declarative_base()
class Person(Base):
    __tablename__ = "people"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    age = Column(Integer,)

SessionLocal = sessionmaker(autoflush=False, bind=engine)
```

Отже, у цьому разі ми будемо підключатися до бази даних SQLite, яка створюватиметься в папці проєкту. Збережені в БД дані будуть представлені класом *Person* з трьома атрибутами: *id*, *name* і *age*.

Створення сервера

У файлі *main.py* визначимо такий код:

```
from database import *
from sqlalchemy.orm import Session
from fastapi import Depends, FastAPI, Body
from fastapi.responses import JSONResponse, FileResponse
```

```

# створюємо таблиці
Base.metadata.create_all(bind=engine)

app = FastAPI()

# визначаємо залежність
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/")
def main():
    return FileResponse("public/index.html")

@app.get("/api/users")
def get_people(db: Session = Depends(get_db)):
    return db.query(Person).all()

@app.get("/api/users/{id}")
def get_person(id, db: Session = Depends(get_db)):
    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == id).first()
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person==None:
        return JSONResponse(status_code=404, content={ "message": "Користувач не знайдений"})
    #якщо користувача знайдено, відправляємо його
    return person

@app.post("/api/users")
def create_person(data = Body(), db: Session = Depends(get_db)):
    person = Person(name=data["name"], age=data["age"])
    db.add(person)
    db.commit()
    db.refresh(person)
    return person

@app.put("/api/users")
def edit_person(data = Body(), db: Session = Depends(get_db)):

    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == data["id"]).first()
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse(status_code=404, content={ "message": "Користувач не знайдений"})
    # якщо користувач знайдений, змінюємо його дані і відправляємо назад клієнту
    person.age = data["age"]
    person.name = data["name"]

```

```

    db.commit() # зберігаємо зміни
    db.refresh(person)
    return person

@app.delete("/api/users/{id}")
def delete_person(id, db: Session = Depends(get_db)):
    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == id).first()

    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse( status_code=404, content={ "message": "Користувач не знайдений" })

    # якщо користувача знайдено, видаляємо його
    db.delete(person) # видаляємо об'єкт
    db.commit()      # зберігаємо зміни
    return person

```

Розберемо в загальних рисах цей код. Насамперед спочатку створюємо всі необхідні таблиці в базі даних (якщо вони відсутні)

```
Base.metadata.create_all(bind=engine)
```

Кращим способом передачі сесії бази даних у функції обробки запиту є впровадження сесії БД як залежності. Тому далі створюємо функцію *get_db()*, через яку об'єкт сесії бази даних буде передаватися у функцію обробки:

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

Тут спочатку створюємо об'єкт сесії бази даних. Потім у конструкції *try..finally* за допомогою оператора *yield* повертаємо створений об'єкт. Таким чином, цей об'єкт буде впроваджено у функцію обробки запиту. Вираз *yield* буде виконуватися при отриманні кожного нового запиту.

Поміщення виразу *yield* у блок *try* дає змогу отримати й обробити будь-яку помилку, що виникла в процесі взаємодії з базою даних. Після завершення операцій з базою даних виконується блок *finally*, в якому закривається підключення до бази даних за допомогою методу *close()*.

Під час звернення до кореня веб-додатка, тобто за шляхом *"/*, він надсилатиме у відповідь файл *index.html*, тобто веб-сторінку, за допомогою якої користувач зможе взаємодіяти із сервером:

```
@app.get("/")
def main():
    return FileResponse("public/index.html")
```

Далі визначаються функції, які власне і представляють API. Спочатку визначається функція, яка обробляє запит типу *GET* на шляху *"api/users"*:

```
@app.get("/api/users")
def get_people(db: Session = Depends(get_db)):
    return db.query(Person).all()
```

За допомогою класу *Depends()* у функцію передається результат функції *get_db()*, тобто сесія бази даних, який передається параметру *db*. І через цей параметр ми зможемо взаємодіяти з базою даних. Подібним чином сесія бази даних впроваджується в усі інші функції-обробники запитів.

Запит *GET* передбачає отримання об'єктів, і в цьому випадку за допомогою методу *db.query(Person).all()* отримуємо список об'єктів *Person*, який надсилається клієнту. Коли клієнт звертається до додатка для отримання одного об'єкта за *id* у запит типу *GET* за адресою *"api/users/{id}"*, то спрацює інша функція:

```
@app.get("/api/users/{id}")
def get_person(id, db: Session = Depends(get_db)):
    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == id).first()
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person==None:
```

```

        return JSONResponse(status_code=404, content={ "message": "Користувач не
знайдений"})
    #якщо користувача знайдено, відправляємо його
    return person

```

Тут через параметр *id* отримуємо зі шляху запиту ідентифікатор об'єкта *Person* і за цим ідентифікатором шукаємо потрібний об'єкт у базі даних. Якщо об'єкт за *id* не було знайдено, то повертаємо за допомогою класу *JSONResponse* статусний код *404* з деяким повідомленням у форматі *JSON*. Якщо об'єкт знайдено, то відправляємо знайдений об'єкт клієнту.

При отриманні запиту типу *DELETE* за маршрутом *"/api/users/{id}"* спрацьовує інша функція:

```

@app.delete("/api/users/{id}")
def delete_person(id, db: Session = Depends(get_db)):
    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == id).first()

    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse( status_code=404, content={ "message": "Користувач не
знайдений"})

    # якщо користувача знайдено, видаляємо його
    db.delete(person) # видаляємо об'єкт
    db.commit()      # зберігаємо зміни
    return person

```

Тут діє аналогічна логіка - якщо об'єкт за *id* не знайдено, відправляє статусний код *404*. Якщо ж об'єкт знайдено, то видаляємо його з бази даних і надсилаємо клієнту.

При отриманні запиту з методом *POST* за адресою *"/api/users"* спрацьовує така функція:

```

@app.post("/api/users")
def create_person(data = Body(), db: Session = Depends(get_db)):
    person = Person(name=data["name"], age=data["age"])
    db.add(person)
    db.commit()
    db.refresh(person)
    return person

```

Запит типу *POST* передбачає передачу додатку даних, що надсилаються. Причому ми очікуємо, що клієнт надішле дані, які містять значення *name* і *age*. Для стислості опустимо валідацію вхідних даних. І для отримання даних з тіла запиту за допомогою класу *Body* отримуємо дані в параметр *data* і потім використовуємо дані з цього параметра для створення об'єкта *Person*. Потім створений об'єкт додаємо в базу даних, за допомогою методу *db.refresh()* оновлюємо його стан і відправляємо назад клієнту.

Якщо додатку надходить *PUT*-запит за адресою *"/api/users"*, то аналогічним чином отримуємо надіслані клієнтом дані у вигляді об'єкта *Person* і намагаємося знайти подібний об'єкт у базі даних. Якщо об'єкт не знайдено, відправляємо статусний код *404*. Якщо об'єкт знайдено, то змінюємо його дані, оновлюємо його стан і відправляємо назад клієнту:

```
@app.put("/api/users")
def edit_person(data = Body(), db: Session = Depends(get_db)):

    # отримуємо користувача за id
    person = db.query(Person).filter(Person.id == data["id"]).first()
    # якщо не знайдений, відправляємо статусний код і повідомлення про помилку
    if person == None:
        return JSONResponse(status_code=404, content={ "message": "Користувач не знайдений" })
    # якщо користувач знайдений, змінюємо його дані і відправляємо назад клієнту
    person.age = data["age"]
    person.name = data["name"]
    db.commit() # зберігаємо зміни
    db.refresh(person)
    return person
```

Таким чином, ми визначили найпростіший API. Тепер додамо код клієнта.

Визначення клієнта

Тепер у проєкті визначимо папку *public*, у яку додамо новий файл *index.html*

Визначимо у файлі *index.html* наступним код для взаємодії із сервером FastAPI:


```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>WebPython 2024</title>
</head>
<body>
  <h2>Список користувачів</h2>
  <div>
    <input type="hidden" id="userId" />
    <p>
      Ім'я:<br/>
      <input id="userName" />
    </p>
    <p>
      Вік:<br />
      <input id="userAge" type="number" />
    </p>
    <button id="saveBtn">Зберегти</button>
    <button id="resetBtn">Скинути</button>
  </div>
  <table>
    <thead><tr><th>Ім'я</th><th>Вік</th><th></th></tr></thead>
    <tbody>
    </tbody>
  </table>

  <script>
    // Отримання всіх користувачів
    async function getUsers() {
      // надсилає запит і отримуємо відповідь
      const response = await fetch("/api/users", {
        method: "GET",
        headers: { "Accept": "application/json" }
      });
      // якщо запит пройшов нормально
      if (response.ok === true) {
        // отримуємо дані
        const users = await response.json();
        const rows = document.querySelector("tbody");
        // додаємо отримані елементи в таблицю
        users.forEach(user => rows.append(row(user)));
      }
    }

    // Отримання одного користувача
    async function getUser(id) {
      const response = await fetch(`/api/users/${id}`, {

```

```

        method: "GET",
        headers: { "Accept": "application/json" }
    });
    if (response.ok === true) {
        const user = await response.json();
        document.getElementById("userId").value = user.id;
        document.getElementById("userName").value = user.name;
        document.getElementById("userAge").value = user.age;
    }
    else {
        // якщо сталася помилка, отримуємо повідомлення про помилку
        const error = await response.json();
        console.log(error.message); // і виводимо його на консоль
    }
}
// Додавання користувача
async function createUser(userName, userAge) {

    const response = await fetch("api/users", {
        method: "POST",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector("tbody").append(row(user));
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}
// Зміна користувача
async function editUser(userId, userName, userAge) {
    const response = await fetch("api/users", {
        method: "PUT",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            id: userId,
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector(`tr[data-
rowid='${user.id}']`).replaceWith(row(user));
    }
}

```

```

        else {
            const error = await response.json();
            console.log(error.message);
        }
    }
    // Видалення користувача
    async function deleteUser(id) {
        const response = await fetch(`/api/users/${id}`, {
            method: "DELETE",
            headers: { "Accept": "application/json" }
        });
        if (response.ok === true) {
            const user = await response.json();
            document.querySelector(`tr[data-rowid='${user.id}']`).remove();
        }
        else {
            const error = await response.json();
            console.log(error.message);
        }
    }
}

// скидання даних форми після відправлення
function reset() {
    document.getElementById("userId").value =
    document.getElementById("userName").value =
    document.getElementById("userAge").value = "";
}
// створення рядка для таблиці
function row(user) {

    const tr = document.createElement("tr");
    tr.setAttribute("data-rowid", user.id);

    const nameTd = document.createElement("td");
    nameTd.append(user.name);
    tr.append(nameTd);

    const ageTd = document.createElement("td");
    ageTd.append(user.age);
    tr.append(ageTd);

    const linksTd = document.createElement("td");

    const editLink = document.createElement("button");
    editLink.append("Змінити");
    editLink.addEventListener("click", async() => await getUser(user.id));
    linksTd.append(editLink);

    const removeLink = document.createElement("button");
    removeLink.append("Видалити");
    removeLink.addEventListener("click", async () => await deleteUser(user.id));

    linksTd.append(removeLink);

```

```

        tr.appendChild(linksTd);

        return tr;
    }
    // скидання значень форми
    document.getElementById("resetBtn").addEventListener("click", () => reset());

    // надсилання форми
    document.getElementById("saveBtn").addEventListener("click", async () => {

        const id = document.getElementById("userId").value;
        const name = document.getElementById("userName").value;
        const age = document.getElementById("userAge").value;
        if (id === "")
            await createUser(name, age);
        else
            await editUser(id, name, age);
        reset();
    });

    // завантаження користувачів
    getUsers();
</script>
</body>
</html>

```

Основна логіка тут міститься в коді *javascript*. Під час завантаження сторінки в браузері отримуємо всі об'єкти з БД за допомогою функції *getUsers()*:

```

async function getUsers() {
    // надсилає запит і отримуємо відповідь
    const response = await fetch("/api/users", {
        method: "GET",
        headers: { "Accept": "application/json" }
    });
    // якщо запит пройшов нормально
    if (response.ok === true) {
        // отримуємо дані
        const users = await response.json();
        const rows = document.querySelector("tbody");
        // додаємо отримані елементи в таблицю
        users.forEach(user => rows.append(row(user)));
    }
}

```

Для додавання рядків у таблицю використовується функція *row()*, яка повертає рядок. У цьому рядку будуть визначені посилання для зміни та видалення користувача.

Посилання для зміни користувача за допомогою функції *getUser()* отримує з сервера обраного користувача:

```
async function getUser(id) {
    const response = await fetch(`/api/users/${id}`, {
        method: "GET",
        headers: { "Accept": "application/json" }
    });
    if (response.ok === true) {
        const user = await response.json();
        document.getElementById("userId").value = user.id;
        document.getElementById("userName").value = user.name;
        document.getElementById("userAge").value = user.age;
    }
    else {
        // якщо сталася помилка, отримуємо повідомлення про помилку
        const error = await response.json();
        console.log(error.message); // і виводимо його на консоль
    }
}
```

І обраний користувач додається у форму над таблицею. Ця ж форма застосовується і для додавання об'єкта. За допомогою прихованого поля, яке зберігає *id* користувача, ми можемо дізнатися, яка дія виконується - додавання або редагування. Якщо *id* не встановлений (дорівнює порожньому рядку), то виконується функція *createUser()*, яка відправляє дані в *POST*-запиті:

```
async function createUser(userName, userAge) {
    const response = await fetch("api/users", {
        method: "POST",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector("tbody").append(row(user));
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}
```

```

    }
}

```

Якщо ж раніше користувач був завантажений на форму, і в прихованому полі зберігся його *id*, то виконується функція *editUser()*, яка відправляє *PUT*-запит:

```

async function editUser(userId, userName, userAge) {
    const response = await fetch("api/users", {
        method: "PUT",
        headers: { "Accept": "application/json", "Content-Type":
"application/json" },
        body: JSON.stringify({
            id: userId,
            name: userName,
            age: parseInt(userAge, 10)
        })
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector(`tr[data-
rowid='${user.id}']`).replaceWith(row(user));
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}

```

І функція *deleteUser()* посилає на сервер запит типу *DELETE* на видалення користувача, і в разі успішного видалення на сервері видаляє об'єкт за *id* зі списку об'єктів *Person*.

```

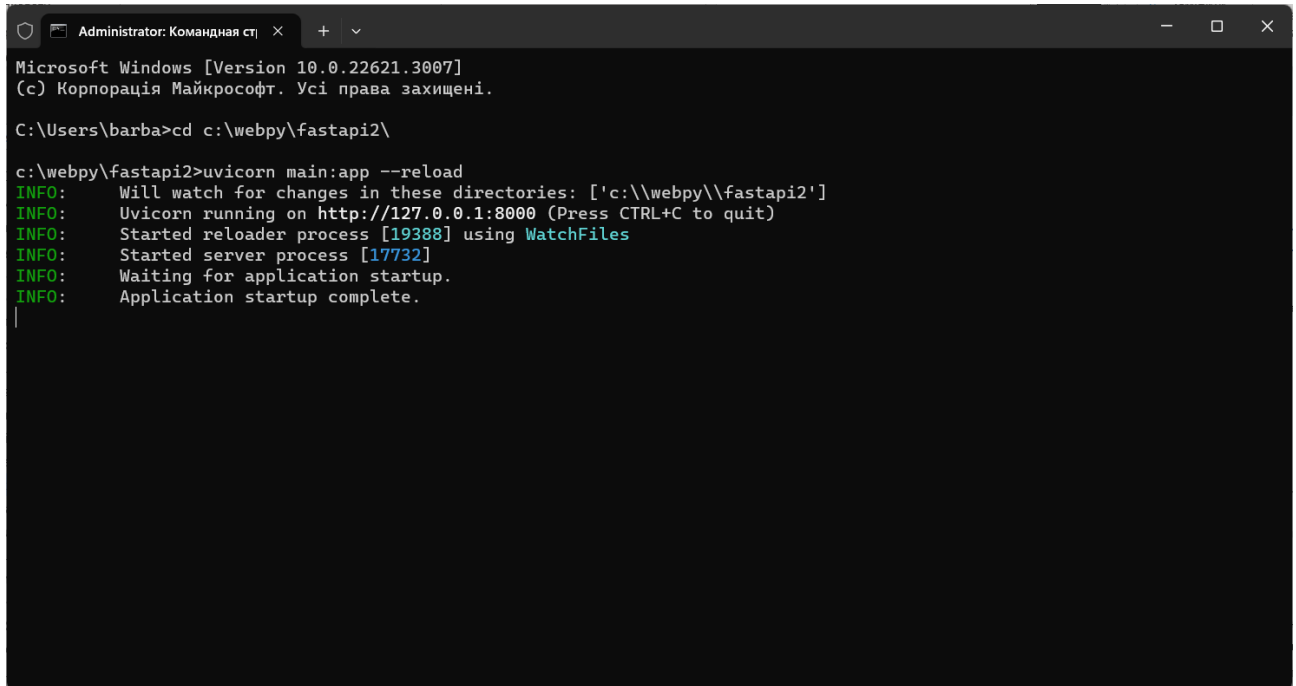
async function deleteUser(id) {
    const response = await fetch(`/api/users/${id}`, {
        method: "DELETE",
        headers: { "Accept": "application/json" }
    });
    if (response.ok === true) {
        const user = await response.json();
        document.querySelector(`tr[data-rowid='${user.id}']`).remove();
    }
    else {
        const error = await response.json();
        console.log(error.message);
    }
}

```

}

2.4 Запуск вебзастосунку

Перш за все треба безпосередньо запустити сервер:

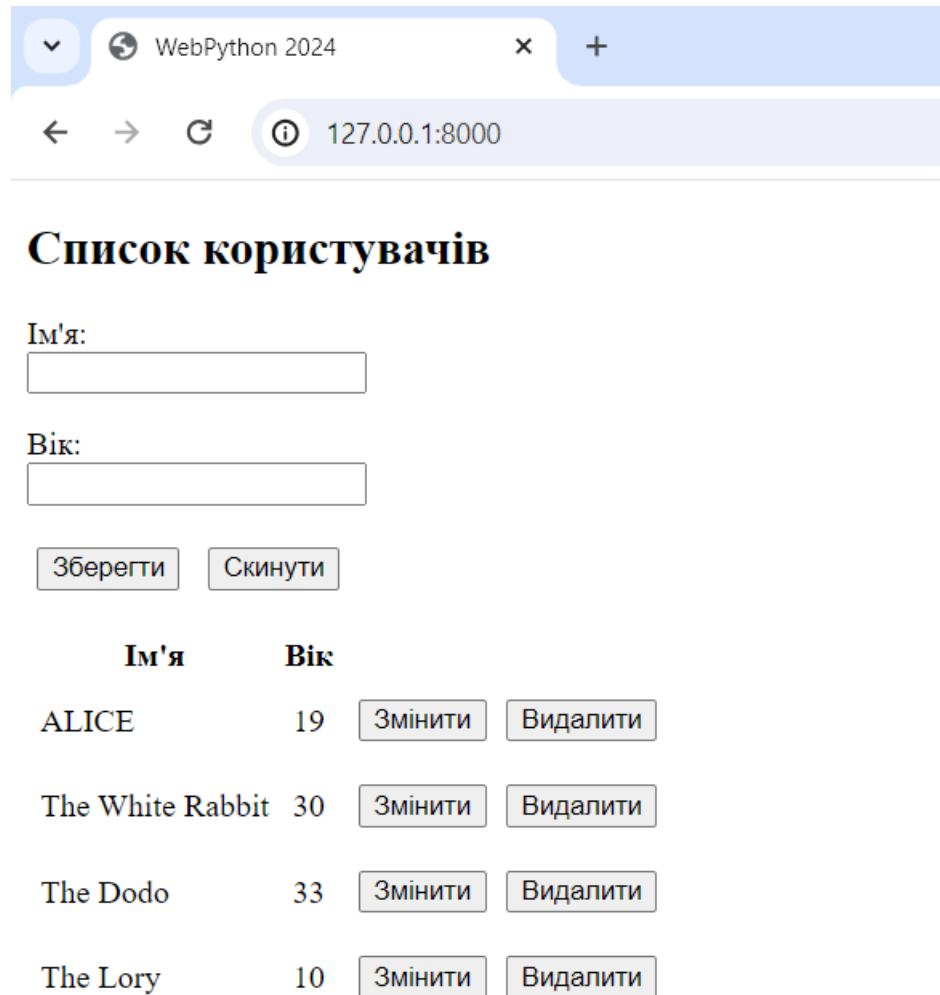


```
Administrator: Командная строка
Microsoft Windows [Version 10.0.22621.3007]
(c) Корпорация Майкрософт. Все права защищены.

C:\Users\barba>cd c:\webpy\fastapi2\

c:\webpy\fastapi2>uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['c:\\webpy\\fastapi2']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [19388] using WatchFiles
INFO: Started server process [17732]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Тепер запустимо проєкт, і за замовчуванням додаток надішле браузеру веб-сторінку *index.html*, яка завантажить список об'єктів:



WebPython 2024

127.0.0.1:8000

Список користувачів

Ім'я:

Вік:

Зберегти Скинути

Ім'я	Вік		
ALICE	19	Змінити	Видалити
The White Rabbit	30	Змінити	Видалити
The Dodo	33	Змінити	Видалити
The Lory	10	Змінити	Видалити

Рисунок 2.8 – Результат взаємодії javascript з Web API і базою даних SQLite через SQLAlchemy на FastAPI

Після цього ми зможемо виконувати всі базові операції з користувачами - отримання, додавання, зміна, видалення. Наприклад, додамо нового користувача:

WebPython 2024

127.0.0.1:8000

Список користувачів

Ім'я:

Вік:

Ім'я	Вік		
ALICE	19	Змінити	Видалити
The White Rabbit	30	Змінити	Видалити
The Dodo	33	Змінити	Видалити
The Lory	10	Змінити	Видалити

WebPython 2024

127.0.0.1:8000

Рисунок 2.9 – Результат додавання нового користувача

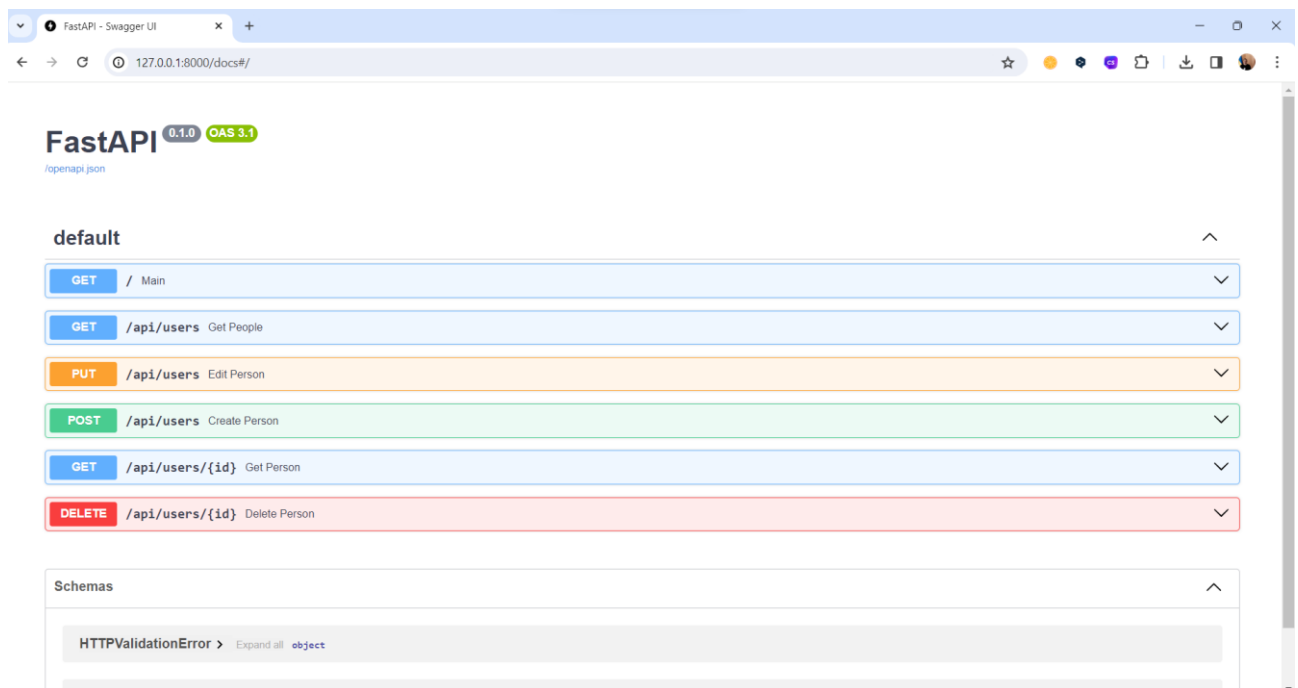


Рисунок 2.10 – Документація застосунка